Intenção em Código

Uma filosofia de engenharia de software com Kof

Mel Santos

Intenção em Código

Mel Santos · 1ª edição, outubro de 2026

Todos os exemplos de Kof deste livro foram verificados contra o compilador do repositório Kof4j (versão 0.6.1; CLI reportando 0.5.0-beta), com medições de 11 e 12 de outubro de 2026.

Sumário

Sumário

Prefácio....7
Introdução — uma tese, não um lema....9
A afirmação e o que ela exige....9
As distinções que o critério exige....10
Por que a linguagem importa para a filosofia....11
O que este livro não é....12
O argumento, em ordem....12
Capítulo 1 — Intenção em código....14
Onde o defeito realmente morava....14
Intenção não é sintaxe....15
A intenção que o compilador guarda....15
O custo de dizer....17
O programa como representação....18
Capítulo 2 — Complexidade essencial e acidental....19
A distinção, com nome e sobrenome....19
O teste de realidade....20
O erro oposto: essencial disfarçado de acidental....21
Acidental não é palavrão: a contabilidade....22
A contabilidade aplicada a mim mesma....22
O que a distinção deixa para o resto do livro....23
Capítulo 3 — O custo cognitivo do software....25
A assimetria que ninguém escolheu....25
Três custos que a palavra "complexo" esconde....26
Onde cada arquivo é limpo e o sistema não....26
O orçamento finito e os tributos invisíveis....27
O que uma linguagem pode cobrar, e pagar....28
A dívida que mora em pessoas....29
A fatura e o próximo capítulo....29
Capítulo 4 — Quando boas práticas viram burocracia....30
A regra é uma decisão comprimida....30
Três perguntas que separam a convenção viva da órfã....31
Clean Code: o que ficou, o que engessou....32
O custo da burocracia, nas duas moedas dos capítulos anteriores....32
Quando a regra mora na máquina....33
Convenções vivas têm manutenção....34
Fim da Parte I....34
Capítulo 5 — Sintaxe é uma decisão de engenharia....37
As três perguntas de toda escolha sintática....37
O caso do tipo que vem primeiro....38
val: uma palavra que compra uma garantia, e a entrega em duas formas....39
class X(...): a forma que mente por sotaque....40
A economia do que não existe....41
Onde a sintaxe encontra a semântica....42
Capítulo 6 — Tipos, nulabilidade e estados inválidos....43
O que o null sempre foi, disfarçado de valor....43
Kof coloca a ausência no tipo, e o tipo passa a valer o que diz....44

A letra miúda: onde a promessa não chega....44
Estados inválidos: o slogan e o que ele realmente pede....46
O que a linguagem não pode, e a arquitetura precisa....47
Próxima parada....47
Capítulo 7 — Abstração, composição e contratos....48
O critério, e o inquilino fantasma....48
Quando abstrair: o que existe do outro lado do nome....49
O contrato: o que a abstração promete, além da forma....50
Composição e herança: o que cada uma carrega....51
Reutilização real e o empréstimo da generalização....52
O que fica para o próximo capítulo....52
Capítulo 8 — O compilador como parte da experiência....54
O catálogo do que a linguagem realmente cobra....54
A mensagem de erro é um artefato de projeto....55
A janela para a máquina (e o que ver por ela)....56
O laço: check, run, test, e o que o laço não vê....56
Próxima parada....57
Capítulo 9 — Interoperabilidade e liberdade arquitetural....58
O que está provado na mesa....58
A frase proibida....59
Três máquinas, três constituições....60
O que a interoperabilidade não é....60
Fim da Parte II, o material e o que ele custou....61
Capítulo 10 — Modelar o domínio antes de modelar as classes....63
As três perguntas, e a alocação....63
O domínio vira estrutura, em Kof medido....64
O dia em que o material mentiu para o modelo....65
Os dois modos de errar....66
Capítulo 11 — Funções, dados e comportamento....68
O material: três casas para a mesma pergunta....68
Casa 1: dentro do estado....69
Casa 2: ao lado do estado....70
Casa 3: acima do estado....70
Anemia por escolha, anemia por abandono....71
O comportamento como valor, até onde medimos....71
Onde o capítulo deixa você....72
Capítulo 12 — Erros, falhas e estados inválidos....73
Erro: eu escrevi errado, e a máquina sabia....73
Falha: o mundo recusou, e isso estava no plano....74
O canal de Kof, com a contabilidade aberta....75
Estado inválido: a mentira que sobreviveu ao processo....75
As três categorias, os três canais, uma mesa....76
Capítulo 13 — Persistência, transações e integridade....77
A transação é um conceito do domínio, não do banco....77
Os dois erros espelhados....78
Integridade entre fronteiras: o mito do exactly-once....79
Defesa em profundidade não é redundância....79
O que a persistência confessa....80
Capítulo 14 — APIs e fronteiras de responsabilidade....81
A fronteira como tradutor de contratos....81
Versionar é dizer a verdade sobre o tempo....82
O contrato de erro é o contrato....83

Quem decide o quê..... 84
O que o material não entrega, e o capítulo não finge..... 84
Capítulo 15 — Concorrência, tempo e efeitos..... 85
A pergunta arquitetural, não a de biblioteca..... 85
O material medido: tarefas e fronteiras..... 86
O contrato que muda por target, e a tabela que o confessa..... 87
O tempo do mundo e o relógio do processo..... 88
Fim da Parte III..... 88
Capítulo 16 — Testar o comportamento, não a aparência..... 90
A pergunta que cada teste responde..... 90
O nome do teste é a especificação..... 91
A economia: juro, aluguel e a ordem vermelha..... 91
A escada de evidência e a falácia do degrau errado..... 92
O teste como intenção em código..... 93
Capítulo 17 — Performance sem superstição..... 94
A pergunta antes do número..... 94
O que este livro pode afirmar com número na mão, e o que não pode..... 95
O método, na ordem..... 96
A economia de quem paga..... 96
O fantasma da otimização prematura, exorcizado sem a frase..... 97
Fechando com a régua e o susto..... 97
Capítulo 18 — Arquitetura como distribuição de complexidade..... 98
A lei de conservação, em escala de sistema..... 98
O critério dos módulos, sem a citação..... 99
As fronteiras cobram pedágio, e este livro tem recibos..... 99
O debate monolito × serviços, devolvido à pergunta certa..... 100
A aposta no tempo, e as costuras..... 101
Capítulo 19 — Manutenção, compatibilidade e evolução..... 102
Quem paga a mudança, a política antes da técnica..... 102
Depreciação é disciplina, não aviso..... 103
Refatorar é mover, e evidência é o piso..... 104
Legibilidade é ativo de balanço..... 104
Evolução é deriva de distribuição..... 105
Capítulo 20 — Segurança como propriedade do projeto..... 106
O que a linguagem tira da mesa, e só da mesa..... 106
A stdlib medida, e o que "medida" quer dizer em segurança..... 107
Distribuição de confiança: a segurança do capítulo 18..... 108
O tempo ataca de graça..... 108
O livro aplica o critério a si mesmo..... 109
Capítulo 21 — Programar com inteligência artificial..... 111
O gargalo mudou de lugar..... 111
Código gerado é código sem memória..... 111
A função que compila, roda e mente..... 112
A revisão muda de foco..... 112
O deslocamento da atenção..... 113
O próximo passo..... 113
Capítulo 22 — O desenvolvedor como responsável pelo sistema..... 114
A posição que detém o "não passa"..... 114
Os quatro deveres da posição..... 115
A delegação não transfere a posição..... 115
O incidente é a unidade de responsabilidade..... 116
O caso do próprio livro, sem romantismo..... 116

Capítulo 23 — Construir linguagens é construir possibilidades.....118
O manifesto confrontado com o compilador.....118
Construir em aberto.....119
Toda equipe legisla sua própria linguagem.....119
Capítulo 24 — Uma engenharia orientada à intenção.....121
O argumento, uma vez, inteiro.....121
Na segunda-feira: o caderno, não o checklist.....122
O que isto não é.....123
A aposta final.....124
Posfácio.....125
Glossário.....127

Prefácio

Todo livro técnico começa com uma promessa implícita: a de que o tempo gasto na leitura se pagará em decisões melhores. Prefiro dizer desde o início qual é a promessa deste livro, e qual ele não faz.

Este livro não promete um conjunto de regras que, aplicadas mecanicamente, produzam software de qualidade. Isso seria uma mercadoria que já existe aos milhares, quase sempre com o mesmo resultado: times seguindo convenções que ninguém sabe justificar. A promessa aqui é outra: oferecer critérios para avaliar regras, inclusive as que este livro apresentar.

A tese que organiza a obra pode ser dita em uma frase, mas uma frase não é um argumento: código de qualidade é código cuja estrutura expressa a intenção do sistema com o mínimo de complexidade acidental possível. Ao longo dos próximos capítulos, essa afirmação será desmontada, testada contra exemplos, confrontada com escolas consagradas de engenharia e aplicada até a linguagem que eu mesma criei. Se ela sobreviver, não será por repetição, e sim porque teve o trabalho de explicar o que custa e onde falha.

Por que escrever este livro agora? A engenharia de software tem um problema antigo disfarçado de novidade: produzimos cada vez mais código com ferramentas cada vez mais expressivas, e continuamos incapazes de explicar, com precisão, por que um sistema é melhor que outro. As respostas disponíveis oscilam entre o gosto pessoal ("código limpo é o que eu leria de novo") e a autoridade ("é assim porque o livro X diz"). Nenhuma das duas sustenta uma decisão que precisará ser defendida em uma revisão de código daqui a dois anos, por alguém que não conhece o contexto de hoje.

O Kof aparece neste livro como laboratório, não como vitrine. É uma linguagem em desenvolvimento aberto, com decisões documentadas que podem ser lidas, executadas e contestadas. Quando o Kof expressa bem um comportamento, o capítulo mostra o código e o resultado medido; quando ele tem limites, o capítulo diz quais são. Essa exigência de honestidade seria hipocrisia editorial se o livro vivesse só de criticar os outros. A credibilidade de uma filosofia de engenharia depende de ela se aplicar à ferramenta de quem a defende, inclusive quando dói.

Uma palavra sobre os exemplos. Todo trecho de Kof deste livro foi executado contra o compilador real do repositório, e a versão utilizada, o backend e o resultado estão registrados nas notas de validação. Onde um exemplo ainda não foi validado, isso está dito no próprio texto, e não escondido em uma nota de rodapé que ninguém lê. Um livro sobre intenção que escondesse suas incertezas trairia a própria tese.

A obra foi planejada para publicação aberta: o texto-fonte ficará em um repositório público, com distribuição gratuita, e a publicação na Amazon está planejada como um desdobramento dessa mesma intenção. A licença definitiva do texto será decidida pela autora antes da publicação, e as opções e seus efeitos práticos estão discutidos nas notas editoriais, não presuma uma licença que ainda não foi escolhida.

Como ler este livro. A Parte I estabelece o problema e o vocabulário: intenção, complexidade e o custo real de compreender um sistema. A Parte II trata da linguagem como instrumento, tipos, abstração, compilador, interoperabilidade. A Parte III aplica os critérios a situações concretas de projeto: domínio, erros, persistência, APIs, concorrência. A Parte IV alarga o foco para além do arquivo: testes, desempenho, arquitetura, evolução e segurança. A Parte V fecha o argumento onde a engenharia de software hoje está de fato: um mundo em que máquinas ajudam a escrever código e a responsabilidade continua humana. Quem tem pressa pode começar pela Parte III; quem quer entender por que ela existe deve começar pela primeira. Os capítulos constroem um argumento cumulativo, lidos fora de ordem, ainda funcionam, mas algumas escolhas parecerão afirmações soltas.

O que se pede do leitor é o contrário do que um livro de regras costuma pedir: discordância informada. Se ao final você rejeitar uma conclusão deste livro com o mesmo rigor que ela exigiu para ser formulada, a obra terá cumprido seu papel. O objetivo nunca foi produzir discípulos de um método. Foi ajudar a construir critérios próprios, e critérios próprios são a única coisa que sobrevive à próxima moda de engenharia.

— Mel Santos

Introdução — uma tese, não um lema

Em alguma revisão de código, você já viu os dois lados desta briga. De um, alguém aponta uma função de trezentas linhas e pede que ela seja quebrada "porque função grande é ruim". Do outro, alguém quebra uma função coesa em cinco métodos auxiliares com nomes genéricos: `validar`, `processar`, `executar`, `preparar`, `finalizar`, e o revisor aprova satisfeito, porque o código agora "segue as boas práticas". As duas pessoas citaram o mesmo livro. As duas destruíram algo diferente: a primeira ignorou que a função longa podia ser perfeitamente legível; a segunda não percebeu que fragmentou um raciocínio único em uma coleção de fragmentos que agora precisam ser remontados na cabeça de quem ler.

O que faltou nos dois casos não foi regra. Faltou um critério para dizer o que a estrutura estava comunicando e o que ela estava custando. É esse critério que este livro tenta formular.

A afirmação e o que ela exige

A tese pode ser dita inteira:

> Código de qualidade é código cuja estrutura expressa a intenção do sistema com o mínimo de complexidade acidental possível.

Dita assim, ela soa como lema de camiseta, e lemas não argumentam. Vale então desmontar as três peças, porque cada uma carrega trabalho:

Intenção do sistema, não a intenção do autor documentada em comentário, nem a intenção do leitor no momento em que abre o arquivo, mas aquilo que o sistema precisa fazer, expresso na estrutura do código de um modo que outra pessoa possa recuperar sem adivinhação. Quando digo que um trecho "expressa intenção", estou fazendo uma afirmação verificável: alguém que conheça o problema consegue, lendo a estrutura, dizer o que o sistema faz e por quê, e consegue discordar de mim se não conseguir.

Complexidade acidental, o termo vem de Fred Brooks, no ensaio "No Silver Bullet" (1987), e a distinção que ele fez continua sendo a ferramenta mais subestimada da engenharia de software. Complexidade essencial é a do problema em si: um sistema de pagamentos tem Estados que não podem ser eliminados porque a realidade da liquidação os impõe. Complexidade acidental é a que nossas ferramentas, métodos e escolhas adicionam ao problema: o adaptador que existe só porque outra camada de adaptador existia, o framework que exige três interfaces para uma operação que uma função resolveria, a convenção que multiplicou arquivos. Brooks escreveu que essa distinção é antiga, vem de Aristóteles, passando pela filosofia da ciência, mas que a engenharia de software a trata como se fosse novo vocabulário. Eu a trato como vocabulário obrigatório.

Mínimo possível, a parte que a maioria das leituras apressadas esquece. Não é "o mínimo de complexidade"; é o mínimo de complexidade *acidental*, e "possível" tem peso: há custos que valem a pena, e o livro vai

discutir quais. Redundância em um sistema de controle de versões de banco de dados, validação duplicada entre aplicação e banco, abstração que só existe para preparar um futuro que talvez nunca chegue, cada uma dessas escolhas adiciona complexidade, e algumas são defensáveis mesmo quando o problema não a exigiria, porque o preço de não tê-las é maior.

Montadas, as três peças viram uma pergunta que cabe em qualquer revisão de código, em qualquer linguagem: **o que esta estrutura diz sobre o sistema, e o que ela custa para dizer?** A primeira metade mede intenção; a segunda, complexidade acidental. As respostas não vêm de regras universais, vêm de análise do problema, do contexto e dos custos. E é por isso que este livro não é, apesar do título, uma nova lista de mandamentos.

As distinções que o critério exige

A tese só opera com uma série de distinções que a conversa sobre qualidade de código costuma atropelar. Elas voltam em todos os capítulos, então vale nomeá-las aqui:

Intenção não é sintaxe. Duas linguagens podem expressar a mesma intenção com aparências muito diferentes, e uma estrutura elegante pode comunicar algo completamente errado. sintaxe é o meio; a pergunta editorial do código é o que ele afirma.

Concisão não é legibilidade. Um one-liner encadeando sete operações pode economizar onze linhas e cobrar vinte minutos de quem depura às duas da manhã. A métrica certa nunca foi "linhas por função"; foi "esforço para recuperar a intenção", e esforço inclui navegação entre arquivos, não apenas leitura da tela atual.

Simplicidade não é superficialidade. Esconder um problema difícil atrás de uma interface amigável não o torna simples; só muda quem paga a conta quando ele aparece. Abstração útil é aquela que reduz mais complexidade acidental da que adiciona, e existem abstrações que falham nesse balanço ainda no primeiro dia.

Código localmente compreensível não é um sistema globalmente compreensível. Cada arquivo pode ser limpo; a arquitetura pode ser ilegível. Este livro trata qualidade como propriedade que existe no arquivo, no sistema e na operação, com critérios diferentes em cada escala.

Correção funcional não é qualidade operacional. Um sistema que passa em todos os testes pode ser impossível de manter, de evoluir ou de segurar em produção. Qualidade não termina no "funciona"; ela começa onde o funciona precisa sobreviver ao tempo.

Uma distinção merece destaque porque organiza boa parte do que vem: a diferença entre a *segurança que a linguagem oferece* e a *segurança que a arquitetura exige*. Nenhuma quantidade de tipos elimina a necessidade de modelar corretamente um domínio; nenhum tipo de sistema impede que alguém modele errado. E nenhuma arquitetura, por cuidadosa, recupera o que o compilador poderia ter verificado no lugar dela, com menos custo, para sempre, em toda revisão de código futura.

Por que a linguagem importa para a filosofia

Há uma tradição em que a linguagem de programação é tratada como detalhe de implementação: "muda o time, o problema de engenharia é o mesmo". Discordo, com respeito pela tradição. A linguagem é a peça mais próxima do código que se escreve, e o que ela torna barato, impossível ou invisível de expressar condiciona diretamente o que os projetistas conseguem pensar. Uma linguagem na qual `null` é um valor silencioso de qualquer tipo produz sistemas com validação defensiva pulverizada por todo lado. Uma linguagem que exige que a possibilidade de ausência seja declarada: `String?` no Kof, por exemplo, desloca a mesma questão para o compilador, onde ela é respondida uma vez, em vez de dez mil vezes no review:

```
String? find(Int id) {
    if (id == 1) { return "mel" }
    return null
}

main() {
    var s = find(1)
    if (s != null) { println(s.length) }   // narrowing → 3
}
```

Este é código Kof real: compilou e rodou, saída `3`, medido com o compilador do repositório em 11 de outubro de 2026. E repare que o exemplo já traz a lição editorial que este livro aplica a tudo: a assinatura `String?` torna a possibilidade de ausência parte do contrato, visível no ponto de chamada, e dentro dela `return null` é aceito; mas atribuir o literal `null` diretamente na declaração: `String? s = null`, o compilador recusa com o diagnóstico SEM048, uma assimetria que o capítulo sobre nulabilidade discute em detalhe. A sintaxe sugere uma garantia; a garantia que o compilador efetivamente oferece pode ser outra, e a diferença entre as duas é exatamente o tipo de coisa que um engenheiro precisa medir em vez de presumir. Sobre a própria linguagem que criei, os capítulos vão medir, com o mesmo rigor aplicado a qualquer decisão de escola alheia, o que a sintaxe sugere contra o que o backend entrega, inclusive porque os três targets (JVM, nativo e JavaScript) ainda têm diferenças documentadas de paridade, e a linguagem está na versão 0.6.1 do repositório, em desenvolvimento aberto, com parcela experimental declarada na própria documentação.

Escolhi o Kof como fio condutor por uma razão incômoda: é mais fácil pensar com clareza sobre uma linguagem que você construiu sabendo exatamente por que cada decisão foi tomada, e cada decisão está registrada como documento de design público, para o bem e para o mal. Quem só programa em linguagens alheias tende a atribuir aos deuses o que foi escolha de engineering de terça-feira com prazo na sexta.

O que este livro não é

Três negativas para evitar mal-entendidos antes do capítulo um.

Não é um manifesto contra Clean Code. Robert C. Martin colocou em circulação perguntas que milhões de programadores passaram a fazer: "como nomear?", "como decompor?", "o que este teste diz sobre o design?", e o valor dessas perguntas não desaparece porque as respostas viraram dogma. Este livro discute a diferença entre uma prática e uma regra, entre um critério e um mandamento. Essa discussão é devedora do Clean Code tanto quanto é crítica dele, e os capítulos vão dizer de qual lado.

Não é um manual de Kof. A referência da linguagem existe em outro lugar, este livro a usa como um corpo de evidências, não como objeto de propaganda. Se a tese se sustenta apenas em Kof, ela não se sustenta; ela precisa ser útil para quem mantém um monolito Rails, um backend Java, um frontend TypeScript ou um sistema embarcado em C.

Não é um livro de opinião. Posições técnicas abundam aqui, e são assumidas como posições, mas cada uma precisa vir com as premissas, o contexto em que se sustenta e o contraexemplo que a derrubaria, separados de fatos verificáveis, que por sua vez precisam de fonte. Frases quantificadas neste livro têm medição documentada ou são marcadas como hipótese. É o mínimo que se pode exigir de uma obra cujo critério central é "decisão justificável".

O argumento, em ordem

A Parte I mostra por que "o problema não é o código": qualidade é relação entre estrutura, intenção e custo, e sem esse vocabulário qualquer discussão sobre boas práticas vira disputa de gosto. A Parte II examina as linguagens como elas são, artefatos de engenharia cujas sintaxe, tipos e compilador empurram o pensamento em direções específicas. A Parte III desce com os critérios para o trabalho de projeto real: modelar domínio, tratar erro, persistir, publicar uma API, lidar com tempo e concorrência. A Parte IV amplia o foco para além do arquivo, testes, desempenho, arquitetura, evolução, segurança, porque um sistema pode ter bons arquivos e ser ruim, e o inverso também é possível. A Parte V fecha na época que habitamos: máquinas que geram código em volume, linguagens como instrumento de pensamento, e uma proposta de engenharia que tenta merecer o nome, orientada à intenção, e responsável pelo que constrói.

Se você chegar ao fim e concordar com tudo, este livro terá fracassado em algum ponto. O objetivo é chegar ao fim com critérios, e critérios se exercitam discordando, testando e escolhendo com os próprios motivos.

Parte I — O problema não é o código

Capítulo 1 — Intenção em código

O diff estava correto. Essa é a parte incômoda.

Um sistema de cobrança guardava em uma única função o percentual aplicado a dois casos diferentes: o desconto promocional de um pedido e o juro de mora quando a fatura atrasava. Ambos os valores vinham de `percentualAplicavel()`, que devolvia `15.0`. A campanha mudou, o desconto passou a 10%, e alguém alterou o retorno da função. O commit dizia "fix: desconto errado (corrigi)". Os testes passaram. A revisão de código aprovou em trinta segundos, a mudança era visivelmente correta para o problema que estava na mesa.

Duas semanas depois, o financeiro reportou que a inadimplência tinha caído... das receitas. O juro de mora também tinha virado 10%. A função compartilhada dizia uma coisa onde o sistema precisava dizer duas, e ninguém, nem o autor da mudança, nem o revisor, nem a suíte de testes, conseguia ver isso no código, porque o código não continha aquela informação em lugar nenhum. Não havia comentário esquecido, não havia nome enganoso. A estrutura simplesmente tinha fundido duas intenções em um único ponto de decisão, e o sistema carregava essa fusão em silêncio até o dia em que as duas intenções precisaram divergir.

Nenhum teste falhou porque nenhum teste verificava a separabilidade. Testes observam comportamento: dados os mesmos argumentos, o mesmo resultado. Mas as duas funções que *deveriam* existir: `descontoPromocional()` e `jurosMora()`, não têm comportamento a comparar enquanto a campanha não muda. O que estava errado não era o que o código fazia; era o que ele não dizia.

Onde o defeito realmente morava

Vale nomear com precisão o que aconteceu, porque a descrição habitual: "faltou cobertura de teste", "faltou revisão mais cuidadosa", leva direto à repetição do problema com mais burocracia. O sistema tinha duas regras de negócio com histórias independentes: uma acompanha decisões de marketing, a outra acompanha política de crédito e regulação. Elas coincidiam em valor por um tempo, e nada no código registrava que a coincidência era acidental. Quando duas coisas mudam por motivos diferentes, compartilhá-las é um empréstimo que cobra juros na primeira divergência.

O programa é uma representação de um problema. Essa afirmação parece abstrata demais até você precisar consertar algo às onze da noite, quando a pergunta útil é exatamente "o que este sistema *é*, e onde esta mudança *mexe*?". Se a resposta precisa ser montada a partir de memória institucional, de conversa com quem escreveu, ou de tentativa e erro em staging, a representação está incompleta. O código funciona, mas não é compreensível, e "funciona até alguém mexer" é uma definição frágil demais de funcionar para sistemas que vão durar mais que a atenção de quem os construiu.

Intenção, neste livro, não é aquilo que o autor quis dizer. É aquilo que a estrutura do código torna recuperável por quem não estava lá. A distinção importa porque as duas coisas divergem o tempo todo: você

pode querer separar desconto e juro e acabar escrevendo uma função compartilhada "só por enquanto"; a intenção psicológica existe, a intenção em código não. E o leitor que vem depois só tem acesso à segunda.

Intenção não é sintaxe

A confusão mais comum é tratar a expressão da intenção como um problema de vocabulário. Trocar `percentualAplicavel()` por `descontoPromocional()` resolve o caso do capítulo? Resolve: *neste* caso, porque a função era o único lugar onde a intenção morava. Mas nomes são apenas uma das camadas em que a intenção pode residir, e a mais barata delas. A pergunta séria é estrutural: **o que esta configuração de tipos, chamadas, dados e fronteiras afirma sobre o sistema, e o que ela deixa de afirmar?**

Sintaxe é o que se digita; intenção é o que a estrutura declara. As duas categorias se separam em ambos os sentidos. Um código pode ser sintaticamente impecável, nomes bons, linhas curtas, formatação limpa, e afirmar coisas erradas: a função bem-nomeada que lê um arquivo sem dizer que lê, o parâmetro `bool` que na verdade seleciona entre três comportamentos, o `null` que significa "não encontrado" em uma camada e "erro interno" em outra. E um código pode ser sintaticamente áspero e carregar intenções precisas: um módulo feio onde cada tipo de erro é um tipo de verdade, onde o que pode faltar está declarado e o que não pode simplesmente não aparece.

A intenção vive em três lugares, nesta ordem de força: no que a estrutura **impede**, no que ela **separa ou compartilha**, e só então no que ela **nomeia**. O primeiro nível é o mais valioso porque não depende de leitura atenta: o que o compilador recusa não precisa ser lembrado pela equipe. O segundo é onde a história de mudança do sistema fica registrada, o que foi separado em tipos, funções e módulos distintos pode divergir sem conflito; o que foi compartilhado diverge junto, como vimos. O terceiro é o mais frágil: nomes comunicam para quem já sabe procurar, e envelhecem em silêncio.

A intenção que o compilador guarda

O primeiro nível merece um exemplo real, porque é onde a tese deixa de ser atitude e vira técnica. Esta é a hierarquia de formas de um domínio qualquer, escrita em Kof, código que compilou e rodou, medido com o compilador do repositório em 11/10/2026:

```
sealed class Forma
class Circulo extends Forma {
    Double raio
    public constructor(Double raio) {
        this.raio = raio
    }
}
class Quadrado extends Forma {
    Double lado
    public constructor(Double lado) {
        this.lado = lado
    }
}

String d(Forma f) {
    return switch (f) {
        case Circulo c -> "círculo"
        case Quadrado q -> "quadrado"
    }
}
```

Repare que o `switch` não tem `default`. Isso não é descuido, é a intenção declarada. A palavra `sealed` afirma "estas são as únicas formas que existem", e o switch sem `default` afirma "este raciocínio cobre todas elas". O compilador fica de guarda nas duas pontas: se alguém acrescentar `class Triangulo extends Forma`, o switch acima para de compilar com o diagnóstico SEM081 (caso não coberto em hierarquia selada). A decisão de projeto, o conjunto de formas é fechado, sobrevive ao autor, à rotatividade da equipe e ao refactor apressado, sem depender de ninguém lembrar de atualizar o switch. No sistema do início do capítulo, não existia nada equivalente: a fusão entre desconto e juro não violaria nenhum tipo, nenhum teste, nenhuma verificação. Ela só cobraria a dívida quando as intenções divergissem.

O mesmo mecanismo aparece em versão mais simples com `record`. Em Kof, `record User(String name, String email)` declara dados por valor: os acessores e o `toString` vêm gerados, e escrever em um componente: `u.name = "outra"`, é recusado em tempo de compilação com SEM038 ("record is immutable"). A intenção "isto é um valor, não um objeto com história" passa a ser uma promessa verificável, não um comentário otimista. A classe comum, ao contrário, declara abertura: `Conta` mantém `saldo` mutável, `c.saldo = 50.0` é escrita direta, e o código diz "aqui há estado que muda". Cada escolha de estrutura é uma declaração sobre o domínio, e o preço de uma declaração errada aparece na primeira mudança que a contraria.

Agora o detalhe honesto, e ele não é um rodapé da argumentação: a própria documentação de referência da linguagem, o arquivo `classes.pt_BR.md` e o status da especificação, afirma que `sealed` foi removido do lexer e que usá-lo produziria erro de parse. O compilador instalado aceita `sealed` e aplica SEM081. Os dois não podem estar certos, e eu medi em vez de citar: o comportamento do compilador é o que descrevi acima. Este livro vai aplicar a si mesmo o critério que defende, a intenção declarada por uma autoridade (uma doc, um livro de regras, uma autora de linguagem) vale até o momento em que a realidade medida diz outra coisa.

Registrar a divergência é mais útil que fingi-la resolvida, e é também o argumento do capítulo aplicado sem piedade à ferramenta do capítulo.

O custo de dizer

Se intenção verificável é tão valiosa, a conclusão óbvia seria codificar tudo: cada regra de negócio um tipo, cada conjunto fechado `sealed`, cada valor imutável `record`. Eu não faria isso sem antes somar a conta, e o motivo é estrutural: **cada afirmação que a estrutura faz é uma afirmação que o sistema precisa manter verdadeira.** Um tipo para cada variação de campanha publicitária compila o marketing dentro da arquitetura; na próxima campanha, você não muda um valor, você muda um tipo, suas importações, seus testes de tipo e meia dúzia de switches. A intenção que ontem protegia uma divergência hoje é o peso morto de uma que nunca aconteceu.

A pergunta operacional que carrego para o resto do livro tem duas metades, e a segunda é a que as discussões de estilo costumam pular:

> O que esta estrutura diz sobre o sistema, e o que ela custa para dizer?

O "custo" não é metafórico. Toda decisão de expressão cobra em três moedas: esforço de compreensão de quem lê (a estrutura nova precisa ser aprendida), rigidez na próxima mudança (afirmações demais estreitam o espaço de manobra), e complexidade acidental, a cota que este livro persegue e que o próximo capítulo define com cuidado. Um `sealed` com dois casos pode ser a diferença entre um bug de cobrança e um erro de compilação; um `sealed` com dois casos e zero probabilidade de um terceiro é burocracia de tipos que alguém vai amaldiçoar na sexta refatoração. A diferença entre os dois não está na estrutura em si: está em o que o sistema realmente precisa que seja lembrado para sempre, e o que é coincidência desta versão.

Há também intenções que a linguagem simplesmente não deixa expressar, e isso tem custo mensurável. Kof não tem operador de navegação segura, o `?.` de outras linguagens é planejado e não existe; a única forma de estreitar um `String?` é o `if (s != null)` explícito. Para um serviço que trata muitos valores opcionais vindos de borda, isso é verbosidade real, linha a linha, para sempre. Para um sistema onde a maioria das ausências é erro estrutural, a mesma ausência do operador força cada leitura de opcional a *mostrar* a decisão no código, o que é uma intenção que fica mais difícil de esconder. Eu escolhi não implementar `?.` ainda por motivos que este capítulo não precisa detalhar (o capítulo sobre nulabilidade sim), mas o exemplo vale como princípio: nenhuma sintaxe é neutra; o que uma linguagem torna fácil de expressar será expressado com frequência, o que ela torna trabalhoso será expressado com parcimônia ou contornado, e o contorno quase sempre é invisível para quem não sabe que existe.

O programa como representação

Reunindo o que o capítulo moveu. O bug do percentual não estava em nenhuma linha: estava na ausência de uma linha, a segunda função que nunca existiu. Os testes não o pegaram porque descreviam comportamento presente, não intenção separável. A revisão não o pegou porque a mudança era localmente correta. E a correção definitiva não é "testar mais" nem "revisar melhor": é fazer o código carregar a informação de que desconto promocional e juro de mora são regras independentes que hoje coincidem, seja em dois pontos de decisão separados, seja em tipos que os backends de cobrança possam verificar.

Um programa é uma representação verificável de um problema. Verificável em dois sentidos que este livro não vai confundir: verificável *contra o comportamento*, que testes e execução entregam; e verificável *contra a intenção*, que a estrutura entrega, pelo que impede, pelo que separa, pelo que nomeia, necessariamente nessa ordem de confiança. Código de qualidade mantém as duas verificações abertas: faz o que o problema pede e diz por que faz, com o menor volume de afirmações que precisa continuar sendo verdade.

Ficou uma promessa no ar, e ela é o assunto do capítulo seguinte. Falei em "complexidade acidental" como a cota que cada decisão de expressão paga, e em "o que o problema exige" como o que ela sobra. Essa fronteira, entre a complexidade que o domínio impõe e a que nós mesmos fabricamos, é a distinção mais antiga e mais ignorada da engenharia de software. Sem ela, "mínimo de complexidade" é só um palpite elegante.

Capítulo 2 — Complexidade essencial e acidental

O capítulo anterior fechou com uma conta aberta: cada afirmação que a estrutura faz é uma afirmação que o sistema precisa continuar mantendo verdadeira, e eu chamei esse preço de *complexidade acidental* sem definir direito o que é a acidental, nem, portanto, o que é a outra. Hora de pagar a promessa com precisão, porque a distinção é a ferramenta mais antiga e mais mal citada da engenharia de software.

Começo pelo caso em que ela falha por ausência.

Um sistema de pagamentos ganhou, certa vez, uma função `estornar()`. A equipe queria "simplificar o modelo": em vez de espalhar o estorno por módulos, um comando só, um status que vira `ESTORNADO`, fim. O código tinha poucas linhas, poucos estados, poucas decisões, era, em qualquer leitura superficial, o pedaço mais simples do sistema.

Três meses depois, a empresa começou a oferecer estorno parcial, e o mundo revelou o que sempre tinha sido. Estornar não é o inverso de cobrar. Uma cobrança acontece num instante; um estorno acontece numa janela: depende de o lote de liquidação ter fechado, de a adquirente aceitar a devolução, do valor parcial ser autorizado antes de ser executado, de o imposto da nota original já ter sido recolhido, de o cliente poder contestar o estorno parcial como cobrança indevida do restante. Cada uma dessas condições é uma regra do *domínio*, não foi inventada por nenhum programador, nenhum framework, nenhum banco. O modelo de um status único não eliminou nenhuma delas; apenas as removeu do código, onde alguém poderia vê-las, e as transferiu para a operação, onde elas apareceram como fila de tickets, planilha de conciliação manual e, na melhor das hipóteses, um segundo sistema construído às pressas ao lado do primeiro para "cuidar dos casos que o modelo simples não alcança".

A complexidade não desapareceu. Ela mudou de endereço, e de dono, e de linguagem.

A distinção, com nome e sobrenome

Fred Brooks formulou a distinção que este livro usa como instrumento no ensaio "No Silver Bullet" (1987), depois incorporado às reedições de *The Mythical Man-Month*. O núcleo, parafraseado sem página: parte da dificuldade de construir software pertence ao **problema**: é a complexidade essencial, aquilo que o usuário faz, e o que o mundo exige, independentemente de como implementamos; o restante pertence à **representação**: é a complexidade acidental, criada pelas ferramentas, métodos e decisões com que escolhemos construir. A conclusão do ensaio é tão importante quanto a distinção, e na direção contrária ao otimismo que ele combateu: enquanto a maior parte da dificuldade de um sistema real for essencial, nenhuma tecnologia ou técnica individual, nenhuma "bala de prata", pode entregar uma melhoria de ordem de grandeza, porque pode apenas atacar a parte acidental.

Duas leituras erradas desse texto circulam há décadas, e vale descartar as duas antes de usar a distinção direito.

A primeira é usar "acidental" como xingamento: tudo que me dá trabalho é complexidade desnecessária, criada por terceiros. A distinção de Brooks não é um veredito moral sobre implementações; é uma contabilidade. Complexidade acidental não é necessariamente ruim, como veremos, quase todo ganho de engenharia é compra de complexidade acidental que elimina uma obrigação essencial de memória ou de disciplina. O erro não é pagar; é pagar sem saber o que comprou.

A segunda é usar "essencial" como absolvição: se é do domínio, não temos escolha. Temos, sim, e é exatamente aí que o trabalho de projeto acontece. A complexidade essencial não pode ser *eliminada*, mas pode ser *representada*: nomeada, separada, distribuída entre as camadas certas, tornada verificável. O estorno parcial não deixa de existir porque o modelo tinha um status só; mas um modelo que declare `EstornoParcial` com seus estados e janelas transforma uma armadilha invisível em uma decisão visível. A diferença entre o sistema de um status e o sistema que nomeia as janelas de liquidação não é menos complexidade, é complexidade **no lugar onde alguém pode enxergá-la e tratá-la**. É a tese do capítulo anterior vista pela outra ponta: a estrutura expressando a intenção do sistema é o *veículo* pelo qual a complexidade essencial é contida em vez de dispersa.

O teste de realidade

"Essencial" e "acidental" são propriedades do que? Da especificação, do código, do mundo? A pergunta é menos filosófica do que parece na prática, e eu uso um teste operacional simples, que vale aplicar em qualquer revisão de arquitetura:

> Se trocássemos todas as ferramentas, linguagens, frameworks e métodos, mantendo o problema, esta dificuldade continuaria existindo?

O lote de liquidação fecha em T+1? Continua. A adquirente pode recusar o estorno depois de certa janela? Continua. O valor de um pedido precisa bater, centavo a centavo, com o somatório dos pagamentos que o cobriram, sempre? Continua. Isso é o problema: qualquer representação fiel vai precisar carregá-lo.

Agora as outras: o pedido precisa existir como entidade em três bancos diferentes porque dois microsserviços decidiram ser donos dele? Não, isso é histórico de decisão, não lei do domínio. Precisamos escrever o adaptador do adaptador porque a biblioteca de integração "exige um contexto de execução"? Não, exigência de ferramenta. O novo colaborador leva duas semanas para descobrir por que existem quatro classes chamadas `Pagamento*`? A dificuldade é real, mas mora na representação, não no pagamento.

O teste separa os dois tipos com uma precisão incômoda: ele obriga a dizer, de cada camada do sistema, *o que ela carrega do problema e o que ela carrega de si mesma*. Uma arquitetura que não passa nesse teste não é complexa demais porque tem muitas camadas, tem muitas camadas porque ninguém fez o teste. E há um caso particularmente revelador: quando a resposta para "esta dificuldade continuaria?" é "sim, mas ela estava *em outro lugar* antes", alguém moveu complexidade essencial sem ter decidido movê-la. O ORM que

apaga transações não eliminou a transação; transferiu-a para o DBA que vai caçar o deadlock às três da manhã.

Uma cautela honesta: "essencial" tem duas camadas que a conversa costuma colar. A complexidade do *domínio em si*, as regras que o mundo impõe, e a complexidade do *problema que escolhemos resolver*, que inclui requisitos que nós mesmos formulamos. Redundância entre três datacenters é essencial para o sistema que promete disponibilidade regional; é acidental para o sistema que promete apenas "um site". Quando dois times discordam sobre se algo é essencial, na maioria das vezes a discordância não é sobre tecnologia: é sobre qual problema foi escolhido. Vale registrar a escolha, porque a fronteira entre os dois casos é exatamente onde a generalização prematura mora, o componente que resolve o problema de amanhã e cobra aluguel no de hoje.

O erro oposto: essencial disfarçado de acidental

Se o vício comum é tratar acidental como insulto, o vício simétrico, e mais caro, é tratar essencial como eliminável. É a versão madura do que o prefácio chamou de confundir simplicidade com superficialidade.

O modelo de um status só é o exemplo puro. Ele tem uma virtude imediata: quem lê, entende. E é exatamente aí que mora o perigo que este livro precisa nomear com todas as letras, porque ele não aparece em nenhum checklist: **um modelo superficial é compreensível para quem ainda não foi confrontado pelo domínio.** A compreensão que ele oferece é emprestada, emprestada do fato de que as regras ignoradas ainda não cobraram. No dia em que cobram, o preço não é medido em linhas de código, e sim em reconciliação manual, em estorno feito no painel do adquirente por uma equipe que não sabe o que o modelo "simples" prometeu ao cliente, em um segundo sistema paralelo nascido para carregar o que o primeiro recusou.

Como distinguir, olhando o modelo, entre o que é essencial bem nomeado e o que é essencial escondido? A pergunta que funciona é: *onde este modelo diz não?* Um modelo de estorno fiel ao domínio precisa recusar, em tempo de compilação, em validação explícita, ou em erro declarado, o estorno depois da janela, o parcial sem autorização, o valor que não bate com o original. Um modelo que não recusa nada não é simples: é mudo. E um sistema mudo não elimina complexidade essencial; delega-a silenciosamente para a produção, que é o pior departamento de engenharia que existe, porque não tem testes, tem clientes.

O teste se aplica igualmente ao outro lado da fronteira. Quando alguém propõe uma nova abstração "para esconder essa complexidade", a pergunta não é "fica mais simples de ler?", quase sempre fica, por um trimestre, mas "para onde foi a complexidade?". Se a resposta honesta for "para o contrato da abstração, que agora precisa ser respeitado por todos sem que o compilador ajude", a abstração não reduziu nada; concentrou em um ponto invisível. Há abstrações boas exatamente por isso: elas concentram a complexidade onde existe uma única pessoa responsável e um único lugar para verificar. Há as outras.

Acidental não é palavrão: a contabilidade

Com os dois erros de leitura afastados, sobra o que a distinção realmente entrega: uma contabilidade. Toda decisão de engenharia compra alguma coisa pagando complexidade acidental; a pergunta séria nunca é "como eliminar o custo", e sim "o que o custo comprou, e para quem".

Coleta de lixo é o exemplo clássico e limpo. O custo é real e permanente: comportamento de memória difícil de raciocinar, pausas, um runtime que você não controla integralmente. A compra é uma classe inteira de bugs de ciclo de vida que deixa de ser obrigação da equipe. Para um serviço de pagamentos, a conta provavelmente fecha. Para o firmware de um dispositivo que precisa responder em 200 microssegundos, não fecha, e o mesmo objeto de programação vira custo puro. Nenhum dos dois lados está "errado"; os problemas são diferentes, e a complexidade acidental é a mesma.

O caso do `sealed` do capítulo anterior é o mesmo cálculo em escala de sintaxe. O custo: mais uma palavra na linguagem, mais uma regra para o leitor aprender, mais uma restrição que às vezes incomoda (o conjunto que precisava abrir exige editar a declaração selada). A compra: a obrigação de "lembrar de atualizar todos os switches quando surgir um caso novo" deixa de existir, o compilador assume a dívida, para sempre, de graça, para toda pessoa que tocar no código. É um caso em que complexidade acidental *mecanizada* elimina complexidade essencial *de manutenção*. A regra geral se formula bem: complexidade acidental vale seu preço quando ela pega para si uma tarefa que o problema exigiria de alguém, lembrar, verificar, não esquecer. Ela é desperdício puro quando não pega tarefa nenhuma.

E há a categoria que ninguém admite: complexidade acidental que foi comprada e nunca chegou. O framework de abstração de banco que existe para trocar de SGBD "quando for preciso", o padrão de plugin para um ponto de extensão com um único plugin, a camada de assemblagem do assembler que só faz repassar chamadas. Cada uma é um juro sobre uma dívida que ninguém contraiu. O teste de realidade as reconhece na hora: trocadas todas as ferramentas, a dificuldade que elas resolvem continua? Não, então a ferramenta resolve uma dificuldade que ela mesma trouxe ao existir.

A contabilidade aplicada a mim mesma

Este livro perdeu a credibilidade em qualquer assunto que não for Kof se não passar o teste de realidade na própria linguagem de quem o escreve. Então, três lançamentos do livro-razão de Kof 0.6.1, com a assimetria de tratamento invertida de propósito: aqui quem cobra o preço é a autora da ferramenta.

O `?.` que não existe. Kof não tem operador de navegação segura; a única forma de estreitar um `String?` é o `if (s != null)` explícito. Custo mensurável: verbosidade linha a linha, para sempre, em todo código que trata ausência. Compra: nenhum, a linguagem poderia ter o operador e o narrowing; a ausência é escolha, não necessidade. Veredito honesto: é acidental de um tipo específico, a complexidade que uma linguagem deixa de propósito. A motivação documentada no repositório não explica por que a conveniência foi recusada, apenas que está planejada; o capítulo sobre nulabilidade vai tratar do que a decisão fez com o código real, que é onde escolhas de linguagem deixam de ser opinião e viram evidência.

`class Circle(Double r) implements Shape`. Este compila e roda, foi medido. E é uma armadilha: em Kof, a forma `class Nome(...)` com parâmetros no cabeçalho *é um record*, não uma classe comum. O leitor vindo de Java ou Kotlin escreve a linha esperando um construtor e obtém dados imutáveis com acessores gerados; a sintaxe idêntica significa outra coisa. Teste de realidade: trocadas as ferramentas, a dificuldade continua? Não, ela existe *porque* a ferramenta escolheu uma sintaxe que conflita com a expectativa de quem chega do ecossistema principal do JVM. É complexidade acidental no estado mais puro: não compra verificabilidade (o `record` explícito compra a mesma garantia com nome próprio), não elimina tarefa essencial nenhuma, e cobra de todo leitor poliglota o peágio do desentendimento. A documentação que verifiquei não registra uma motivação para a forma compacta com esse significado; o resultado, sim, e é o que estou avaliando.

Exceções como String. `throw "mensagem"` e `catch (String e)` funcionam na JVM, medidos. A promessa sintática: "isto é o mecanismo de erro de Kof", esconde uma divergência de backend que a documentação oficial registra: no alvo nativo, o primeiro `catch` captura tudo, sem despacho por tipo, e não há stack trace. O custo essencial de existir erro é real em qualquer sistema; o custo acidental de *o contrato de erro significar coisas diferentes em targets diferentes* é escolha de implementação. E aqui a contabilidade fecha com um lançamento que este livro não pode registrar contra ninguém além de si mesmo: a sintaxe sugere uma garantia que só um dos backends entrega. Enquanto for assim, o capítulo correspondente vai dizer isso com todas as letras, porque a alternativa, descrever a linguagem como ela pretende ser, seria exatamente o erro do modelo de um status só, aplicado a mim.

Nenhum desses lançamentos desqualifica Kof. Dois deles (o narrowing explícito, o record com sintaxe compacta) podem ser defendidos em seus méritos; o terceiro é um gap de paridade honestamente codificado na documentação da própria linguagem. O ponto não é o veredito por item: é que a distinção essencial × acidental só vale alguma coisa se ela for aplicada *a tudo*, inclusive, principalmente, ao que a gente construiu com as próprias mãos. Uma filosofia de engenharia que só funciona contra terceiros não é filosofia; é marketing com bibliografia.

O que a distinção deixa para o resto do livro

Três heranças concretas deste capítulo, que os próximos vão cobrar.

A complexidade se move, se concentra, se mecaniza, mas não se destrói. Essa frase é uma heurística de trabalho, não uma lei física; há casos em que simplificar o problema (cortar o requisito, fechar a exceção, decidir o que o sistema *não* faz) elimina complexidade de verdade, e o capítulo sobre critérios vai tratar deles. Mas como reflexo de revisão de código, "para onde foi essa complexidade?" é a pergunta que separa projeto de cosmética, e ela viaja para todos os capítulos daqui em diante: tipos, abstração, arquitetura, testes.

A segunda herança é o teste de realidade como instrumento de conversa técnica. Quando dois engenheiros discordam sobre se uma camada é necessária, a discordância quase nunca é sobre a camada: é sobre qual

problema foi escolhido e qual dificuldade é do mundo. Nomear isso transforma debate de gosto em contabilidade, e contabilidade tem como resolver.

A terceira é a que abre o capítulo seguinte, e ela estava implícita desde a primeira página deste: toda a contabilidade de complexidade cobra de alguém. O sistema tem um orçamento de dificuldade que precisa caber em cabeças humanas, das pessoas que vão lê-lo, depurá-lo, estendê-lo, segurá-lo num incidente às duas da manhã. Chamar de "acidental" o que é do domínio é um erro técnico; esquecer que existe um custo cognitivo que nenhuma dessas categorias elimina é o erro que transforma sistemas tecnicamente corretos em sistemas que ninguém consegue manter. É sobre essa fatura, a que nunca aparece em diagrama, que o próximo capítulo trata.

Capítulo 3 — O custo cognitivo do software

O incidente era pequeno. Uma cobrança duplicada, três clientes, nenhum valor alto. O alerta apontava para o serviço de pagamentos, e o serviço de pagamentos passou a madrugada inteira negando participação: os logs mostravam uma chamada por pedido, os testes de contrato passavam, o time de plantão conhecia o módulo de cor. A duplicação aconteceu porque o serviço de retry, criado oito meses antes para um problema que já não existia, reenviava pedidos cujo *timeout* o gateway tinha estendido sem avisar ninguém, e a informação de que os dois comportamentos conversavam não estava escrita em lugar nenhum. Ela existia em duas cabeças. Uma estava de férias; a outra tinha esquecido.

O conserto levou quatro minutos quando alguém finalmente fez a pergunta certa, às três da manhã, para a pessoa errada, que por acaso lembrava. O custo da madrugada não foi o conserto. Foi a reconstrução: refazer na cabeça, a partir de fragmentos, um modelo mental do sistema que o sistema se recusava a fornecer.

Este capítulo trata dessa conta. O capítulo anterior estabeleceu que a complexidade se move mas não se destrói; agora importa dizer onde ela sempre acaba morando: na cabeça de alguém, que tem espaço finito, dorme, esquece, entra de férias e um dia muda de emprego. Qualquer filosofia de qualidade de software que não abra espaço para essa restrição está descrevendo sistemas para serem admirados, não mantidos.

A assimetria que ninguém escolheu

Todo mundo já ouviu a frase de que código é lido muito mais vezes do que escrito. Não vou transformá-la em número, não existem medições públicas honestas que sustentem uma razão específica, e um livro que inventa estatística para vender princípio está fazendo exatamente o que o capítulo 1 proíbe. Mas a direção da assimetria é inescapável, e a razão dela é estrutural, não numérica: a escrita acontece uma vez, num contexto que *você* escolheu, você estava lá, decidiu, tinha o problema na cabeça. A leitura acontece muitas vezes, em contextos que *ninguém* escolheu: o incidente às três da manhã, a revisão de código de um estranho, o diff urgente, o "por que isso funciona?" de alguém que herdou o módulo.

A consequência para o projeto é imediata e vale mais que a estatística: a estrutura do código é otimizada para o leitor no pior contexto, não para o autor no melhor. O autor sabe o que quis dizer; ele perdoa ambiguidade, atravessa lacunas, completa o que não foi escrito. Quem lê no pior contexto não tem nada disso, e é para essa pessoa, sem memória e sem paciência, que a intenção precisa estar na estrutura. Não é gentileza com o leitor; é a única forma de o sistema continuar compreensível quando a compreensão do autor já não estiver disponível.

Há um segundo efeito da assimetria, menos óbvio: cada linha que você escreve é uma linha que alguém, geralmente você, em outra semana, sem o contexto de hoje, vai precisar ler antes de poder ignorar com segurança. O código que existe é sempre maior que o código que está sendo escrito, e é o primeiro que decide quanto do segundo vai custar.

Três custos que a palavra "complexo" esconde

Quando alguém diz "esse sistema é complexo demais", a afirmação é verdadeira e inútil ao mesmo tempo. Complexidade cognitiva não é uma grandeza só; pelo menos três custos diferentes se disfarçam sob a palavra, e eles têm sintomas e remédios distintos. Confundi-los é a causa de muita refatoração cara que não melhora nada.

Custo de compreender: o esforço para responder "o que este trecho faz?" com o trecho na tela. Ele cresce com a densidade de decisões por linha, com nomes que não carregam significado, com o que a linguagem não deixa expressar e vira convenção. É o custo mais visível, e o que mais recebe atenção, porque é o único que aparece na revisão de código, onde só se vê o arquivo aberto.

Custo de navegar: o esforço para responder "onde está, o que chama o quê, o que quebra se eu mexer aqui?". Ele não mora em nenhum trecho; mora no grafo. Um sistema pode ter cada função trivial de compreender e ser caríssimo de navegar, dez camadas de indireção onde cada uma é limpa e o sentido só existe na sequência. É o custo que cresce com o tamanho e não aparece na revisão de arquivo único; aparece na primeira mudança que atravessa três módulos e descobre o quarto que ninguém sabia existir.

Custo de lembrar: o esforço de manter vivas, enquanto se trabalha, as verdades que o sistema *não carrega*. Invariantes que só existem na prática. A ordem em que os inicializadores precisam rodar. O fato de que o retry conversa com o timeout. Nada disso aparece na tela de quem edita; tudo isso precisa estar em alguma cabeça, e as cabeças são intercambiáveis só até alguém pedir demissão. É o custo mais perigoso dos três, porque ele é invisível para quem já o paga sem perceber, o veterano que "sabe como as coisas funcionam aqui" confunde a própria memória com a clareza do sistema.

Os três custos se pagam com moedas diferentes. Compreensão se melhora no nível do trecho: nomes, estruturas, sintaxe que expressa em vez de esconder. Navegação se melhora no nível da arquitetura: menos saltos, fronteiras que coincidem com o domínio, dependências que apontam para um lado. Lembrança se melhora no único nível que a maioria das equipes ignora: movendo a verdade do que as pessoas *sabem* para o que o sistema *verifica*, e é aqui que a distinção do capítulo anterior vira ferramenta.

Onde cada arquivo é limpo e o sistema não

O caso mais traiçoeiro é o sistema que passa no teste da compreensão e falha nos outros dois. Ele existe em toda parte, porque cada camada foi escrita por alguém competente, com boas intenções, defendendo as regras que este livro ainda vai questionar.

A requisição entra pelo controlador, que valida e delega ao serviço de aplicação, que orquestra o serviço de domínio, que fala com o adaptador de infraestrutura, que chama o cliente HTTP do parceiro. Cinco arquivos. Cada um tem uma responsabilidade, nomes razoáveis, testes próprios. Qualquer revisão local aprovaria os cinco.

Agora a pergunta de manutenção real: *onde mora a decisão sobre o que acontece quando o parceiro demora?* A resposta não está em nenhum dos cinco. O timeout é configurado no quarto, o retry mora num bean anônimo do segundo, o circuito aberto vive num terceiro que não aparece no fluxo porque é aspect, e a interação entre os três, que é o comportamento observável do sistema, não está escrita em lugar nenhum como uma coisa só. O engenheiro que precisa mexer no tempo de resposta do parceiro tem que *reconstruir* o mecanismo: abrir os cinco arquivos, lembrar do aspect, descobrir o bean, e montar na cabeça o grafo que o código se recusa a mostrar. Cada arquivo é localmente compreensível; o fluxo é globalmente ilegível.

Esse é o ponto exato onde a promessa da introdução cobra cumprimento: compreensão local e compreensão global são propriedades diferentes, e a segunda não é a soma da primeira. A metáfora cartográfica é literal para quem navega código: você pode ler todas as ruas de uma cidade sem conseguir explicar o caminho entre dois bairros, porque o caminho não é uma rua, é uma relação entre elas. Sistemas legíveis são os que mostram as relações no nível em que elas são decididas. Quando a decisão "como tratamos lentidão do parceiro" existe em um lugar só, um arquivo só, com um nome só, o custo de navegar despenca, e o custo de compreender *aquele lugar* aumenta, porque ele agora carrega o que estava disperso. Não é contradição; é troca. Toda arquitetura é uma decisão sobre *onde concentrar a dificuldade*, e o capítulo de arquitetura vai levar essa frase a sério. O que este capítulo registra é o critério da troca: concentre onde a mudança bate, disperse onde a leitura fica.

O orçamento finito e os tributos invisíveis

Existe uma restrição biológica que nenhuma ferramenta eliminou: a quantidade de coisas que uma pessoa consegue raciocinar ao mesmo tempo é pequena, pequena o bastante para que "vamos abstrair isso em outro lugar" quase sempre mude mais o problema do que resolve. Não citarei números de memória de trabalho como se fossem leis; a literatura que os media é disputada, e a precisão aqui seria falsa. O que importa para engenharia não é o limite exato, é sua existência e sua rigidez: o espaço de raciocínio simultâneo é finito, não treinável até a irrelevância, e todo projeto opera dentro dele.

Esse orçamento paga dois tipos de despesa: o que está na tela, e o que *não está*. A segunda categoria é a que me interessa, porque ela é invisível para quem a cobra. Cada mecanismo silencioso, cada comportamento que acontece sem que a estrutura o mostre, ocupa espaço na cabeça do leitor sem aparecer no arquivo que ele lê. Convenções não escritas. Configuração mágica que algum framework aplica sem perguntar. Herança que injeta comportamento de longe. Estado global lido de um lugar distante. Cada um deles é um tributo: o leitor precisa *lembrar de levar em conta* antes de poder confiar no que vê.

Um sistema pode ser feito inteiramente de peças compreensíveis e ainda assim ser ilegível, se o conjunto das peças que não aparecem na tela crescer demais. A pergunta que expõe isso não é "esta classe é clara?", é: **"quantas coisas fora desta tela precisam ser verdadeiras na cabeça do leitor para que este código faça o que parece fazer?"** Quando a resposta honesta passa de duas, três, você não tem um sistema de arquivos limpos; tem um sistema que vive na cabeça de meia dúzia de pessoas, e essas pessoas têm preço de mercado e direito a férias.

O que uma linguagem pode cobrar, e pagar

Aqui o instrumento do livro entra na conta, porque custo cognitivo não é só propriedade do projeto: é também do meio de expressão. Linguagens distribuem os três custos de formas muito diferentes, e Kof serve de estudo de caso, para o bem e para o mal, com as medições do dossiê em mãos.

O caso de transferência bem-sucedida: nulabilidade. Em Java, a pergunta "este valor pode faltar?" não tem resposta local, qualquer referência pode, então o custo de lembrar é permanente e o custo de compreender cresce com a desconfiança generalizada. Em Kof, a pergunta migra para a assinatura: `String? find(Int id)` diz no ponto de declaração o que antes era rumor, e o narrowing é uma única forma: `if (s != null)`, que o compilador fiscaliza; atribuir `null` fora do contrato medido é recusado com SEM048. O custo de lembrar vira custo de compreender, pago uma vez por assinatura, cobrável pelo compilador em todo uso futuro. É a mesma operação da contabilidade do capítulo 2: complexidade essencial (ausência existe no domínio) deixa de ser obrigação da memória e passa a ser verificação mecânica.

O caso de transferência mal paga: a mesma nulabilidade cobra pedágio em cada leitura, porque o `?.` planejado não existe, todo consumo de `String?` exige o `if` explícito, e em código que trata muitos opcionais de borda isso é texto que o leitor percorre sem que nada de novo lhe seja dito. E há o tributo invisível que eu mesma já paguei duas vezes ao rodar exemplos para este livro: o diretório de um arquivo `.kf` é o módulo. Dois exemplos com `main()` na mesma pasta não compilam com PKG002, e a mensagem não explica o modelo mental que faltava; ela cobra a explicação de quem já a conhece. Um recém-chegado esbarra no erro e aprende uma regra que ninguém escreveu onde ele pudesse ver. Isso é custo de lembrar puro, criado pela ferramenta, e a correção não é sintaxe nova: é o erro dizer o que assume.

E o caso ambíguo, que vale como lição geral: `class Circle(Double r) implements Shape` compila e significa *record*, não construtor. Para quem chega do ecossistema JVM, cada leitura dessa forma cobra o esforço de lembrar a exceção: "aqui, parênteses no cabeçalho querem dizer outra coisa". O custo é pago por todo leitor poliglota, para sempre; o benefício, que eu defendo no capítulo de sintaxe, é a economia de uma palavra em toda declaração de valor. A conta fecha ou não conforme a frequência de leitura de quem chega de fora versus quem fica. Uma linguagem desenhada para a primeira comunidade que a usasse teria escolhido diferente. Essa honestidade tem um preço editorial que o livro aceita pagar desde já: admitir que uma decisão minha cobra tributo alheio não é propaganda.

A generalização que o caso Kof ensina vale para qualquer ferramenta: **meça onde a linguagem move o custo, não quanto ela tem de recursos.** Sintaxe que elimina uma classe de erros transfere custo de lembrar para compreender, bom negócio quando o erro era provável e a leitura, barata. Mecanismo que exige conhecimento fora da tela cria custo de lembrar novo, mau negócio quando o problema que ele resolve era raro. A pergunta não é "a linguagem é expressiva?"; é "a expressão dela cobra de quem, e com que frequência?".

A dívida que mora em pessoas

Voltando ao incidente da madrugada. O retry que conversava com o timeout não era um mistério técnico: era conhecimento sem endereço. Ele morava em duas cabeças porque o sistema não oferecia um lugar onde aquela verdade pudesse morar, nenhum arquivo, nenhum teste, nenhuma fronteira que dissesse "a política de lentidão do parceiro é *aqui*". Quando a verdade não tem endereço, ela mora em pessoas, e o nome contábil para conhecimento que mora em pessoas e não pode ser transferido por documento é dívida impagável em dinheiro: ela só vence quando a pessoa vai embora, e o juro é cobrado em medo.

"Não mexe nisso que funciona" é o sintoma clínico. Cada equipe conhece a frase; quase nenhuma a trata como o que ela é: o momento em que o custo de lembrar ultrapassou a disposição de pagar, e o sistema entrou em modo de preservação em vez de evolução. Um time que evita tocar num módulo não está sendo conservador; está pagando a fatura de um projeto que concentrou decisões onde ninguém consegue navegar, e a dívida cresce com cada mudança feita *ao redor* do módulo intocável para não *dentro* dele.

O teste prático para qualquer sistema, e que vale aplicar no seu na próxima reunião: *uma engenheira competente, sem o contexto do time, consegue fazer uma mudança correta usando só o que o código e os testes mostram?* Se a resposta exigir "bom, tem que conhecer o histórico", o que está sendo medido não é a competência dela, é a quantidade de sistema que vive fora do sistema. E a correção raramente é documentar mais. Documentação é o band-aid do custo de lembrar; o tratamento é mover a verdade para dentro da estrutura, um tipo, um módulo, um teste que falha quando a regra é violada, que é a operação que o capítulo 1 chamou de intenção em código, vista agora pelo lado de quem paga.

A fatura e o próximo capítulo

Os três capítulos até aqui formam uma linha só. O código é representação, e qualidade é o que a estrutura diz e o que ela custa (capítulo 1). O custo se chama complexidade, e a contabilidade séria separa o que o problema cobra do que nossas escolhas cobram (capítulo 2). E o cobrador real, aquele que paga todas as contas com a mesma moeda finita, é a cabeça humana, compreendendo, navegando, lembrando (capítulo 3).

Existe uma tecnologia social antiga para reduzir essa fatura, tão antiga quanto as equipes: combinar significados antecipadamente, para que o leitor não precise decifrar o que o autor quis dizer. Convenção: o acordo de que certas coisas querem dizer sempre a mesma coisa. Ela é genuinamente poderosa, e este livro não a critica, o próximo capítulo critica outra coisa: o momento em que o acordo para de servir ao entendimento e passa a servir a si mesmo, e a convenção cobrada vira burocracia que alguém paga sem lembrar por que assinou.

Capítulo 4 — Quando boas práticas viram burocracia

Começo pelo lado em que a convenção funciona, porque um capítulo que começasse pela crítica já teria escolhido o erro que pretende denunciar.

Uma equipe de verdade, genérica o bastante para não expor ninguém, específica o bastante para ser reconhecível, tinha uma regra: nenhum código entra em produção sem um teste que falhe se a regra de negócio que ele implementa for violada. Não "cobertura mínima", não percentual: um teste que prove o comportamento, escrito por quem entende o comportamento. A regra nasceu de uma madrugada cara, dessas em que um deploy de sexta-feira ensina alguma coisa.

O efeito visível apareceu em semanas. Ninguém mais discutia se o código estava pronto: a discussão era se o teste provava o que precisava provar. O revisor novo entendia o domínio escrevendo o primeiro teste, porque descobrir o que testar obriga a descobrir o que importa. E o time parou de negociar o óbvio, a regra respondia de antemão a uma pergunta que antes consumia reunião atrás de reunião. Uma convenção que funciona tem esse sabor: o desacordo desce para o nível onde é barato resolver, e o que todo mundo já sabe não precisa ser dito de novo a cada pull request.

Era uma boa prática. Nascida de um problema real, com dono, com justificativa viva na memória de quem a criou. E é exatamente por isso que vale estudar a sua degeneração, porque ela não precisou de nenhum vilão. Ninguém votou por burocracia.

A regra é uma decisão comprimida

Toda convenção útil é uma decisão tomada uma vez para não ser tomada de novo. Quando a equipe decidiu "teste que prova comportamento antes do merge", ela congelou um julgamento: *aqui, este é o ponto onde o erro custa mais caro que o esforço de preveni-lo*, e distribuiu o congelado como padrão. É o mesmo mecanismo de qualquer abstração: paga-se uma vez para não pagar sempre. O capítulo 2 chamou essa conta pelo nome; o que importa agora é a parte que a metáfora da compressão revela: **todo arquivo comprimido precisa de um descompressor, e o descompressor é o contexto que criou o arquivo.**

Enquanto as pessoas que viveram a madrugada de sexta estão no time, a regra carrega a própria justificativa. Alguém conta a história, o novato entende o tamanho do problema, e a convenção funciona pelo motivo certo. Mas equipes têm metabolismo: pessoas se movem, produtos mudam de escala, a arquitetura que tornava aquele ponto o mais caro vira outra. A regra continua lá, idêntica, executando uma decisão cuja razão de existir já foi embora, e agora ninguém no raio de alcance sabe *qual* decisão ela congelava.

É aqui que a degeneração acontece, e ela tem um mecanismo reconhecível em retrospecto: a justificativa da regra é substituída pela *autoridade* da regra. "Por que todo código precisa de teste que prove comportamento?" deixa de responder-se com a madrugada de sexta e passa a responder-se com "sempre foi assim", "está no onboarding", "é o padrão da indústria", "o livro manda". Nenhuma dessas frases é um

argumento técnico; todas são transferências de autoridade. E a transferência de autoridade é o sintoma exato do momento em que a convenção virou burocracia, não quando ela incomoda, que ela pode incomodar e continuar certa, mas quando ela não consegue mais dizer *contra o que* ela está protegendo você.

A burocracia não é a regra chata; é a regra órfã.

Três perguntas que separam a convenção viva da órfã

"Sense the smell" não é critério, e este livro inteiro é uma recusa de parar no cheiro. A fronteira entre convenção e burocracia dá para operar com três perguntas, e eu as aplico em qualquer processo que esteja me vendendo disciplina, inclusive os que eu mesma inventei.

Primeira: a regra ainda resolve o problema que a criou? A pergunta é histórica e factual. Toda regra decente nasceu de um problema nomeável, o deploy que quebrava em produção, o bug que voltava porque ninguém sabia o comportamento esperado, o incidente que a equipe do capítulo 3 passou a madrugada reconstruindo. Se ninguém no time consegue apontar o problema original, você não sabe se a regra continua necessária; você só sabe que ela continua *presente*, que é a definição operacional de burocracia. E note a assimetria: uma regra cuja origem se perdeu continua cobrando o preço integral enquanto entrega um benefício que ninguém consegue mais demonstrar. O capítulo 2 chamaria isso de juro sobre dívida que ninguém contraiu, ou contraiu por um motivo que já foi pago.

Segunda: quem paga quando a regra erra, e essa pessoa pode dizer não? Toda regra genérica produz, com regularidade estatística, casos onde ela custa mais do que entrega. O bug de desempenho que exige o hotfix fora do processo; o protótipo de duas horas que provaria a hipótese antes de qualquer teste; o módulo experimental onde o contrato ainda nem existe para ser provado. Em uma convenção viva, esses casos são tratados pelo mecanismo de exceção, que é a regra admitindo que alguém, em algum lugar, sabe mais que ela. Em burocracia, o mecanismo de exceção não existe ou é caro o bastante para ninguém usar: pedir exceção exige mais esforço do que obedecer, e a obediência vence por atrito, não por mérito. O teste é simples de aplicar: *conte quantas exceções a sua regra teve no último trimestre.* Zero exceções não significa zero casos excepcionais; significa que o processo abafou os que existiram, ou que ninguém teve autoridade para nomeá-los. A regra que não pode dizer "não" não é uma regra: é uma paisagem.

Terceira: a regra consegue dizer *por quê* no nível em que pode ser contrariada? Esta é a mais sutil. "Funções devem ter no máximo dez linhas" não pode ser contrariada com bom senso, porque não há bom senso dentro dela, é um número, e números obedecem ou desobedecem, não argumentam. "Uma função deve caber na compreensão de quem a lê sem rolar a tela para entender o fluxo" pode ser contrariada: às vezes rolar é barato, às vezes dez linhas fragmentadas custam mais que trinta coesas. A diferença não é a utilidade, é o *tipo* de critério. Regras que sobrevivem ao "por quê" são as que continuam endereçando um problema e permitem que o leitor avalie, naquele caso, se o problema está lá. Regras que não sobrevivem viram checklist, e checklist sem problema é o que o capítulo 3 descreveu como custo de lembrar puro: você carrega a regra na cabeça sem carregar a razão, e a única forma de usá-la é obedecendo.

Clean Code: o que ficou, o que engessou

Não dá para falar de boas práticas virando burocracia sem nomear o caso mais bem-documentado da história, e não é honesto fazê-lo sem reconhecer primeiro o tamanho da contribuição. Quando *Clean Code* circulou, a discussão dominante sobre qualidade de código era a máquina: performance, correção formal, especificação. Robert Martin colocou na mesa um conjunto de perguntas que pareciam subjetivas demais para engenharia, como nomear, como decompor, como escrever para o leitor humano, e as tratou como engenharia. Isso foi correto e foi necessário. As perguntas continuam corretas: *este nome carrega o significado? esta função faz uma coisa? este fluxo pode ser lido sem reconstruir estado na cabeça?* Um livro que ensinou milhões de programadores a fazer essas perguntas tem um lugar que a crítica nenhuma pode tirar.

O que degenerou não estava no livro; estava no transporte. Uma parte da comunidade que o adotou converteu *raciocínios* em *vereditos*, porque vereditos são mais fáceis de distribuir. Martin escreve sobre o tamanho de função argumentando com custo de leitura e coesão; o mercado reciclou como "função de mais de sete linhas é ruim". A frase original pode ser contrariada com argumento; a reciclada só pode ser obedecida ou violada. E aí aconteceu o fenômeno que este capítulo estuda: a regra sobreviveu à razão, ganhou autoridade de citação: "está no Clean Code", e passou a produzir exatamente o que o capítulo 3 chamou de tributos invisíveis. Times que extraíram cinco funções auxiliares de um raciocínio coeso para caber no número, e em troca espalharam o fluxo por vinte linhas de navegação. Times que recusaram um arquivo de trezentas linhas legíveis porque "função grande é ruim", sem perguntar o que a grandeza comunicava. A introdução deste livro abriu com os dois lados dessa briga; aqui importa o diagnóstico: **as duas equipes estavam aplicando a regra corretamente e produzindo código pior, porque a regra era o substituto do julgamento, não a ferramenta dele.**

Vale registrar a defesa justa: nada disso é culpa do autor, e quase tudo é culpa do meio editorial que transforma livros técnicos em listas de regras, porque listas vendem melhor que raciocínios. A lição que o caso ensina é geral e não é sobre Martin: *toda prática que viaja sem seus argumentos chega ao destino como dogma.* É por isso que este livro vai repetir uma frase até ela perder a graça: não quero que você concorde comigo; quero que você saiba em que condição cada ideia aqui deixa de valer.

O custo da burocracia, nas duas moedas dos capítulos anteriores

Agora dá para precificar, e a precificação é o que separa a crítica vaga da engenharia.

Pelo lado do capítulo 2, burocracia é complexidade acidental sem contrapartida. Cada artefato que a regra órfã exige, o sétimo arquivo de interface para uma abstração com uma implementação, o documento de design para a mudança de três linhas, a reunião de alinhamento sobre o alinhamento, adiciona estrutura que não pega tarefa essencial nenhuma. O teste de realidade aplicado à regra: se trocássemos todas as ferramentas e métodos, esta exigência ainda resolveria algum problema? Quando a resposta honesta é "ela resolve o problema de *cumprir a exigência*", você está pagando aluguel pela própria burocracia.

Pelo lado do capítulo 3, burocracia é custo de lembrar disfarçado de custo zero. A regra que não explica o por quê ocupa espaço na cabeça do novo integrante como fato bruto: "aqui é assim", e ele precisa carregar dezenas desses brutos para funcionar, sem conseguir reorganizar nenhum, porque reorganizar exige a razão que ninguém tem. Pior: a regra órfã *aumenta* o custo de compreender, porque o código que a obedece literalmente fica cheio de estruturas cujo único propósito é satisfazê-la, e o leitor precisa descobrir isso, sem ajuda, para saber o que pode ser ignorado.

Há ainda um terceiro custo, que nenhum dos dois capítulos nomeou: o custo de seleção. Burocracia não é neutra entre boas e más aplicações, ela é *cega* entre elas. Um processo que exige o mesmo ritual do hotfix de emergência e da mudança cosmética não mede risco nenhum; mede obediência. E times aprendem rápido a jogar o jogo: o ritual vira teatro, o teatro vira atraso, e o atraso vira a reputação do time ("eles são lentos"), atribuída às pessoas erradas. O custo real da burocracia nunca esteve nas horas de ceremony; esteve nas decisões que o time parou de tomar porque o processo já "decidiu" por elas, sem saber do que se tratava.

Quando a regra mora na máquina

Este livro usa Kof como laboratório porque a linguagem é o lugar onde a fronteira entre convenção e verificação fica visível, e, para variar, é um laboratório onde eu não sou avaliadora de terceiros.

Pegue o `switch` sobre hierarquia selada do capítulo 1: a regra "todo caso deve ser tratado" é imposta pelo compilador com SEM081, medido. É burocracia? Pelo contrário, é o caso ideal: a regra resolve um problema nomeável (a adição de um caso que esquece um ponto de decisão), cobra o preço no momento exato da violação (compilação, não reunião), e o mecanismo de exceção existe e é honesto: escrever `default` é a forma de a linguagem dizer "eu confio que você quer cobrir o resto sem nomear". Uma regra mecanizada bem-feita é o que a convenção social tenta ser e raramente consegue: uma decisão tomada uma vez, aplicada sem custo de memória, com saída explícita.

Agora o mesmo critério na direção incômoda, porque a máquina também erra. A regra de módulo de Kof, o diretório de um arquivo `.kf` define o módulo, produziu o PKG002: dois exemplos com `main()` na mesma pasta, e a mensagem de erro que recebi não explica o modelo mental que violou. A regra é legítima (módulo por diretório é uma decisão de engenharia com custos e benefícios reais), mas *enquanto vive apenas na máquina, ela é inquestionável de dentro*. O usuário não pode pedir o "por quê" ao compilador; a mensagem não devolve nada. Toda regra transferida para a ferramenta ganha a obediência perfeita da ferramenta e a mudez da ferramenta, e o remédio não é voltar à regra social frouxa, é exigir que a regra mecanizada carregue a justificativa na própria violação: um erro que diz *o que assume, por que, e como dizer não*. Uma linguagem que cobra sem explicar é burocracia com compilação rápida. Aplico o critério sem exceção para mim mesma porque, se não aplicasse, o resto deste capítulo seria retórica.

E há a forma mais sofisticada de burocracia de linguagem: a regra que virou estética. `fun`, `fn`, `let`, `const`: Kof recusa todos: as duas primeiras sondadas como erro de parse (PARSE085), as outras duas registradas na lista de anti-formas da referência de sintaxe. A decisão é defensável (uma só forma de declarar função, sem

guerra de dialetos dentro da linguagem), mas note o que a recusa *não* faz: ela não resolve o problema de ninguém além do problema de ter que escolher entre formas. É o tipo de regra que uma comunidade mantém por coerência, o que é um motivo, e transforma em identidade, o que é o começo do dogma. O teste das três perguntas vale para sintaxe como vale para processo: se a regra sobrevive ao por quê, fique com ela; se só sobrevive à autoridade, ela já começou a apodrecer.

Convenções vivas têm manutenção

A conclusão não é "elimine as regras". É o oposto: leve-as a sério o bastante para mantê-las, porque regra é artefato de engenharia como qualquer outro, acumula dívida, fica obsoleta, precisa de refatoração. O que distingue uma convenção viva tem formato de prática:

Toda regra tem um problema nomeado e um dono. Não "está no onboarding": o documento da regra começa pelo incidente ou pela razão que a criou, e tem alguém responsável por reavaliá-la. Se não existe problema nomeável, a regra já morreu e o velório é opcional.

Toda regra tem saída documentada. O mecanismo de exceção não é furo no processo; é o sensor de que o processo chegou no limite do contexto para o qual foi desenhado. Exceções explícitas e baratas batem em violações implícitas e caras, quando o time começa a "quebrar a regra em segredo", a auditoria a fazer é na regra, não no time.

Toda regra tem data de reavaliação. Não porque toda regra vá mudar, mas porque o custo de reavaliar é baixo e o custo de não reavaliar é o mecanismo inteiro deste capítulo: a regra sobrevivendo à razão em silêncio. Uma reunião semestral de "o que ainda vale?" é mais barata que um ano de artefatos órfãos.

Regras migram para a máquina quando podem, e a máquina fala quando migram. O que é verificável, tipos, contratos, fronteiras, pertence ao compilador, que não esquece, não tem reuniões e cobra no momento certo. O que é julgamento, o que testar, o que nomear, o que separar, não migra, e fingir que migra produz a burocracia automatizada que Kof me ensinou a detectar nos meus próprios erros. E quando a regra migrar, a mensagem de violação carrega o por quê, senão você só trocou reunião por erro de compilação.

Fim da Parte I

Os quatro capítulos até aqui formam um argumento único, e vale dizer qual, antes de virar a chave da Parte II. O código é uma representação do sistema, a qualidade está no que a estrutura diz e no que custa dizer (capítulo 1). A dificuldade que ela carrega se divide entre o que o mundo exige e o que nós adicionamos, e a contabilidade honesta é o que separa projeto de cosmética (capítulo 2). O cobrador de ambas as contas é a cabeça finita de quem compreende, navega e lembra, e sistemas que ignoram essa restrição transferem a fatura para as pessoas, onde ela vira medo e madrugada (capítulo 3). As regras que inventamos para distribuir essa fatura entre a equipe funcionam enquanto carregam suas razões, e quando as perdem, viram o imposto silencioso que este capítulo descreveu (capítulo 4).

Nada disso ainda diz *com que material* construir. Mas já deixou claro o que o material precisa fazer: expressar intenção verificável, cobrar custo onde o problema mora, e falar quando for violado. A pergunta sobre linguagens de programação, sintaxe, tipos, compiladores, a máquina inteira entre a ideia e o comportamento, é a pergunta sobre esse material. Ela abre a Parte II, e começa pelo lugar-comum mais falso e mais útil da profissão: a ideia de que sintaxe é cosmética. Não é. Sintaxe é uma decisão de engenharia, e o capítulo 5 vai abrir cada uma das escolhas de Kof na mesa, com as medições, as divergências de documentação, e as perguntas difíceis que uma linguagem precisa aguentar de quem a criou.

Parte II — A linguagem influencia o pensamento

Capítulo 5 — Sintaxe é uma decisão de engenharia

"Há uma semana nossa equipe discute se devo usar `var` ou o tipo explícito nas declarações locais. Isso é relevante ou estamos só debatendo estética?"

A pergunta veio de um desenvolvedor honesto, e a resposta curta é: a discussão *dele* provavelmente é irrelevante, mas o fato de a discussão existir responde ao resto do capítulo. Estética não gera guerra de uma semana. Estética não faz time arquivar convenção de estilo de quarenta itens. Estética não faz alguém, às duas da manhã, ler uma linha de código e traduzi-la na cabeça antes de entendê-la. Quando a forma de escrever passa a consumir reunião, documento e atenção de madrugada, ela deixou de ser aparência e revelou o que sempre foi: **uma decisão de engenharia, tomada com ou sem consciência, distribuída por todo o sistema.**

A tese popular é a inversa. "Sintaxe é açúcar", diz o senso comum da profissão; "engenheiro de verdade lê além da sintaxe; o que importa é semântica". Há um núcleo verdadeiro aí, nenhuma sintaxe salva um pensamento confuso, e um bom engenheiro de fato atravessa dialetos, e um erro de categoria embutido. Porque "atravessar a sintaxe" não é um custo zero pago uma vez; é um custo de tradução pago *a cada leitura, por toda a vida do código*. O capítulo 3 mostrou que a leitura é onde o software acumula custo; a sintaxe é o meio onde esse custo acontece. Chamar o meio de cosmético é como chamar a planta de um prédio de cosmética porque "o que importa é a estrutura".

As três perguntas de toda escolha sintática

Uma forma sintática, uma palavra, um sinal, uma omissão, uma ordem de tokens, faz três coisas ao mesmo tempo, e a decisão de engenharia séria avalia as três:

Primeira: o que ela torna barato dizer? Linguagens têm economia. O que é barato de expressar será expressado com frequência, não porque os programadores sejam racionais, mas porque ninguém resiste a mil incentivos diários de atrito zero. Se declarar intenção custa uma palavra, a palavra aparece. Se custa uma classe wrapper, a intenção migra para o comentário, que o compilador não lê, o revisor não executa, e o tempo apaga.

Segunda: o que ela torna caro dizer, e, portanto, o que será pulado? Todo custo de expressão é um filtro seletivo sobre o que chega ao código. Kof não tem operador de navegação segura; o `?.` de outras linguagens é planejado e não existe, medido como erro de parse. Consequência: cada leitura de `String?` exige o `if` explícito. Isso não é neutro. Em algumas equipes o atrito produz código mais honesto, porque força cada consumo de opcional a *declarar a decisão de tratá-lo*; em outras, produz o contorno, o `as String` que "resolve" o erro e transfere o problema para a madrugada de outra pessoa. A mesma sintaxe, economias opostas. Uma linguagem também decide pelo silêncio: o que ela não facilita será feito de outro jeito ou não será feito.

Terceira: o que a máquina pode verificar contra a forma? É o nível mais alto da hierarquia do capítulo 1, a intenção que o compilador guarda. Uma forma sintática que vem com verificação embutida compra algo que nenhuma convenção de estilo entrega: obediência garantida, sem reunião. `val cidade = "São Paulo"` em Kof não é só declaração; é a promessa de que reatribuir falha com SEM037, medido. `var` é a promessa contrária, e o leitor não precisa consultar ninguém para saber qual das duas está na tela. Com uma ressalva que a própria medição impôs e que este capítulo vai encarar na seção sobre `val`: a promessa não é uniforme entre as formas de escrever a mesma coisa.

As três perguntas juntas dão o critério: **uma escolha sintática é boa quando o que ela torna barato dizer é o que o sistema precisa dizer, o que ela torna caro dizer é o que o sistema precisa evitar, e a máquina vigia a diferença.** Nenhuma sintaxe no mundo fecha esse triângulo em todos os casos, mas a maioria das guerras de estilo acontece porque alguém avalia sintaxe pela primeira pergunta ("é mais bonito escrever assim") e ignora as outras duas.

O caso do tipo que vem primeiro

Kof tem uma decisão de projeto visível logo na primeira linha: a ordem da declaração. O arquivo validado do dossiê mostra as três formas aceitas:

```
Int idade = 26          // type-first explícito
var pi = 3.14           // inferido Double
var idade2: Int = 30    // anotado
```

As três compilam, as três produzem exatamente a mesma verificação de tipos, medido, sem diferença semântica entre elas. Então por que as três existem? A resposta honesta exige separar duas coisas que a conversa sobre sintaxe cola: *verificação* e *leitura*. O que o compilador faz é idêntico nos três casos; o que o leitor faz não é.

`Int idade = 26` começa pelo tipo. A frase abre dizendo "isto é um inteiro", o tipo ocupa a posição de destaque, e a leitura começa pelo contrato. `var pi = 3.14` começa pela intenção de nomear e confia a inferência ao leitor e à máquina. A diferença de custo é real e pequena: quem lê `var` precisa, às vezes, olhar o lado direito para saber o tipo, um microcusto por linha que se acumula por mil linhas. Quem lê type-first não precisa, mas paga o preço inverso: `var` existe justamente porque escrever `Double valorArea = calcularRaio(...)` é uma frase que ninguém quer dizer em voz alta, e que repete informação que a inicialização já declarou.

A escolha de Kof, oferecer as duas ordens, mais a forma anotada `var x: T`, é uma decisão *de estilo de leitura*, não de verificabilidade, e é exatamente o tipo de decisão que precisa ser tomada conscientemente. Uma linguagem que oferece dois caminhos para a mesma coisa cobra o pedágio do capítulo 3: o leitor precisa manter na cabeça a convenção da codebase ("aqui usamos type-first para campos públicos, `var` para temporários"), uma verdade que mora fora da tela, porque a linguagem se recusou a decidir. O debate da

equipe da primeira página deste capítulo não era irrelevante; era a equipe pagando, em reunião, uma decisão que a linguagem deixou de tomar. A resposta certa não é votar: é escrever a convenção com o *porquê* das três perguntas acima, e migrar o que der para a máquina vigiar.

`val`: uma palavra que compra uma garantia, e a entrega em duas formas

`val` era para ser o melhor caso-escola deste capítulo: sintaxe que paga o próprio custo. A forma:

```
val cidade = "São Paulo"
```

e a promessa: reatribuir depois falha na compilação. Medido:

```
val cidade = "São Paulo"
cidade = "Campinas"   // p.kf:3:5: error: cannot assign to immutable
                      // 'val' variable 'cidade' [SEM037]
```

Uma palavra de três letras converte "isto não muda" de *costume* em *verificação*. Antes dela, a intenção de imutabilidade vivia no nome (`CIDADE_FINAL`), no comentário (`// não alterar`), na revisão de código, três formas de pedir educadamente que o capítulo 4 classificou como convenção social, com toda a degradação que ela sofre. Depois dela, a intenção mora na estrutura, no nível mais alto da hierarquia do capítulo 1: a máquina recusa.

Era esse o capítulo que eu ia escrever. Aí o método deste livro, medir antes de afirmar, fez o que ele existe para fazer.

A primeira linha do exemplo acima é a forma *inferida* de `val`. Kof aceita também a forma *anotada* da mesma intenção, e foi ela que a seção anterior deste capítulo usou ao prometer SEM037:

```
val cidade: String = "São Paulo"
cidade = "Campinas"
println(cidade)
```

Na forma anotada, o `kof check` não reporta erro nenhum. O `kof run` executa e imprime `Campinas`. Repeti o par com `Int` (`val x = 1; x = 2` → SEM037; `val x: Int = 1; x = 2` → compila, roda, imprime 2) e o resultado é consistente nas duas combinações de tipo: **a imutabilidade de `val` é aplicada na forma inferida e silenciosamente liberada na forma anotada.** Duas escritas da mesma palavra, dois contratos diferentes, e o contrato que vaza é justamente o que *parece* o mais formal dos dois, porque é o que declara o tipo na cara do leitor.

Pelas três perguntas, isso é o pior resultado possível para uma escolha sintática. A forma anotada torna barato dizer uma promessa que a máquina não vigia: `val cidade: String = ...` comunica imutabilidade com todas as letras, e o compilador deixa você atribuir por cima. Não é um erro de compilação que instrui, é a

ausência dele, e ausências não deixam rastro: o bug que `val` existe para prevenir volta a viver exatamente onde vivia antes dele, na atenção do revisor. O capítulo 1 definiu o defeito central: sintaxe que sugere uma garantia que o compilador não entrega. O caso agora é meu, medido nesta máquina, com as duas sondas arquivadas no repositório do livro (`cap05-sonda-val-inferido.kf`, `cap05-sonda-val-anotado.kf`) e o registro na tabela de validação.

Duas lições saem daqui, e a segunda vale mais que a primeira. A primeira é sobre Kof: a garantia de `val` deve ser lida com ressalva de forma até que o gap seja tratado, quem escreve `val` anotado hoje está comprando costume com aparência de verificação. A segunda é sobre método, e é por isso que o caso entrou no livro em vez de sair para o issue tracker: **a frase que eu ia escrever como exemplo de sintaxe bem-sucedida estava errada, e só a medição do exemplo a ser publicado a descobriu.** Documentação certa sobre comportamento certo não protege um livro, nem uma linguagem, do ponto exato onde os dois casos se encontram. Toda afirmação deste capítulo sobre o que o compilador "faz" vale enquanto a próxima medição não disser o contrário, inclusive esta.

O caso tem ainda a honestidade de escala que este livro deve ao que funciona. A referência de sintaxe documenta outro limite da compra: `val` não é aceito em corpo de classe, campos imutáveis por essa via dão PARSE016; a imutabilidade de campo depende de `record` ou de disciplina. A palavra barata de dizer existe para locais, e a linguagem ainda não facilita dizer a mesma coisa para o estado que vive mais tempo, que é justamente onde a imutabilidade vale mais. Somando as duas lacunas: `val` cobre bem o caso local inferido, mal o local anotado, e não existe para campos. Um capítulo de filosofia pode se dar ao luxo de admirar `val`; um livro de engenharia anota o tamanho exato do que falta, e mede duas vezes, porque a primeira vez pode ser o exemplo que ia virar propaganda.

`class X(...)`: a forma que mente por sotaque

Agora o caso que doeu escrever, porque é uma decisão minha.

```
class Circle(Double r) implements Shape {
    Double area() { return 3.14 * r * r }
}
```

Este código compila e roda, medido, saída `12.56`. E a primeira leitura dele está errada para todo mundo que chega de fora. O cabeçalho `class Circle(Double r)` *parece* construtor, é a forma como Java e Kotlin declaram construtor compacto, é o sotaque que o leitor da JVM traz de fábrica. Em Kof, essa forma **é um record**: dados imutáveis, acessores gerados, igualdade por valor. O construtor de verdade, mutável, é a outra forma: `class Conta { ... public constructor(...) }`, também medida no dossiê.

Pelas três perguntas, a conta fecha mal:

Barato dizer o quê? A declaração de valores: uma linha em vez de cinco. Compra real, não nego.

Caro dizer o quê, e o que será pulado? Nada; a forma não bloqueia caminho nenhum. O custo não está na escrita, está na leitura: cada poliglota da JVM que traduz a linha na cabeça, e traduz errado na primeira vez, aprende na segunda, e paga o imposto do susto em cada revisão que envolve a forma. É o custo de lembrar do capítulo 3 em sua forma mais pura: uma regra fora da tela ("aqui, parênteses no cabeçalho significam outra coisa") que cada novo leitor precisa carregar antes de confiar no que vê.

O que a máquina verifica? Nada a mais. `record User(String name, String email)` na forma explícita entrega as mesmas garantias, imutabilidade medida com SEM038, acessores, `toString`. A forma compacta não comprou verificabilidade; comprou apenas menos teclas.

Uma decisão que gasta a clareza de quem chega e não ganha vigilância da máquina é o que a economia sintática chama de **troca ruim**: e eu a defendo com as cartas na mesa: a aposta era que uma linguagem nova formaria leitores novos, sem sotaque, para quem a forma compacta não conflita com nada. Essa aposta tem um público real, quem aprende Kof do zero lê `class X(...)` sem trauma nenhum. Mas o público que *decide* o futuro de uma linguagem nascente é justamente o que vem carregado de sotaque: os que avaliam Kof a partir de Java. Para eles, a sintaxe não foi cosmética; foi uma frase que disse uma coisa e quis dizer outra, o defeito de intenção do capítulo 1, aplicado à própria linguagem. O capítulo 23 vai encarar de novo essa decisão com a distância que ele exige; aqui fica o registro: sintaxe se avalia como arquitetura, com custo nomeado e beneficiário identificado, inclusive quando o beneficiário sou eu e o custo é do leitor.

A economia do que não existe

Toda linguagem também decide pelo que recusa, e Kof recusa com parse: `fun`, `fn`, `let`, `const`, sondado, PARSE085 nas duas primeiras formas; o resto consta da lista de anti-formas da referência. Não há guerra de dialeto interno: existe uma forma de declarar função, uma de declarar variável imutável, e a documentação gasta energia descrevendo o comportamento, não arbitrando entre três caminhos para o mesmo destino.

Pelas três perguntas, é a escolha sintática mais defensível do Kof. Barato dizer: uma única forma canônica, o leitor nunca duvida se `fun` faria algo diferente. Caro dizer: nada que uma convenção não resolvesse; a recusa não bloqueia intenção nenhuma. E o ganho silencioso é o maior: a linguagem se poupa do custo que a equipe do `var` pagou em reunião, sem três caminhos, não existe convenção de estilo a escrever, a memorizar, a vigiar. A familiaridade de quem vem do Kotlin é o preço, pago uma vez por pessoa; a ambiguidade é paga por todas, sempre, a economia manda escolher o segundo.

Vale generalizar, porque a lição não é sobre palavras-chave: **a recusa também é decisão de engenharia, e é a mais barata de manter.** Cada forma que uma linguagem não tem é um debate que nenhuma equipe dela vai ter, um dialeto que nenhuma migração vai traduzir, uma linha de estilo que ninguém vai violar sem querer. Times que reclamam da rigidez sintática de uma linguagem quase nunca sentiram o peso da flexibilidade alheia, o ano inteiro traduzindo o que o colega quis dizer com a quarta forma de escrever o mesmo `if`.

Onde a sintaxe encontra a semântica

Falta a ressalva que impede este capítulo de virar o que ele critica. Sintaxe não é tudo: a melhor forma do mundo não salva um modelo errado. Se `descontoPromocional()` e `jurosMora()` compartilham uma função, nenhuma sintaxe do planeta conserta o sistema, conserta dividir a função, e a divisão é decisão de projeto, não de pontuação. O capítulo 1 estabeleceu que intenção mora no que a estrutura impede e separa; a sintaxe é o meio pelo qual a estrutura fica legível, não a estrutura em si. Uma equipe que troca de linguagem sem trocar de modelo só aprende a cometer os mesmos erros com vocabulário novo.

Mas o inverso da ressalva também é verdade, e é o que a tese popular nega: o modelo certo fica *mais modelo* em sintaxe que o expressa direto. Um `sealed` com switch exaustivo não é um jeito bonito de dizer "conjunto fechado", é um jeito pelo qual o compilador passa a *saber* que o conjunto é fechado, e SEM081 cobra a dívida quando alguém mente. Uma linguagem onde ausência se declara (`String?`) produz sistemas onde ausência é verificada; uma onde ausência é rumor produz sistemas onde ausência é *surpresa*. A mesma equipe, o mesmo domínio, o mesmo cuidado, economias sintáticas diferentes, faturas cognitivas diferentes. É por isso que a pergunta "a linguagem influencia o pensamento?" abre esta parte do livro: influencia não por mística de hipertexto linguístico, mas por economia pura, o que a forma torna barato será dito, e o que for dito passa a existir no sistema.

A próxima parada dessa economia é o item onde sintaxe e verificação colidem com mais força: o sistema de tipos e a nulabilidade. Kof declara ausência no tipo, e o capítulo 6 vai mostrar, com as medições na mão, exatamente onde essa promessa é honrada pelo compilador, onde ela cobra um preço que a sintaxe não mostra, e por que `String? s = null`, uma linha que *parece* dizer "variável opcional inicialmente vazia", é recusada com SEM048. Se o capítulo 5 estabeleceu que a forma tem consequências, o capítulo 6 testa a afirmação na feature mais filosófica da linguagem: a pretensão de nunca deixar `null` vazar por acidente.

Capítulo 6 — Tipos, nulabilidade e estados inválidos

O contrato de câmbio tinha uma cláusula que o código não conhecia.

Quando o cliente não tinha taxa contratada, a função de lookup devolvia `null`. Era a resposta certa para a pergunta que fazia: "existe taxa para este cliente?", não existe, `null`. O chamador que exibia o extrato tratava `null` como devia: mostrava um traço. O chamador do lote de fechamento tratava `null` como não devia: caía no caminho padrão, que interpretava "sem taxa própria" como "use a taxa de mercado". Nenhum dos dois estava errada no dia em que escreveu a linha; entre os dois dias, o negócio decidiu que cliente sem taxa contratada não entrava em lote nenhum, e a função de lookup continuou devolvendo o mesmo `null` para uma pergunta que agora tinha duas respostas possíveis: "não encontrei" e "não se aplica", codificadas na mesma ausência. O fechamento rodou com taxa de mercado para quem não deveria rodar. O bug não estava em tipo nenhum: estava no lugar onde três significados moravam em um token.

Este capítulo é sobre o que um sistema de tipos faz com essa história, e, com a honestidade que o método deste livro exige, sobre o que ele *não* faz.

O que o null sempre foi, disfarçado de valor

A origem é conhecida e verificada: Tony Hoare, ao refletir sobre a referência nula que introduziu em 1965 no ALGOL W, chamou-a de "meu erro de um bilhão de dólares", a frase, dita em sua palestra no QCon London de 2009 (não na do Turing Award, como às vezes se repete por aí), virou folclore da profissão: *"I call it my billion-dollar mistake. It was the invention of the null reference in 1965."* O mérito da confissão não está em null ser um erro, é a onipresença do problema que ela nomeia, e a razão dele é estrutural: `null` é a ausência de valor escrita na gramática de valor. Todo tipo de referência passa a significar "um objeto deste tipo *ou nada*", e o *ou nada* não vem anotado em lugar nenhum.

As consequências são o capítulo 3 inteiro em miniatura. O custo de lembrar: para cada valor que circula, alguém precisa saber ou adivinhar se pode estar vazio. O custo de compreender: a assinatura `Conta buscarConta(Int id)` não diz se devolve `null` quando não acha, quando o banco falha, ou quando o cliente existe sem conta, os três casos têm o mesmo rosto. E o custo de navegar: quando um `null` atravessa dez camadas, cada uma com sua hipótese sobre o que ele significa, a pergunta "o que acontece aqui se vier nada?" tem dez respostas, escritas em nenhum lugar.

O problema não é a ausência, ausência é real, o domínio do capítulo a tinha em triplicata. O problema é a ausência *anônima*: um fato do mundo que o tipo se recusa a registrar, deixando para a adivinhação coletiva o que cabia na declaração.

Kof coloca a ausência no tipo, e o tipo passa a valer o que diz

A jogada de Kof é a mesma das linguagens modernas de tipagem: a ausência deixa de ser um valor escondido dentro dos tipos e passa a ser parte declarada do tipo. `String` nunca fica vazio; `String?` pode. A diferença parece tipográfica e é epistêmica: a pergunta "isso pode faltar?" passa a ter resposta na assinatura, verificável pelo compilador em todo ponto de uso.

O exemplo da introdução, agora lido com a lente deste capítulo:

```
String? find(Int id) {
    if (id == 1) { return "mel" }
    return null
}
```

`return null` é aceito *porque* o contrato anunciado admite ausência, a função diz na cara o que devolve, e o chamador que tentar `s.length` sem tratar o caso `null` não passa da compilação. O narrowing é uma única forma, medido:

```
var s = find(1)
if (s != null) { println(s.length) }   // dentro do bloco, s é String
```

Vale nomear o que acontece dentro do `if`, porque é mais interessante do que parece: não é um cast, não é uma anotação que o programador jura de olhos fechados. É o compilador acompanhando o fluxo de verdade: *naquele ponto do programa, depois daquele teste, o valor não pode ser `null`, então ele não é*. A garantia é local e verificada; quem mudou o código depois e quebrou a premissa recebe o erro na hora, não na madrugada. E a ausência flui pelos lugares certos: função auxiliar devolvendo `String?` com narrowing no chamador funciona como se espera, medido nesta rodada, `buscar(1)` imprime `achou`, `buscar(99)` imprime `vazio`.

A entrada também fala: `Int tamanho(String? s)` aceita `tamanho("abc")` e `tamanho(null)`, medido, `3` e `0`. A ausência é cidadã de primeira classe no contrato das funções, declarável nos dois sentidos. É pouco, comparado ao que a literatura de tipos promete? É. Mas é o pouco que a máquina vigia, e o capítulo 1 já estabeleceu de quem é o voto.

A letra miúda: onde a promessa não chega

Um livro que só mostra a face declarada da linguagem é propaganda; a seção anterior era a face declarada. Agora a miúda, com as medições da mesa.

`null` literal na declaração é recusado. `String? s = null` não compila: SEM048, medido no capítulo 1 e confirmado nas sondas do dossiê. A ausência pode *chegar* de um retorno, pode *entrar* por um parâmetro, mas não pode ser *criada* ali, na declaração de uma local. A leitura que faço do desenho, e digo que é leitura,

porque a motivação documentada não explica o caso [PENDENTE DE REVISÃO EDITORIAL, a autora deve confirmar a intenção original do SEM048], é uma recusa elegante: uma variável que nasce vazia e só ganha valor depois é uma variável com dois estados que o tipo não distingue, e o `null` de hoje pode ser o esquecimento de amanhã. Proibir o literal força a ausência a vir de um lugar nomeado, um retorno, um parâmetro, onde alguém declarou que ela é possível. É a mesma filosofia do narrowing aplicada ao berço: a ausência tem que ter procedência.

Não existe `?.`, não existe `?:`. Medido: parse error; a documentação de nulabilidade lista os dois operadores como planejados. O custo, na moeda do capítulo 3, é real e pago por todo mundo: cada consumo de opcional exige o `if` explícito, e código que trata muitos opcionais de borda afoga em blocos de três linhas. Já escrevi a defesa dessa escolha no capítulo 5 e não vou repeti-la; o que acrescento aqui é o aviso de assimetria: *a ausência dos operadores é o estado atual, não uma decisão eterna*, quando os dois entrarem na linguagem, tudo neste parágrafo precisa ser reescrito, e este livro vai medir de novo antes de escrever.

O narrowing não tem binding. `if (o instanceof String s)`, a forma que outras linguagens usam para "testar e amarrar o valor numa vez", não é suportada; o teste devolve booleano e o estreitamento exige o `as` explícito depois, como mostrou o capítulo 1 no exemplo de pattern matching. Detalhe pequeno, consequência conhecida: mais uma linha em cada teste de tipo, e uma linha a menos para o compilador interpretar. O balanço é do leitor; que o leitor saiba o preço.

Troolean: o erro existe, o tipo não foi validado. A sonda do dossiê mediu o SEM095, escrever `Bool?` é recusado com a mensagem apontando para `Troolean`, o tipo de três valores (verdadeiro, falso, desconhecido) que Kof oferece no lugar do booleano furado. O *erro* está medido; o *uso* de `Troolean`, literais, operadores de lógica de Kleene, o que a biblioteca entrega, não foi validado nesta produção. Fica registrado como documentação diz, não como fato verificado, e o apêndice de exemplos só lista o que rodou.

E o switch tem uma armadilha de diagnóstico, descoberta ao medir este capítulo. A gramática documentada pede `case Subtipo v ->`, com variável de vínculo. Escrevi o caso de um subtipo vazio sem vínculo (`case SemTaxaContratada -> "sem taxa"`), porque uma classe sem campos não tem o que vincular, e o compilador fez algo pior que rejeitar: aceitou o parse e declarou, no SEM081, que *aquele caso* não estava coberto, apontando como faltante uma linha que existe dois centímetros acima. Com vínculo (`case SemTaxaContratada s ->`), compila e roda. O registro da armadilha virou issue no repositório da linguagem (#817, com as duas variantes medidas), e ela merece estar aqui porque é o capítulo inteiro em uma mensagem de erro: a sintaxe sugeriu uma coisa que a máquina não fez, silenciosamente até o momento do diagnóstico, e o diagnóstico mentiu por omissão. É também a demonstração prática de por que este livro mede cada exemplo antes de publicá-lo: a hierarquia `sealed` da seção seguinte entrou no texto porque funcionou; a forma que não entrou é a que parecia mais natural de escrever.

Estados inválidos: o slogan e o que ele realmente pede

Há um princípio que circula tanto na comunidade de tipagem que virou chavão: "tornar os estados inválidos inrepresentáveis", frase que se atribui correntemente a Yaron Minsky [a verificar: origem exata antes de citar com nome], e chavões merecem dissecação, não repetição. A versão rasa do slogan é "modele tudo com tipos ricos", que produz o pior tipo de código: enums de trinta valores, wrappers de wrapper, um tipo novo por variação de campanha publicitária, o capítulo 1 chamou essa conta de custo de afirmação, e o capítulo 2 de acidental não contabilizada.

A versão séria é mais fria, e o SEM095 é a demonstração perfeita: *pergunte quais combinações o seu tipo deixa expressar, e feche as que o domínio proíbe, uma por vez, quando o fechamento pagar o próprio custo.* `Bool?` expressa "verdadeiro, falso, ou não sei ainda" como um booleano com um buraco, um tipo cujo conjunto de valores mente sobre o que é. A recusa do compilador com uma sugestão nomeada: `Troolean`, é o slogan em ação: não conserte o estado inválido na leitura; impeça-o na escrita, e dê ao estado legítimo (o "não sei") um nome próprio, com operadores próprios, em um tipo que não finge ser outra coisa.

O lookup de câmbio da abertura merece a mesma cirurgia. As três respostas que o `null` fundia: "não encontrei", "não se aplica", "falha na consulta", não pedem três flags opcionais; pedem um espaço de estados nomeado, e Kof tem a peça exata para isso, medida no capítulo 1:

```
sealed class ResultadoTaxa
class TaxaEncontrada extends ResultadoTaxa { Double valor
    public constructor(Double valor) { this.valor = valor } }
class SemTaxaContratada extends ResultadoTaxa
class ConsultaFalhou extends ResultadoTaxa { String motivo
    public constructor(String motivo) { this.motivo = motivo } }
```

Com `sealed` + switch exaustivo, o chamador do lote é *obrigado* a olhar para os três casos separadamente; fundir "não se aplica" com "não encontrei" deixa de ser um mal-entendido silencioso e vira SEM081 na compilação de quem tentar ignorar o quarto. A ausência anônima deu lugar a ausências nomeadas, que é o que o slogan sempre quis dizer quando parou de ser frase de palestra.

E aqui vale a precisão que separa engenharia de entusiasmo: o fechamento acima é *possível* em Kof e *medido* no mecanismo (sealed, switch, SEM081, tudo verificado), mas o custo da codificação é real. Três classes no lugar de um `null` é mais tipo, mais nome, mais arquivo; o capítulo 5 perguntaria "o que isso torna barato dizer, e o que a máquina vigia de novo?". Resposta: barato dizer a distinção que o domínio exige, e vigiado o esquecimento de tratá-la. Onde a distinção não existe, um lookup que genuinamente só tem "achou/não achou", o `String?` basta, e o sigilo do `sealed` triplicado seria exatamente a burocracia do capítulo 4. O princípio não diz "sempre modele"; diz "modele onde o inválido já mordeu".

O que a linguagem não pode, e a arquitetura precisa

A distinção prometida na introdução chega ao balcão: segurança oferecida pela linguagem × segurança exigida pela arquitetura. Kof vigia a *ausência*, no tipo, no fluxo, na declaração. O que ela não vigia, com as ferramentas medidas até aqui:

Combinações de valores. `record Conta(Double saldo, Bool estornada)` aceita `Conta(-50.0, true)`, saldo negativo com estorno marcado, um estado que o domínio pode muito bem proibir e o tipo não vê. Sem tipos dependentes, sem refinamentos, sem invariantes declaráveis no construtor (a referência não oferece nada disso, e `val` em campo de classe, como o capítulo 5 registrou, nem existe), a regra "conta estornada não pode ter saldo" mora fora do tipo. Onde mora? No construtor com validação lançando erro, e aí o capítulo 8 cobra a fatura: um `throw "string"` que só o runtime conhece, numa linguagem onde o Native captura qualquer catch sem despacho. A verificação que o tipo não faz, *alguém* faz; a escolha é só se ela será dita em um lugar e por uma voz.

Ordem e tempo. `String?` diz que a ausência pode chegar; não diz que depois do fechamento o lote não pode mudar. Invariantes temporais: "estornado uma vez, estornado para sempre", não têm onde morar no sistema de tipos de Kof. Moram na arquitetura: máquina de estados explícita, teste de transição proibida, imutabilidade dos registros de evento.

E até o que vigia, vigia enquanto o compilador estiver certo. O achado do capítulo 5: `val` anotado aceitando reatribuição, aberto como issue #816 no repositório da linguagem, é a lição final deste capítulo em forma de cicatriz: a garantia de um tipo não é propriedade eterna da sintaxe que a anuncia; é comportamento de uma versão de um compilador, medível, refutável, e às vezes refutado. A postura que este livro defende não é "o tipo diz, logo é verdade", é "o tipo diz, e eu verifiquei que a máquina faz o que o tipo diz; onde a máquina não faz, o bug vira issue e a premissa vira ressalva".

A divisão honesta do trabalho fecha assim: a linguagem pega para si o que é verificável mecanicamente e frequente o bastante para pagar o custo, ausência, exaustão de casos, imutabilidade local. A arquitetura carrega o resto, invariantes de domínio, tempo, combinações, com validação em fronteiras, testes que nomeiam o proibido, e modelos que tornam o erro difícil antes de torná-lo detectável. Nenhuma das duas camadas substitui a outra, e sistema que confia em só uma já escolheu qual madrugada vai perder.

Próxima parada

A nulabilidade resolveu uma pergunta: *este valor pode faltar?*, e transferiu a resposta do rumor para o contrato. O capítulo 7 alarga a pergunta para o componente inteiro: *o que este módulo promete, o que exige, e quem vigia o quê*, contratos, composição, e a fronteira movediça entre a abstração que esconde o que importa e a que esconde o que não importa. A ferramenta do capítulo 6, o tipo como declaração verificável, vira lá a ferramenta maior: o contrato como declaração que sobrevive a quem o escreveu.

Capítulo 7 — Abstração, composição e contratos

O módulo de pagamentos precisou de um segundo provedor, o original entrou em instabilidade, e o contrato de nível de serviço não deixava esperar. Quem foi mexer no código encontrou doze chamadas HTTP espalhadas por doze fluxos de negócio, cada uma com sua montagem de cabeçalho, seu timeout copiado e colado, sua decodificação de erro com nome de variável local. Adicionar o provedor backup significava, na prática, reescrever o sistema inteiro no nível do transporte.

Uma semana depois, existia uma interface `Cobrador` com dois métodos: `cobrar`, `estornar`, e duas implementações: a do provedor original, com todo o HTTP que já conhecia, e a do backup. Os doze fluxos de negócio passaram a depender da interface e não sabiam mais quem estava do outro lado. O segundo provedor virou uma linha de configuração. O terceiro, meses depois, foi a mesma operação de sempre.

Essa é a abstração no seu dia de vitória, e o dia de vitória merece ser descrito primeiro, porque a crítica à abstração quase sempre começa pela derrota e termina sem explicar o que perdeu. A interface `Cobrador` não "reduziu duplicação", essa é a descrição contábil, verdadeira mas superficial. O que ela fez foi *gerenciar atenção*: decidiu, uma vez, o que quem escreve fluxo de negócio precisa saber sobre cobrança (que se cobra, que se estorna, que pode falhar) e moveu o resto, autenticação, roteamento, dialeto de erro de cada fornecedor, para um único lugar por provedor. O leitor de um fluxo de cobrança parou de precisar pensar em HTTP. O custo: uma indireção a mais para quem precisa *escrever* um provedor novo, e um vocabulário (`Cobrador`) que a equipe inteira precisa compartilhar. O balanço foi claramente positivo, e a razão da clareza é que o problema escondido era real, doze vezes real.

O critério, e o inquilino fantasma

O capítulo 3 estabeleceu o orçamento: a cabeça que lê o sistema é finita, e cada elemento que ocupa espaço precisa justificar a estadia. A abstração é o maior locatário do orçamento, ela existe para *liberar* espaço no leitor, removendo um mecanismo da frente dele. Daí o critério que uso em qualquer revisão, e que vale o capítulo inteiro se você ficar só com uma frase:

> Se a abstração exige mais esforço para ser compreendida do que o problema que ela esconde, alguma coisa merece ser revista.

O esforço é mensurável em prática, não em sentimento: quantos arquivos o leitor precisa abrir para chegar ao comportamento; quantos nomes novos precisa aprender que não existem no domínio; quantas decisões ele precisa *desconfiar* que a abstração tomou por ele. A interface `Cobrador` cobra um nome (que o domínio já tinha) e devolve doze mecanismos. Bom negócio. Agora o caso degenerado, que todo mundo reconhece de memória própria:

`ICobradorDePagamentos`, interface com um método, implementada por `CobradorDePagamentosAdapter`, que delega integralmente para `CobradorDePagamentos`, obtido via

`CobradorDePagamentosFactory` com um único `create()`. Quatro arquivos. O comportamento real, a chamada HTTP, está no quarto, e só chegou lá por uma cadeia de repasses que não filtra, não valida, não adiciona nada. O leitor que quer entender a cobrança percorre três indireções vazias e um mecanismo.

A pergunta que desmonta essa arquitetura: *o que ela deixa de fora da minha cabeça?* Nada. Cada camada esconde um mecanismo que nunca esteve na frente de ninguém, o adapter esconde uma classe que ele mesmo não adiciona, a fábrica esconde um construtor sem argumentos. A abstração prometeu liberar atenção e não liberou; ela é o mecanismo que o leitor agora precisa carregar. É o capítulo 2 na escala da camada: complexidade acidental pura, juro sobre dívida que ninguém contraiu. Uma imagem que ajuda a nomear: é alugar um depósito para guardar a chave do depósito.

E vale registrar a origem honesta dessas camadas, porque raramente alguém as projeta por maldade: quase sempre cada uma foi *a abstração certa* no dia em que foi criada, o dia em que o mecanismo escondido era real e estava na frente de alguém. O adapter nasceu quando o código chamava o SDK antigo direto; a fábrica nasceu quando existiam três construções diferentes. A degeneração não foi decisão, foi sedimentação: as razões partiram, os arquivos ficaram. O capítulo 4 chamou de regra órfã; aqui tem nome de camada.

Quando abstrair: o que existe do outro lado do nome

A heurística mais citada: "espere a terceira repetição antes de extrair", é melhor do que o dogma em que virou, mas continua sendo regra de *sintoma*: a repetição sinaliza que algo pediu para ser nomeado, não nomeia sozinha. A pergunta que decide é outra, e é a pergunta deste livro inteiro aplicada à camada: **o que existe do outro lado do nome, um conceito do domínio, ou uma conveniência do código?**

`Cobrador` nomeia algo que existia antes do software: provedores de cobrança são intercambiáveis por contrato de negócio, e a interface registra essa intercambiabilidade, que é *do domínio*, não nossa. `ICobradorDePagamentos` nomeia uma classe que não sabia conversar com outra classe. O primeiro sobrevive à troca de implementação porque a implementação era o acessório; o segundo morre com ela porque era só ela.

Daí a posição que defendo, com a ressalva de que ela é uma posição: a primeira implementação de uma interface não desqualifica a interface, às vezes o contrato é o descobrimento do domínio, e escrevê-lo antes do segundo uso é engenharia, não especulação. O que desqualifica é o contrato não *verificar* nada hoje enquanto *promete* algo para amanhã. Uma abstração que hoje não fecha nenhum caso, não checa nenhuma regra e não impede nenhum erro, e cuja única justificativa é uma segunda implementação hipotética, é um empréstimo sobre um futuro que cobra juros no presente. O teste sobrevive à abstração? A seção seguinte mostra o que "verificar" pode significar, com camadas e tudo.

O contrato: o que a abstração promete, além da forma

Uma interface sem contrato é um formulário. `Double area()` diz que existe um método com esse nome e esse tipo, não diz o que ele garante, o que exige, nem o que nunca pode violar. A tradição que formalizou essa lacuna é verificada e citável: Bertrand Meyer descreveu o *Design by Contract* a partir de 1986, para a linguagem Eiffel, com a tríade que virou vocabulário da área: **pré-condições** (o que o chamador garante antes de ligar), **pós-condições** (o que a rotina garante ao devolver), **invariantes** (o que vale sempre, em qualquer ponto de observação). O valor histórico da ideia não depende do Eiffel nem da sintaxe: ela nomeia a diferença entre "esta função aceita um `Double`" e "esta função aceita saldo *não negativo* e devolve saldo *incrementado*, preservando *titular intacto*". A primeira frase é tipo; a segunda é contrato.

O ponto onde a maioria das discussões sobre abstração escorrega é que "contrato" não é uma coisa, é três coisas, vigiadas por três guardas diferentes, com custos e confianças radicalmente distintos. Vale montar a mesa com as peças medidas deste livro.

Contrato verificado pelo compilador. É o mais forte porque não depende de ninguém lembrar. Em Kof, a interface é cobrada na implementação: a sonda deste capítulo declara `class Quadrado implements Shape` sem implementar `area()`, e o `kof check` responde, medido nesta máquina, hoje:

```
p.kf:11:1: error: class 'Quadrado' does not implement method 'area' of
interface 'Shape' [SEM043]
```

O contrato de *forma*, quem promete `area()` tem `area()`, é verificação, não revisão. E a composição sobre contrato funciona como se espera: `ComForma` guarda um campo `Shape`, recebe `Circulo(2.0)` no construtor, delega `forma.area()`, medido, imprime `25.12` (a área dobrada de raio 2). O chamador pensa em capacidades, o compilador fiscaliza as promessas, e o mecanismo fica em um lugar por implementação. É o capítulo 1 operando no nível de componente: a estrutura impede o que não deveria existir.

Agora a letra miúda, porque o contrato compilado tem fronteira e o livro não vai fingir que não: o que Kof verifica é *presença e assinatura*, não *termos*. Não existe no idioma medido uma forma de declarar "pré-condição: saldo não negativo" que o compilador leia. O `Double area()` aceito por SEM043 é uma promessa de forma; o conteúdo, o que o método *faz de verdade com o que recebe*, escapa para as outras duas camadas. É a distinção do capítulo 6 vista de outro ângulo: a linguagem vigia o que é expressível na linguagem; o resto é nosso.

Contrato verificado em runtime. Pré e pós-condições que não cabem no tipo viram checagem explícita no ponto de entrada, em Kof, a ferramenta disponível é a exceção-String medida no capítulo 1:

```
class Conta {
    String titular
    Double saldo
    public constructor(String titular, Double saldo) {
        if (saldo < 0.0) { throw "invariante violada: saldo inicial negativo" }
        this.titular = titular
        this.saldo = saldo
    }
}
```

A invariante "saldo nunca negativo" passa a existir como decisão no código, não como costume na documentação, e o `throw` na fronteira do construtor é o contrato se defendendo. O preço, na moeda do capítulo 2: roda em produção, no caminho do usuário, e só pega quem executa o caminho errado. E o preço específico de Kof, com a ressalva de honestidade que a paridade exige: a checagem acima é medida no JVM; no alvo nativo, a documentação registra que o primeiro `catch` captura tudo, sem despacho por tipo, e não há stack trace: *documentado, não re-medido nesta produção contra o backend nativo*. O contrato de runtime é real; em três backends ele não é o mesmo contrato. Quem escolhe a arquitetura multi-backend precisa ler a tabela de paridade como quem lê especificação de peça, não como rodapé, e o capítulo 9 leva essa exigência até o fim.

Contrato social. O que sobra quando o tipo não alcança e o runtime não vigia: a convenção documentada, o nome que comunica, o comentário que diz "chame `fecharLote()` antes de `liquidar()`". Não é lixo, é a camada onde moram os termos que *ainda* não têm verificação; um sistema sem contratos sociais é um sistema onde tudo já foi mecanizado, e nenhum domínio grande o bastante para importar chega lá. O problema do contrato social é o problema de toda regra social (capítulo 4): ele decai em silêncio. A documentação que não é cobrada por ninguém envelhece mentindo, e o leitor não tem como saber qual parágrafo ainda é verdade. A disciplina de engenharia não é eliminar essa camada, é tratá-la como fila: *tudo o que aparece aqui repetidamente é candidato a migrar para baixo*, para o runtime ou para o compilador, e o que não migra é porque alguém decidiu o custo.

A pergunta operacional da seção fecha: para cada termo do seu contrato, forma, pré, pós, invariante, sequência, idempotência: *qual guarda o vigia hoje?* Termo sem guarda é contrato de boa-fé, e boa-fé é recurso finito de equipe.

Composição e herança: o que cada uma carrega

A escolha clássica da abstração orientada a objetos costuma ser apresentada como preferência moral: "prefira composição a herança", e moral é o lugar errado para guardar uma ferramenta. As duas compõem comportamento; o que muda é *o que gruda*.

A herança compõe *implementação*: o filho recebe a estrutura do pai, incluindo o mecanismo que ele não pediu e as decisões que ele não pode desfazer. O critério de novo é o esforço de compreensão: *quanto o filho precisa saber sobre o pai para estar correto?* Quando a resposta é longa, o "reuso" é acoplamento com nome

bonito, e o capítulo 6 deu o exemplo exato de onde isso cobra: `sealed` com `extends` funciona bem *porque* o contrato da hierarquia é fechado e verificado (SEM081 vigia a exaustão); a mesma herança em hierarquia aberta acumula conhecimento tribal sobre o que cada ancestral faz por trás. E há o caso documentado que nenhum manual de herança menciona: em Kof, a paridade registra que `super.metodo()` tem gap no backend nativo (SUP001): *documentado, não re-medido aqui*, o que é a lição da seção anterior em forma de mecanismo: o contrato que depende do comportamento do pai precisa ser verdadeiro em três backends, e não é o filho que decide.

A composição sobre contrato gruda menos: `ComForma` não sabe *quem* é seu `Shape`, só o que ele *promete*, e a promessa é o que SEM043 checa. O preço é honesto também: a delegação tem um custo de navegação (o leitor persegue o campo até a implementação real) e a estrutura fica mais verbosa para comportamentos genuinamente comuns. Este livro não vai cometer o erro que denuncia: "sempre composição" é regra sem porquê, e regra sem porquê é a matéria-prima do capítulo 4. O critério continua o mesmo, aplicado à escolha: o que a herança deixa de fora da cabeça do filho, e o que a composição obriga o leitor a carregar?

Reutilização real e o empréstimo da generalização

Falta o irmão mais velho da abstração prematura: a generalização. O `Cobrador` que aceita "qualquer tipo de pagamento, de qualquer moeda, por qualquer canal, com regras configuráveis" *antes* de existir o segundo caso concreto. Ele quase nunca nasce do domínio; nasce do medo de mudar, e o medo de mudar é a emoção menos confiável para projetar interfaces, porque cobra na moeda mais cara do capítulo 3: um vocabulário inteiro de conceitos que ainda não existem.

A distinção prometida na introdução, reutilização real × generalização prematura, tem teste simples: *a generalidade é cobrada por casos que existem, ou paga por casos que a imaginação inventou?* A interface `Cobrador` com duas implementações reais é reutilização: o contrato fecha dois mecanismos que já divergiam. O `Cobrador` com um parâmetro `Map` de "configurações extensíveis" é empréstimo: você está pagando hoje, com juros de complexidade, por uma flexibilidade que ninguém pediu e que provavelmente virá com formato diferente do imaginado. A regra prática que uso, sem transformá-la em mandamento: generalize o *contrato* cedo, nomear a capacidade do domínio é barato e estabiliza o vocabulário, e o *mecanismo* tarde. O que muda devagar merece estrutura; o que muda rápido merece apenas um lugar onde morar.

O que fica para o próximo capítulo

A abstração é gestão de atenção; o contrato é o que permite confiar no que ela esconde; e a força de um contrato está no guarda que o vigia, compilador, runtime, ou a boa-fé de quem leu a documentação. A escolha de camada é a decisão de engenharia; o esquecimento dela é a burocracia.

O capítulo 8 pega o assunto pelo lado do vigia mais forte e o trata como o que ele é: não um oráculo, não um portão, uma *interface de usuário*. As mensagens que o compilador escreve são a experiência de quem não estava na cabeça de quem decidiu, e este livro já carrega dois casos na mesa: o SEM043 que nomeia o método faltante e o SEM081 do capítulo 6 que apontou como faltante um caso que *estava escrito* (issue #817). Um verificador que mente é pior que a ausência de verificação, porque cobra a confiança que ele mesmo quebrou. É sobre construir, e medir, essa confiança que o próximo capítulo trata.

Capítulo 8 — O compilador como parte da experiência

O compilador é o primeiro leitor do seu código, e o único que nunca pula. Ninguém relê todas as linhas de cada pull request com a disciplina de um `kof check`; revisores humanos escaneiam, confiam no nome, pulam o que parece boilerplate. A máquina não. Essa asfixia que reclamamos nos primeiros dias de uma linguagem tipada é, vista de perto, a qualidade mais cara de qualquer processo de revisão: atenção infinita, aplicada a todo o código, toda vez, sem dia ruim.

Este capítulo trata o compilador não como o portão que decide quem passa, mas como o que ele realmente é numa engenharia orientada à intenção: **uma interface de usuário**: a superfície onde a linguagem conversa com a pessoa, educa sobre seus modelos, mente quando está mal projetada e define, pela velocidade do seu ciclo, o ritmo do pensamento de quem constrói. É a tese do capítulo 1 lida pelo lado da ferramenta: se intenção precisa ser recuperável por quem não estava lá, o compilador é o recuperador automático das intenções que a linguagem soube tornar verificáveis, e um péssimo garoto de recados das que ela não soube.

O catálogo do que a linguagem realmente cobra

Toda linguagem tem duas constituições. A declarada, marketing, documentação, "Kof oferece nulabilidade segura", e a efetiva: o conjunto exato de regras que o compilador aplica com código de erro, coordenada e mensagem. A segunda é a única que a equipe vai obedecer de verdade, porque é a única que cobra. Um livro sobre intenção não pode descrever a primeira sem medir a segunda, e a produção deste livro produziu, sem planejar, o inventário exato: os diagnósticos que aparecem nestas páginas foram todos executados contra o compilador do repositório, e formam um retrato mais honesto da linguagem do que qualquer resumo de recursos.

Do que foi medido: SEM037 recusa reatribuição a `val` inferido; SEM038 recusa escrita em componente de `record`; SEM043 recusa classe que promete `implements Shape` e não entrega `area()`; SEM048 recusa `null` literal na declaração; SEM081 recusa switch sobre hierarquia selada sem cobertura exaustiva; SEM095 recusa `Bool?` e aponta o caminho; PARSE085 recusa `fun`; PKG002 recusa dois `main()` no mesmo módulo. Cada código desses é uma intenção que a linguagem assumiu como obrigação de vigilância, o nível mais alto da hierarquia do capítulo 1, o que a estrutura impede. E a leitura do catálogo inteiro com atenção revela algo que nenhum resumo revela: onde a constituição efetiva termina. A nulabilidade vigia o retorno e o parâmetro mas recusa o literal (capítulo 6); a imutabilidade vigia a forma inferida e dorme na anotada (issue #816); a exaustão vigia os casos com vínculo e mente sobre os sem (issue #817). A constituição efetiva tem emendas abertas, e é ela, não a declarada, que merece a confiança do engenheiro, calibrada caso a caso, medição a medição.

A mensagem de erro é um artefato de projeto

Se o compilador é uma interface, a mensagem de erro é a tela onde essa interface ganha ou perde o usuário. E aqui vale uma posição assumida, sem estudo por trás e sem pedir desculpas por não ter: **mensagens de erro merecem mais trabalho de design do que quase tudo que construímos, porque são o único documento da linguagem que o leitor consulta no momento em que precisa aprender.** O resto da documentação é lido por quem já decidiu ler; a mensagem de erro é lida por quem estava construindo outra coisa e foi interrompido. A qualidade pedagógica dela não é polidez, é função estrutural.

O catálogo medido permite classificar as mensagens da Kof 0.6.1 em três qualidades, e as três existem na linguagem que este livro usa:

A que ensina. A melhor mensagem que já vi neste projeto de escrita é o SEM095. Quando alguém escreve `Bool?`, um booleano com três valores, o erro que toda equipe comete uma vez, o compilador não diz "tipo inválido". A medição registrou: *"'Bool' has two values; … use 'Troolean'"*. A mensagem nomeia o equívoco conceitual (você pediu um estado que este tipo não tem), explica por que a recusa faz sentido (dois valores são dois valores), e aponta a porta de saída pelo nome (o tipo de três valores existe, chama-se assim, use). É uma mensagem que ensina o modelo mental da linguagem no exato momento em que o leitor provou não tê-lo. Uma equipe deveria tratar cada erro recorrente dos seus sistemas com esse padrão: nomear o equívoco, explicar a restrição, apontar o caminho.

A que localiza. SEM043 devolve coordenada (`p.kf:11:1`), a classe, o método faltante e o contrato violado: *class 'Quadrado' does not implement method 'area' of interface 'Shape'*. A frase é a revisão; não preciso abrir quatro arquivos para saber o que a máquina sabe. E a experiência moderna dessa tela não é só humana: `kof check --json`, medido nesta rodada, devolve `{"success":true,"filesChecked":1,"errorCount":0,"diagnostics":[]}`, diagnóstico estruturado para o editor, o CI, o agente de automação. A mesma constituição efetiva, três leituras: a pessoa, a ferramenta, o pipeline. Uma linguagem sem saída estruturada de diagnóstico hoje é uma linguagem que escolhe ser lida por metade do seu público.

A que mente, e a que omite o modelo. Issue #817, aberta por este livro: o caso sem vínculo dentro de switch selado faz parse, e o SEM081 responde que aquele subtipo *não está coberto*, apontando para uma linha escrita dois centímetros acima. Uma mensagem que localiza errado é pior que uma mensagem vaga: a vaga mantém o leitor desconfiado e procurando; a precisa e falsa o manda consertar o que está certo. E o PKG002: *"module has 2 main() functions; expected exactly one"*, é a variante da omissão: tecnicamente verdadeiro, pedagogicamente mudo. Ele não diz que diretório é módulo em Kof; pressupõe o modelo mental cuja ausência causou o erro. Quem já tem o modelo resolve em dez segundos; quem não tem aprende que a linguagem tem uma regra invisível, da pior forma possível, pela exceção. O critério que eu aplicaria a qualquer mensagem de erro da minha linguagem e de qualquer sistema que eu construa tem três deveres: **localizar com verdade, explicar o modelo violado, apontar a saída.** Um dos três faltando, a mensagem é um bug de UX com código de erro.

A janela para a máquina (e o que ver por ela)

Há uma experiência de compilador que raramente é nomeada porque a maioria das ferramentas esconde: poder olhar por baixo. `kof inspect`, medido nesta rodada, imprime o tamanho do que você escreveu em termos da máquina: uma tabela de operações por função, contagem de classes, bytes de bytecode, e quantas operações o otimizador removeu. Rodei num exemplo pequeno com `listOf(1, 2, 3)` e um `reduce`: a tabela mostrou `zipPairs` com 37 operações e três classes `Pair` no meu programa de cinco linhas.

Duas lições honestas, uma sobre a máquina e uma sobre este livro. A da máquina: abstração de biblioteca tem corpo: `reduce` não é mágica, é código de outra pessoa que compila junto, e eu posso *ver* o corpo sem pedir permissão ao autor da biblioteca. É o capítulo 2 em modo observável: a complexidade acidental da conveniência, medida em vez de sentida, disponível para decisão em vez de fé. A do livro: a mesma tabela registrou *"ops removed by optimizer: 0"*, zero. Publicar isso é mais útil que escondê-lo: não é um escândalo, é um dado; um otimizador que não removeu nada num programa de cinco linhas pode ser exatamente o que deveria fazer, e pode ser a fronteira do que ele faz hoje. O que este capítulo recusa é as duas reações preguiçosas ao número: "Kof não otimiza nada" (uma amostra não é um comportamento) e "Kof não precisa otimizar" (a tabela existe justamente para ninguém ter que acreditar em nada). O compilador como experiência é isto: a porta que abre para a máquina responder por si, em vez de o time responder por fé.

E a velocidade dessa porta é parte da experiência, sim, sem que isso faça do livro um benchmark: as latências medidas nesta máquina para o ciclo completo de um exemplo pequeno, da ordem de meio a dois segundos no alvo JVM, um segundo e meio no nativo, seis no JavaScript (ordens de grandeza do ambiente, não números de publicação; o capítulo 17 exigirá metodologia quando números importarem), explicam uma diferença qualitativa que quem programa já sentiu na espinha: um ciclo de meio segundo permite *tentar*; um ciclo de dez segundos obriga a *prever*. A frequência com que um compilador responde define quantas hipóteses um engenheiro consegue checar por hora, e hipóteses checadas são o que separa projeto de aposta. Quando um time diz que "o build lento está machucando a arquitetura", não é metáfora de conforto: está dizendo que o orçamento de verificação encolheu, e que as decisões vão migrar silenciosamente da evidência para a intuição.

O laço: check, run, test, e o que o laço não vê

A experiência do compilador não é um comando, é um laço, e o CLI da Kof mede os três elos dele. `check` valida o código sem emitir nada, o portão de qualidade barato, o que o CI roda em cada commit, e cujo `--json` alimenta as ferramentas do capítulo anterior. `run` compila e executa, o laço de descoberta, onde o programa encontra a realidade. E `kof test`, medido nesta rodada: um programa que roda e termina bem vira `PASS p.kf` no relatório `1 passed, 0 failed`; um que lança exceção vira `0 passed, 1 failed`, com o stack trace do substrato JVM impresso no caminho, `KofJvmMain.main` e tudo. O critério de verdade do harness é o código de saída: o programa é o teste. É uma escolha minimalista, sem asserções dedicadas, sem relatório estruturado por caso, e o capítulo 16 terá o que dizer sobre o que esse minimalismo ganha e o que ele deixa na mesa por não separar "rodou sem exceção" de "provou o comportamento". Por ora, o ponto

deste capítulo: o laço fecha. Escrever, checar, rodar, testar, quatro comandos, segundos cada volta, nenhum ponto de fé obrigatória.

Falta o último, e é o que o título deste capítulo promete e o livro inteiro exige: **o compilador é parte da experiência, não o fim dela.** Passar no `check` é a condição mais barata que o código pode satisfazer, é a máquina confirmando que você não violou a constituição efetiva, aquela do catálogo com emendas abertas. #816 é a cicatriz que prova a regra: um `val` anotado reatribuível passa em todos os checks e quebra em produção, porque o compilador vigia o que é expressável na linguagem, e alguém escreveu uma forma onde a vigilância não foi ligada. Os termos do contrato que não cabem no tipo, saldo não negativo, lote fechado antes da liquidação, a idempotência que ninguém declarou, continuam órfãos de verificação até que runtime ou arquitetura os adote (capítulo 7). E a correção do programa, o que ele faz com os dados do mundo, nunca esteve, em nenhuma linguagem, no escopo do compilador.

A postura que este livro defende com o capítulo inteiro cabe numa frase e vale para qualquer verificador automático, do `tsc` ao pipeline do time: **confie no compilador, e confira o compilador.** Ele é o leitor que nunca pula; você é o engenheiro que lê a constituição efetiva, sabe onde ela dorme, e trata cada mensagem de erro, sua ou da máquina, como o artefato de projeto que ela é.

Próxima parada

O compilador é a janela para a máquina da linguagem. A interoperabilidade é a janela para as máquinas das outras, e a decisão arquitetural que ela esconde: quando um sistema que compila para três mundos promete liberdade, o que exatamente ele garante em cada um? A tabela de paridade, os gaps codificados, e o custo real de dizer "rode onde quiser", capítulo 9.

Capítulo 9 — Interoperabilidade e liberdade arquitetural

A biblioteca que exporta o relatório em Excel existia há quinze anos, tinha correções de formato que nenhuma equipe de domínio vai reescrever em um trimestre, e era Java. O time que avaliou Kof para o serviço de cobrança tinha, diante de si, a pergunta que decide o futuro de qualquer linguagem nova em ecossistema estabelecido: *aquela biblioteca entra no meu sistema hoje, ou eu escolhi uma linguagem que me obriga a escolher entre o relatório e o serviço?*

A resposta curta é a que este capítulo desenvolve: interoperabilidade não é um recurso da linguagem, como `match` ou genéricos. É uma **decisão sobre onde ficam as fronteiras do sistema**: quais partes do mundo externo a linguagem aceita tratar como cidadãs, a que preço, e o que ela finge sobre esse preço quando prefere não falar dele. E a versão multi-target de Kof acrescenta à pergunta uma segunda camada, que a retórica de portabilidade quase sempre esconde: quando o *mesmo* código roda em três máquinas, qual contrato é verdadeiro em cada uma delas?

O que está provado na mesa

As medições deste capítulo foram refeitas hoje, nesta máquina, com o CLI do repositório (versão 0.6.1; o binário se identificando como 0.5.0-beta, a ressalva de versão acompanha todo número deste livro). O programa é o mesmo nos três casos:

```
main() {
    println("tres alvos")
}
```

`--target jvm` imprime `tres alvos`. `--target native` imprime `tres alvos`, e no caminho mostra o que promete: `NativeBackend: assembling .../Main.s`, o programa descendo a assembly de verdade, não a um interpretador com nome de compilador. `--target js` imprime `tres alvos`. Três alvos, um código, medido. A promessa de implantação, serviço no JVM onde vive o ecossistema corporativo, binário nativo onde não há JVM para instalar, JavaScript onde o navegador é o campo de batalha, é real, e este livro não vai fingir que é trivial: a maioria das linguagens que tentou esse jogo ou morreu nele ou virou três linguagens com sintaxe parecida.

O interop Java pelo caminho idiomático também foi medido hoje:

```
import java.util.ArrayList

main() {
    var lista = new ArrayList<String>()
    lista.add("Kof")
    lista.add("legal")
    println(lista.size())
}
```

Saída: 2. Import, construtor genérico, chamada de método, a biblioteca de quinze anos entra no sistema hoje, sem wrapper, sem reescrita, sem a reunião sobre "como vamos exportar Excel". A premissa documentada (`learn/21`) é a certa para explicar o resultado: Kof gera bytecode JVM padrão: V21, com exception table real; o programa Kof *é* um programa JVM, e o que roda ao lado dele é o ecossistema inteiro da plataforma.

E a borda, medida, porque liberdade sem fronteira é folheto: `java.util.ArrayList<String>()` escrito sem `import`, o nome qualificado cru, devolve `Undefined variable or type: 'java' [SEM011]`. A documentação acerta neste ponto específico: o caminho é `import` + nome simples; o receiver qualificado direto não é idioma. Uma borda explícita e um erro honesto é exatamente o que o capítulo 8 pediu, a mensagem localiza, e a restrição é do desenho, não um mistério. O que *não* está medido por esta produção e o texto diz sem maquiagem: jars de terceiros via `kof deps --deps` (a `learn/21` registra medições verdes de BouncyCastle feitas em 01/10 por terceiros, confiáveis, datadas, não re-feitas aqui), e os comandos `translate`/`migrate`/`compare` do CLI, documentados no usage e nunca exercitados por este projeto. Ponte existe, pedágio desconhecido: antes de atravessar, meça.

A frase proibida

Aqui mora o ponto onde a interoperabilidade vira questão de intenção, e não de capacidade. O programa Kof roda na JVM, gera bytecode, chama `ArrayList`. Daí a frase que circula em conversa de corredor: *"Kof usa as exceções do Java."* A frase é falsa, e a falsidade é instrutiva.

`throw "não encontrado"` em Kof lança uma **String**: o idioma central de erro da linguagem, medido no capítulo 1. O que o JVM vê quando esse código roda depende de como o compilador decide materializar a String, e o que o código Java ao lado captura com `catch (IOException e)` não é o que o código Kof ao lado captura com `catch (String e)`. A compatibilidade de *bytecode*, o programa roda na plataforma, liga com as bibliotecas, obedece à máquina virtual, não implica compatibilidade de *semântica*, o que seus erros significam, quem os pega, o que acontece quando um Java deixa vazar um `NullPointerException` para dentro de um `try` Kof. As duas perguntas têm respostas diferentes, e a primeira é a que os folhetos respondem.

Por que isto importa tanto quanto parece? Porque a frase errada compra a decisão certa com o dinheiro errado. O time que acredita em "exceções do Java" importa a biblioteca de Excel achando que o contrato de erro dela é o mesmo do seu código, e na primeira falha de formato, o `catch (String e)` que devia pegar tudo pega outra coisa, ou nada. O time que lê a distinção importa a mesma biblioteca *sabendo onde a junta*

fica: envolve a chamada em um adaptador que traduz `IOException` em `String` no ponto exato onde os dois mundos se tocam, e o contrato da fronteira vira código, não esperança. É a segurança oferecida pela linguagem e a exigida pela arquitetura (capítulo 6) reencontradas como o que sempre foram: a mesma distinção vista do lado da interoperabilidade. Liberdade arquitetural real é saber exatamente onde ela termina; a versão que ignora as bordas não é liberdade, é boleto.

Três máquinas, três constituições

A promessa multi-target merece o mesmo tratamento dado ao interop: ler a constituição efetiva, não a declarada. Kof documenta seus gaps de paridade, e a tabela existe, codificada por código de gap, datada. É o documento mais honesto e mais perigoso do repositório. Mais honesto porque não esconde; mais perigoso porque ninguém o lê como lê especificação. Alguns exemplos do que a tabela registra:

`kof.ui` renderiza apenas no alvo JavaScript; em JVM e nativo, é *no-op*, o botão que não aparece não falha, simplesmente não está lá. `super.metodo()` tem gap registrado no nativo (SUP001). JSON com `decode<List<Record>>` tem gap no nativo (JSN004). FFI `extern`: escalares funcionam nos três mundos, callbacks com String na ABI são o gap FFI001, e o alvo Script recusa `extern` por decisão. Criptografia: várias primitivas só existem na JVM. Cada linha é uma frase que a sintaxe diz uma vez e a máquina executa de três jeitos.

O capítulo 8 chamou catálogo de diagnósticos de "constituição efetiva" da linguagem num alvo. A tabela de paridade é a constituição efetiva *entre* alvos, e a leitura correta é arquitetural: **a interseção dos contratos é o único lugar onde "rode em qualquer lugar" não é slogan.** Tudo o que está na interseção, o `println` deste capítulo, os tipos, o narrowing, o `sealed`, você pode escrever uma vez e confiar nos três mundos. Tudo o que está fora é decisão por alvo, e decisão por alvo tem que ser *dita em algum lugar visível*: na fronteira do sistema, no adaptador, no build que escolhe o target. Um time que escreve contra a interseção e trata as diferenças como exceções nomeadas está usando a liberdade; um time que escreve contra o alvo mais rico e descobre os gaps em produção está pagando por ela.

E as duas issues abertas por este livro (#816, #817) acrescentam o aviso final que a tabela não contém: mesmo *dentro* de um alvo, a constituição efetiva tem emendas abertas, a garantia que o `val` anotado promete e não entrega, o caso que o parser aceita e o exaustor nega. A regra que fecha o parágrafo vale multiplicada por três máquinas: **meça onde você implanta.** A documentação de paridade é ponto de partida, não oráculo; foi escrita por humanos que mediram em alguma data, e a data está lá justamente para ser consultada.

O que a interoperabilidade não é

Duas negativas, porque os dois erros são comuns e opostos.

Interoperabilidade não é licença para importar tudo. Cada `import java.util.ArrayList` é um contrato com o versionamento *de outra* plataforma: a biblioteca evolui com as regras dela, quebra com as regras dela,

e a fronteira entre o tipo que o Kof vê e o objeto que o Java entrega é onde esse atrito cobra. O adaptador que traduz, no ponto de contato, com o contrato de erro explícito, não é burocracia; é o preço de vizinhança, e a alternativa não é morar sozinho. Uma arquitetura que importa dez bibliotecas Java "porque funciona" acumulou dez fronteiras sem guarda.

Interoperabilidade também não é justificativa para reescrever tudo. O `translate` do CLI: Java para esqueleto Kof, documentado, não exercitado por esta produção, existe para migração, e migração é ponte: o código traduzido é ponto de partida para repensar o modelo com as ferramentas da nova linguagem (o `sealed` que faltava, o `String?` que diz a verdade), não destino final. Sistema re-escrito linha a linha em nova sintaxe é o mesmo sistema com dois históricos de bug.

Entre os dois erros, o critério é o de sempre neste livro: a decisão se paga pelo problema que resolve. A biblioteca de Excel resolve o problema do relatório com quinze anos de correções, importa-se com adaptador. O framework inteiro que só existe do outro lado importa uma fronteira por dia, e a pergunta "por que esta linguagem?" merece resposta melhor que "porque o framework".

Fim da Parte II, o material e o que ele custou

Os cinco capítulos desta parte percorreram a linguagem como meio de pensamento, e cada um terminou no mesmo lugar incômodo: o meio não é neutro, mas também não é destino. A sintaxe de Kof distribui atenção como qualquer escolha de forma, o `class X(...)` que mente por sotaque, o `fun` recusado que poupa uma guerra (capítulo 5). Os tipos carregam intenção até onde a máquina vigia, e a vigilância tem turnos vagos: `val` anotado, `?.` ausente, SEM048 de intenção não documentada (capítulo 6). Os contratos têm três guardas de confiança decrescente e custo crescente, e a disciplina de engenharia é migrar termos para cima, não para baixo (capítulo 7). O compilador é interface de usuário com deveres pedagógicos, e a mensagem que mente custa mais que a ausência de mensagem (capítulo 8). A interoperabilidade decide onde o sistema termina, e a multi-target cobra o contrato por máquina, não por código (capítulo 9).

O que a Parte II estabeleceu para o que vem: você agora conhece o custo de cada peça do material, forma, tipo, contrato, verificação, fronteira. A Parte III começa a construir com ele. O domínio volta ao centro do palco, o mesmo câmbio do capítulo 6, o mesmo lote do capítulo 2, agora tratados não pela pergunta "a linguagem expressa?" mas pela anterior, que nenhuma linguagem responde: *o que o problema é, antes de qualquer classe existir*. Capítulo 10: modelar o domínio antes de modelar as classes.

Parte III — Escrevendo software com intenção

Capítulo 10 — Modelar o domínio antes de modelar as classes

A Parte II terminou com você conhecendo o material: a forma cobra pedágio, o tipo carrega intenção até onde a máquina vigia, o contrato tem guardas de confiança decrescente, o compilador é uma interface que pode mentir, e a interoperabilidade decide onde o sistema termina. A Parte III começa a construir. E a primeira lição da construção não é sobre nenhuma linguagem: é sobre a ordem em que as perguntas são feitas.

Quando um time se senta para construir o sistema de faturamento de uma empresa que nunca teve sistema de faturamento, a primeira coisa que existe não é código, é um acordo tácito sobre o que o mundo contém. Alguém diz "fatura", alguém entende um documento com valor e data; alguém diz "liquidou", todos entendem que o dinheiro entrou; ninguém pergunta, ainda, quantos significados cabem dentro de "estornou parcial". Esse acervo de palavras e seus significados compartilhados é o domínio, e ele vai entrar no sistema de um de dois jeitos: desenhado, com suas costuras nomeadas e suas regras distribuídas entre verificadores conscientes, ou acidental, herdado da primeira tabela de banco que alguém criou às pressas, das doze branches de `if (tipo == 'E')` que se espalharam quando a segunda exceção chegou, e da memória de quem está lá desde o começo. O modelo existe nos dois casos. A escolha nunca é *modelar ou não*; é *quem modela, você, com o especialista do negócio, ou o acidente, sem testemunhas.*

Eric Evans deu nome e método a essa escolha em *Domain-Driven Design* (Addison-Wesley, 2003), com a tese que este capítulo assume como espinha: a estrutura e a linguagem do código devem corresponder ao domínio do negócio, e o vocabulário que a equipe usa para falar do problema deve ser o mesmo vocabulário que o código fala, a *linguagem ubíqua*. Não vou importar o aparato inteiro da escola; vou usar a ideia central como ferramenta de engenharia, do jeito deste livro: modelar o domínio antes das classes significa responder três perguntas sobre o mundo **antes** de responder qualquer pergunta sobre o código, e depois alocar cada resposta para o vigia certo.

As três perguntas, e a alocação

Primeira: **que estados existem?** Não "que campos tem a entidade", que situações o negócio reconhece como distintas. Para a fatura: emitida, liquidada, estornada. E aqui a conversa com quem entende do negócio vale ouro, porque o código descobre estados que o organograma esqueceu: "estornada parcial" é um estado ou uma liquidação com histórico? A resposta não é técnica, é do domínio, e a pergunta técnica vem depois: *se é estado, nomeie-o; se é variação de valor, modele o valor.*

Segunda: **que eventos movem o sistema entre estados, e quais transições são impossíveis?** Emitir cria; liquidar move de emitida para liquidada; estornar move de liquidada para estornada; e o impossível mora aqui, estornada não liquida de novo, liquidada não vira emitida quando o banco estorna por conta própria. Essa lista de transições é a máquina de estados que vai existir de qualquer forma; a pergunta é se ela vai existir no código, num lugar, ou na cabeça de quem atende telefone.

Terceira: **que regras nunca podem ser violadas?** As invariantes: o líquido de uma estornada parcial é o valor menos o estornado; uma liquidação do mesmo valor na mesma data é o mesmo fato; fatura cancelada não aparece em lote. Cada invariante, uma vez escrita, pede a pergunta do capítulo 7: *qual guarda a vigia, o compilador, o runtime, ou a boa-fé?*

A ordem importa menos que o conteúdo, mas a prática que funciona é esta: as três perguntas primeiro, com palavras do negócio, sem teclado; a alocação depois, com o material que a Parte II ensinou a ler. Modelar antes das classes não é desenhar diagrama de classe bonito, é saber o que as classes terão que prometer antes de escolher quem confere.

O domínio vira estrutura, em Kof medido

Aplicando as três respostas à fatura, o material de Kof oferece o que a Parte II catalogou. Os estados são uma hierarquia fechada, a primeira pergunta virou `sealed`, com o espaço de estados explícito e a máquina vigiando (SEM081, medido no capítulo 1):

```
sealed class StatusFatura
class Emitida(Double valor) extends StatusFatura
class Liquidada(Double valor, Long data) extends StatusFatura
class Estornada(Double valor, Double valorEstornado) extends StatusFatura
```

A segunda pergunta, transições, pede uma capacidade nomeada, e a capacidade vira interface, com o contrato de implementação checado (SEM043, medido no capítulo 7):

```
interface ServicoLiquidacao {
    StatusFatura liquidar(StatusFatura atual, Long data)
}

class LiquidadoraHoje implements ServicoLiquidacao {
    StatusFatura liquidar(StatusFatura atual, Long data) {
        switch (atual) {
            case Emitida e: { return Liquidada(e.valor(), data) }
            case Liquidada l: { return l }
            case Estornada es: { throw "fatura estornada nao liquida" }
        }
        return atual
    }
}
```

Repare na distribuição: a transição legal (emitida → liquidada) e a idempotente (liquidada → ela mesma) estão escritas; a impossível (estornada → liquidada) está *recusada*, não por convenção, mas por código que roda. O `throw` é a invariante no vigia de runtime, imperfeito, com os limites de exceção-String que o capítulo 6 registrou, mas no lugar visível onde uma mudança de política encontra um `grep`. E a terceira pergunta tem seu próprio cálculo: o líquido de uma estornada é regra de negócio, e virou função exaustiva, se

amanhã aparecer "estornada em processamento", o compilador lista os switches que mentem sobre o novo mundo:

```
Double valorLiquido(StatusFatura s) {
    return switch (s) {
        case Emitida e -> 0.0
        case Liquidada l -> l.valor()
        case Estornada es -> es.valor() - es.valorEstornado()
    }
}
```

Medido: `valorLiquido(Emitida(100.0))` imprime `0.0`; liquidar e somar imprime `100.0`; liquidar duas vezes devolve a mesma liquidação. O modelo coube no material, quase. O "quase" é a melhor descoberta deste capítulo, e ela não veio de ler a documentação.

O dia em que o material mentiu para o modelo

A segunda invariante da lista: *duas liquidações do mesmo valor na mesma data são o mesmo fato*, parece grátis. Os estados foram escritos na forma compacta com argumentos no cabeçalho; a Parte II ensinou que essa forma gera dados por valor, com igualdade por conteúdo (capítulo 5), e a referência de gramática permite explicitamente `record` com `extends` na base selada. O modelo exige comparação por valor; o material promete; o código confiaria.

O código não pode confiar. A verificação de rotina, a mesma que já tinha virado issue #816 e #817 neste projeto, perguntou ao compilador o que o modelo exigia dele, e a resposta foi uma traição silenciosa:

```
sealed class B
record S extends B(Int x)
main() {
    println(S(7) == S(7))   // medido: false
    println(S(7))           // medido: S@70dea4e
}
```

A forma documentada parseia e roda, com igualdade e `toString` de **identidade**. A variante `class S(Int x) extends B`, que nem está na gramática, também é aceita com a mesma degradação; e a ordem que qualquer leitor de Java esperaria (`record S(Int x) extends B`) é rejeitada com PARSE007. Os controles fecham o quadro: o mesmo `record Ponto(Int x, Int y)` sem pai mantém a semântica de valor (`Ponto[x=1, y=2]`, `==` verdadeiro), a perda acontece exatamente no `extends`, sem aviso, sem erro, sem nota na documentação consultada. A issue #816 protege `val`; a #818, aberta por esta medição, protege o modelo de domínio inteiro: hierarquias seladas de estados-como-valor, o padrão que este capítulo recomenda e o capítulo 6 usou: **não podem hoje depender de igualdade nativa**: aparentemente porque um record JVM real não pode estender classe do usuário, e o compilador escolheu degradar em silêncio em vez de recusar com clareza, a causa provável, não confirmada; o comportamento medido, esse não tem "aparentemente".

O workaround medido é explícito e feio, e é melhor que a traição: comparar campo a campo pelos acessores, que funcionam —

```
Bool mesmoDia(StatusFatura a, StatusFatura b) {
    switch (a) {
        case Liquidada la: {
            switch (b) {
                case Liquidada lb: { return la.valor() == lb.valor() && la.data()
== lb.data() }
                default: { return false }
            }
        }
        default: { return false }
    }
}
```

— medido: `x == y` imprime `false`, `mesmoDia(x, y)` imprime `true`. Duas linhas de verdade contra uma biblioteca de engano silencioso.

A lição vale mais que o bug, e é a tese do capítulo em forma de cicatriz: **modelar o domínio antes das classes inclui modelar as promessas que o domínio exige do material**: igualdade por valor, exaustão vigiada, imutabilidade real, e *medi-las uma por uma antes de confiar*. O modelo disse "estados são valores"; a documentação disse que o material entregava; o material entreguiu identidade. Quem modela primeiro percebe a traição como contradição; quem só escreve classes percebe como mistério, meses depois, num `Set` que não deduplica.

Os dois modos de errar

O capítulo fecha com os dois erros que a disciplina evita, porque cada um tem sua elegância e seu preço.

O **modelo anêmico** é o domínio colonizado pelo banco de dados: `Fatura` com catorze campos espelhando a tabela, um `char tipo` valendo `'E'`, `'L'`, `'S'`, e a máquina de estados diluída em doze serviços que cada um sabem um pedaço das transições. Nada no código diz que estornada não liquida; a regra existe onde? Em algum lugar do capítulo 3, no custo de lembrar de todo mundo. O modelo anêmico não é falta de modelagem; é modelagem feita por outra pessoa, em outra época, com outro material (a tabela), e aceita sem leitura crítica. Se o `char tipo` um dia precisa de um estado novo, o SEM081 não vai avisar ninguém, porque não havia switch exaustivo nenhum, havia `if` solto.

A **catedral** é o erro oposto e simetricamente sedutor: o domínio inteiro modelado antes do primeiro caso existir, quarenta classes, cinco bounded contexts declarados no README, a máquina de estados da máquina de estados, para a primeira feature descobrir que "estornada parcial" era, na verdade, um campo. A catedral confunde modelar com adivinhar, e paga o aluguel do capítulo 2 por afirmações que o mundo ainda não pediu. O critério que a distingue do modelo bom é o mesmo que distinguia a abstração do depósito de chave: o que está sendo vigiado *hoje*, e o que está apenas prometido para amanhã.

Entre os dois, o teste do modelo vivo é a **rastreabilidade da mudança**: quando o negócio muda, o código muda onde o negócio mudou. Estornada parcial chegou? No modelo anêmico, doze arquivos tocam no `char tipo` e algum esquece. Na catedral, três classes e um registro de contexto. No modelo deste capítulo: uma subclasse nova, e o compilador, que é o único revisor que nunca pula, aponta cada switch que precisa encarar o novo estado de frente. O modelo é bom quando o atrito da mudança no mundo e o atrito da mudança no código são o mesmo atrito.

As classes vêm depois, e vêm bem-vindas; elas são o lugar onde o modelo finalmente mora. Mas vêm como representação de algo que já foi dito, com estados nomeados, transições escritas e invariantes alocadas a vigias escolhidos, não a vigias disponíveis. A Parte III inteira trabalha nessa ordem. O próximo capítulo pega a primeira peça que este capítulo deixou pulsando: as funções que movem o modelo, e a briga centenária sobre onde o comportamento deve viver.

Capítulo 11 — Funções, dados e comportamento

`valorLiquido` começou como método de `Fatura`. Na segunda iteração, um consultor explicou com segurança e diagramas que domínio não deve conhecer regra de cálculo, e ela virou função livre num módulo `Regras`. Na terceira, o time de relatórios provou que a função precisava ser substituída por duas interpretações diferentes do mesmo cálculo (a contábil e a gerencial), e ela virou método de novo, agora numa interface. Na quarta iteração, descobri a função no meio de um branch esquecido, reescrita pela terceira vez por quem não achou as outras duas cópias.

Ninguém nesse time era incompetente, e é isso que torna a cena pedagógica: as quatro versões estavam *erradas juntas*. A pergunta "onde deve morar o comportamento?" foi respondida quatro vezes por estética, carisma e moda, e nenhuma vez pelo que ela realmente decide. Porque "onde" não é questão de gosto nem de pertencimento a tribo, é a escolha que determina três coisas verificáveis: **quem consegue ver as invariantes dos dados, o que a máquina confere quando o mundo muda, e até onde uma mudança precisa chegar.** O resto é retórica.

Este capítulo trata a pergunta como economia, não como moral. E usa o material medido: as formas de função de Kof, o corpo de `record` que verifiquei hoje (a gramática previa, ninguém na documentação tinha exercitado), funções de ordem superior com tipos-função, e uma mensagem de erro que merece aplauso.

O material: três casas para a mesma pergunta

Primeiro, o inventário. Kof tem uma só palavra para função, sem `fun`, sem `fn` (o capítulo 5 explicou por que isso é economia e não privação), e três formas medidas de escrevê-la: bloco com `return`, corpo-expressão (`Bool positivo(Int x) = x > 0`), e parâmetros com valor default. A função é cidadã de topo: declara-se fora de qualquer classe, e o módulo é o diretório. Até aí, nada que o capítulo 5 não tenha coberto.

O que medi hoje acrescenta duas peças que a conversa sobre comportamento precisa ter. A primeira: **record com corpo funciona.** A gramática de `classes.pt_BR.md` previa `record-body` opcional; a sondagem confirmou com o compilador instalado:

```
record Ponto(Int x, Int y) {
    Int somaCoord() { return x + y }
}
main() {
    var p = Ponto(3, 4)
    println(p.somaCoord())   // 7
    println(p.x())           // 3
}
```

Repare no detalhe fino, que é onde a linguagem pensa: *dentro* do corpo, o componente é acessado por nome cru (`x + y`); *fora*, é accessor (`p.x()`). O corpo do record é o único lugar onde os dados se apresentam sem

cerimônia, dentro, a classe é o vizinho íntimo; fora, é a interface pública. Um dado com comportamento próprio, com a imutabilidade do capítulo 5 (SEM038) e o comportamento do capítulo... este. A segunda peça: **funções de ordem superior existem, com tipo-função declarado**:

```
Int aplicar(Int x, (Int) -> Int f) { return f(x) }
main() {
    println(aplicar(21, (v: Int) -> v * 2))   // 42
}
```

Com uma borda que o capítulo 5 previa e a sonda de hoje confirmou: o lambda exige a forma nome-primeiro: `(v: Int) ->` funciona, `(Int v) ->` falha com PARSE041. E função nomeada *não é valor*: `aplicar(21, dobro)` é recusado com uma mensagem que vale transcrever inteira, porque é o capítulo 8 acontecendo:

> `dobro is a top-level function, not a value in argument position, pass the call wrapped in a lambda: () -> dobro()`

A mensagem nomeia a categoria do erro (função ≠ valor), explica o porquê, e ensina o remédio. É o padrão SEM095→Troolean de novo: o compilador que educa em vez de acusar. (O remédio funciona, medido: `(v: Int) -> dobro(v)` devolve 42.)

Com o inventário na mesa, a pergunta do título fica concreta: para qualquer comportamento do seu domínio, qual das três casas, dentro do estado, ao lado do estado, acima do estado, paga as três contas?

Casa 1: dentro do estado

O comportamento como método do próprio dado: `Emitida.liquidar(data)` devolve `Liquidada`, `Estornada.liquidar(data)` se recusa. Cada estado conhece suas transições; a pergunta "o que uma emitida faz?" tem uma resposta, no lugar onde a pergunta nasce.

O argumento clássico contra: "assim cada estado precisa saber de todas as operações, adicionar uma operação toca N classes", é verdadeiro e incompleto, e incompleto de um jeito específico em hierarquias seladas, onde o material *muda o balanço*. Num mundo sem `sealed`, métodos vencem a economia da adição de estado (classe nova, métodos existentes herdam/dispatch prontos) e perdem a da adição de operação (tocar N arquivos). Com `sealed` + switch exaustivo, a segunda metade dessa tabela some: adicionar um estado **já obriga** você a tocar cada switch do sistema: SEM081 garante, e o capítulo 10 mostrou isso como virtude. Se o preço de adicionar estado é igual nas duas arquiteturas (o compilador cobra igual), o critério real fica menor e mais honesto: **onde a regra fica visível para quem vai quebrá-la?**

Método no estado responde: quem mexe em `Emitida` vê `liquidar` ali mesmo, com a invariante do próprio tipo à mão, sem precisar conhecer o resto da hierarquia. A função livre responde: quem mexe na *regra*, e regra muda mais que tipo, no negócio, vê todas as transições juntas, lado a lado, comparáveis. As duas

respostas são certas para times que mudam coisas diferentes. O time que ganha novos tipos de estado (um meio de pagamento novo, um tipo de fatura novo) prefere a função. O time que ganha regras (estorno parcial, liquidação escalonada) prefere o método. Ninguém pergunta isso ao escolher a arquitetura, e por isso a quarta cópia de `valorLiquido` existe.

Casa 2: ao lado do estado

`valorLiquido` como função sobre a hierarquia inteira, o capítulo 10 já escreveu esse código, e ele é o exemplar da casa do meio:

```
Double valorLiquido(StatusFatura s) {
    return switch (s) {
        case Emitida e -> 0.0
        case Liquidada l -> l.valor()
        case Estornada es -> es.valor() - es.valorEstornado()
    }
}
```

O que esta forma compra de específico: **a regra existe uma vez, inteira, visível de uma olhada.** O cálculo do líquido não está distribuído entre três métodos que precisam ser lidos em sequência para entender o domínio; ele é uma decisão com três ramos, e a exaustividade vigiada garante que o quarto ramo será cobrado quando existir. Para regra de negócio, que é exatamente o tipo de conhecimento que o especialista quer *ler*, não deduzir de dispatch, a função exaustiva tem uma vantagem que o debate OO×funcional raramente concede: ela é legível como regra, não como colaboração. O auditor não precisa perseguir um método através de três subclasses; o switch é o relatório.

O que custa: a função conhece o interior de todos os estados (acessa `valor` e `valorEstornado` de quem tem), então ela só funciona bem quando os estados são abertos, dados por valor, sem invariantes privadas. Um estado que esconde campos e valida internamente transforma a função vizinha em violadora de encapsulamento, e aí o pêndulo puxa de volta para a casa 1. A escolha não é livre: a visibilidade dos dados decide a arquitetura das operações sobre eles.

Casa 3: acima do estado

`liquidar` como serviço por trás de interface, o `ServicoLiquidacao` com `SEM043` vigiando as implementações, do capítulo 10. Esta casa é reservada para o comportamento que **depende do mundo**: a data de liquidação vem de um relógio que não é seu, o câmbio vem de um provedor que pode falhar, a decisão de aceitar estorno vem de uma política que muda por configuração. Colocar isso dentro do estado contamina o modelo com o mundo; deixar como função livre esconde que existem implementações intercambiáveis. A interface nomeia a capacidade, o compilador cobra o contrato, e o estado permanece o que o modelo dele exige: puro.

O erro simétrico, o mais caro dos três, é morar na casa 3 por padrão: serviço para tudo, injeção para cada regra de três linhas, o `Manager`/`Handler`/`Processor` que virou sinônimo de "não sei onde isso deveria estar, então está aqui". Cada camada de serviço sobre estado puro adiciona indireção sem adicionar contrato verificável, é o depósito do capítulo 7, alugado para guardar a chave do depósito. Se a regra depende só do estado, ela mora com o estado. Se depende da hierarquia inteira, mora ao lado. Só mora acima quando depende do mundo.

Anemia por escolha, anemia por abandono

Com as três casas nomeadas, dá para refinar o veredito do capítulo 10 sobre o modelo anêmico. Lá, a classe-espelho-da-tabela era o sintoma do domínio não modelado. Aqui, a mesma forma, dados puros, funções ao lado, pode ser engenharia deliberada e boa, e a diferença está exatamente nas três contas: **quem vê as invariantes, o que a máquina confere, até onde a mudança chega.**

A anemia por abandono espalha `if (tipo == 'E')` sem exaustão nenhuma, e a máquina não confere nada. A anemia por escolha declara os estados como soma fechada (sealed), centraliza as operações em switches exaustivos, e a máquina confere tudo, o mesmo formato, com o compilador como aliado. Ela vence por escolha quando as operações são estáveis e as interpretações são múltiplas: o mesmo `StatusFatura` precisa ser renderizado, exportado, auditado e somado por quatro times que não podem acoplar suas regras uns aos outros. Cada interpretação é uma função; o estado não conhece ninguém; ninguém conhece o interior de ninguém além dos campos públicos. É a arquitetura do capítulo 7 realizada como economia de dependências.

O critério unificador dos dois casos: a anemia é escolha quando **o fechamento está no tipo** (sealed, exaustivo, vigiado) e abandono quando o fechamento está na memória da equipe. A forma externa é idêntica. É a intenção, de novo, e a intenção, neste livro, é sempre o que a estrutura torna recuperável.

O comportamento como valor, até onde medimos

As funções de ordem superior medidas abrem a quarta possibilidade, que as três casas não cobrem: comportamento *passado como dado*. `aplicar(21, (v: Int) -> v * 2)` não é só conveniência de biblioteca, é o mecanismo por trás de `map`/`filter`/`reduce` (medidos no capítulo 1), de política de retry configurável, de estratégia de arredondamento injetada sem interface. Onde a interface (casa 3) declara capacidade com contrato nomeado e vigiado por SEM043, o lambda entrega capacidade anônima, sem nome, sem contrato além do tipo.

A escolha entre os dois tem a cara de tudo neste capítulo: comportamento com nome, história e implementação intercambiável pede interface; comportamento de uma linha, sem estado, que é parâmetro e não arquitetura, pede lambda. Um time que injeta `IArredondador` para passar `x -> x * 2` construiu um depósito; um time que esconde a política de liquidação do banco num lambda anônimo perdido em linha 340 escondeu um contrato. E as bordas do material, com a honestidade de praxe: função nomeada não é valor

(SEM011 didático, medido); a semântica de captura de variável pelos lambdas, o que exatamente um lambda leva na mochila quando viaja para dentro de um `aplicar`, esta produção não mediu, o dossiê não cobre, e nada aqui depende dela. Onde a promessa não foi verificada, ela não é feita.

Onde o capítulo deixa você

A pergunta "onde mora o comportamento" tem resposta, e ela não é tribo: **o comportamento mora onde a invariante que ele protege é visível para quem pode quebrá-la, e o custo dessa visibilidade é pago na casa cuja matéria-prima é a mudança que mais acontece no seu domínio.** Estado muda mais: método. Regra muda mais: função exaustiva. Mundo muda mais: serviço. Tudo é medível na sua base de código: `git log` responde quem muda mais, e ele não tem opinião sobre escolas.

Falta a peça que nenhuma das três casas resolve sozinha: o que acontece quando o comportamento *falha*, quando o mundo recusa, quando o estado ilegal aparece apesar do modelo, quando a exceção é String e o backend é nativo. O capítulo 12 é sobre erros, falhas e os estados que o domínio jurou impossíveis.

Capítulo 12 — Erros, falhas e estados inválidos

O plantão noturno recebeu três alertas na mesma hora, e a fila de tickets os tratou como a mesma classe de coisa porque a palavra no painel era a mesma: *erro*. O primeiro era um `;` que faltava, o compilador de uma linguagem vizinha pegou antes do deploy, e alguém tinha corrigido de manhã. O segundo era o provedor de câmbio que não respondeu em quatro segundos; o valor voltou depois, o lote seguiu, ninguém precisou de ninguém. O terceiro era uma fatura com `estornada = true` e `valor_liquido = -50`, que não devia existir em nenhum universo, e cujo SQL de correção levou duas horas porque ninguém sabia *desde quando* aquele estado estava lá nem *quantos* mais havia.

Três eventos, três naturezas, três respostas certas radicalmente diferentes. O compilador era o lugar do primeiro; o modelo de domínio era o lugar do segundo; o terceiro era a denúncia de uma mentira, e a pergunta certa não era "quem conserta" mas "quem jurou que isso era impossível". A fila de tickets, ao chamar tudo de erro, cometeu o erro arquitetural que este capítulo existe para desmontar: tratar as três categorias com o mesmo canal.

Erro: eu escrevi errado, e a máquina sabia

A primeira categoria tem dono (eu), um verificador barato (o compilador) e um custo que decresce com a distância até a escrita. O catálogo de diagnósticos que as medições deste livro produziram: SEM043 cobrando o método prometido pela interface, SEM048 recusando `null` na declaração, SEM095 explicando por que `Bool?` é uma contradição e apontando o `Troolean`, SEM081 listando os switches que o estado novo obriga a encarar, é a materialização do princípio do capítulo 1: intenção que a máquina vigia. Cada um desses erros foi impedido de existir; nenhum acordou alguém às três da manhã.

A Parte II inteira pode ser lida como a economia dessa migração: empurrar falhas potenciais para a categoria "erro", onde o verificador é o compilador, o custo é uma linha e a correção é imediata, é o único investimento em qualidade cujo retorno composto é garantido. Uma linguagem que verifica nulabilidade transforma um crash de produção em um erro de build; uma hierarquia selada transforma um esquecimento em uma lista de arquivos a corrigir. O catálogo não é burocracia; é o preço que o futuro cobrava hoje, antecipado.

Mas a migração tem fronteira, e ela é de categoria, não de esforço: **só erros migram para o compilador. Falhas não.** O provedor de câmbio não compila; o timeout não tem tipo; o "não encontrei" é fato do mundo, não defeito da escrita. Toda arquitetura que trata isso como limitação de ferramenta: "se a linguagem fosse mais expressiva, o timeout seria um erro de tipo", está confundindo o mapa com o território. O território inclui coisas que se recusam a obedecer, e o trabalho de engenharia é dar a elas um lugar no modelo, não prometer que um dia o compilador as pega.

Falha: o mundo recusou, e isso estava no plano

A segunda categoria é a mais mal tratada do ofício, quase sempre por um acidente de vocabulário: a palavra "exceção" descreve duas coisas diferentes, o canal de transporte (throw/catch) e o estatuto semântico (inesperado, raro, terminal). Quando o canal e o estatuto coincidem, tudo bem. Quando não, nasce o monstro que todo plantonista conhece: a exceção que todo mundo captura e ninguém lê.

O teste que separa as duas coisas é a pergunta que o chamador faz, não a que o chamante sente: **existe decisão de negócio a tomar no ponto do retorno?** Se para cada resultado do cálculo existe um tratamento diferente e rotineiro, liquidar a taxa, pular o cliente, reagendar o lote —, a falha não é exceção: é *domínio*, e domínio mora no tipo de retorno. O exemplo do plantão, escrito como o modelo exige, medido nesta máquina hoje:

```
sealed class ResultadoTaxa
class TaxaEncontrada(Double valor) extends ResultadoTaxa
class SemTaxaContratada extends ResultadoTaxa
class ConsultaFalhou(String motivo) extends ResultadoTaxa

ResultadoTaxa buscarTaxa(Int clienteId) {
    if (clienteId == 1) { return TaxaEncontrada(5.25) }
    if (clienteId == 2) { return SemTaxaContratada() }
    return ConsultaFalhou("timeout do provedor")
}

String tratar(ResultadoTaxa r) {
    return switch (r) {
        case TaxaEncontrada t -> "liquidar a " + t.valor()
        case SemTaxaContratada s -> "pular cliente do lote"
        case ConsultaFalhou f -> "reagendar: " + f.motivo()
    }
}
```

Saída medida: `liquidar a 5.25`, `pular cliente do lote`, `reagendar: timeout do provedor`. Nenhuma `throw` no caminho. Repare no que o `sealed` faz aqui: os três desfechos do mundo estão no contrato da função, e o switch sem `default` obriga o chamador a *ter uma opinião* sobre cada um, o SEM081 é o preço de ignorar, cobrado na compilação, não no plantão. "Sem taxa contratada" não é anormal para o lote; é um dos estados do lote. Modelá-lo como exceção seria confessar que alguém achou que o cliente sem taxa não existia.

O outro lado do teste: quando a decisão *não* existe no ponto de retorno, o arquivo de configuração corrompido na inicialização, o bug de programação que produziu saldo negativo onde nenhuma regra o permite, transportar o problema até onde alguém pode decidir é exatamente o que o canal de exceção faz bem: não polui o caminho feliz com ramificações que ninguém trata. Os dois canais coexistem em qualquer sistema honesto; o crime é a escolha por inércia. Timeouts como exceções geram catch-all que engole bugs; bugs como valores geram `if (erro != null)` em toda linha que chama algo. O critério, de novo, não é emoção: **quem decide, decide onde?**

O canal de Kof, com a contabilidade aberta

A Parte II já abriu este livro-caixa e o capítulo fecha a partida. O idioma de exceção de Kof é medido e funcional no JVM: `throw "mensagem"`, `catch (String e)`, e a ordem fina do DD-01, verificada no capítulo 1, de o `finally` rodar antes da entrega do valor do `return`, que é exatamente o tipo de contrato que distingue um mecanismo pensado de um empréstimo sintático.

Agora as três linhas que o multi-target cobra, cada uma com seu rótulo de evidência. **Documentado, não re-medido por esta produção:** no alvo nativo, o primeiro `catch` captura tudo, sem despacho por tipo, e não há stack trace (`learn/14`). **Bug aberto no repositório, conhecido e registrado:** lançar algo que não é String gera bytecode JVM inválido (#25 na lista de pendências do projeto), o idioma aceita o erro do programador com uma fragilidade que só o compilador de destino descobre. **Emenda deste livro (#818):** estados selados escritos na forma compacta perdem igualdade por conteúdo em silêncio. As três linhas apontam para a mesma conclusão arquitetural, e ela não é "Kof tem defeitos", linguagens em desenvolvimento aberto carregam pendências declaradas, e é mais honesto conhecê-las numeradas do que descobri-las em produção; a de Java tem seu próprio capítulo histórico sobre exceções que o ecossistema passou a vida tentando domar. A conclusão é que **no Kof de hoje, o canal de exceção é um contrato de um backend só**, e a tabela de paridade deveria dizer isso com todas as letras. Para código que pretende viver nos três alvos, o padrão `sealed`-Result medido acima não é preciosismo funcional: é o único canal cujo contrato não muda de significado por target. Uma `catch` escrita para "timeout" no nativo captura também o índice fora do intervalo, o catch-all como buraco negro acidental, não por escolha de quem programou, mas por definição de backend.

E a nuance medida hoje, que qualifica o workaround da #818: o canal `sealed`-Result funciona *como canal* mesmo com a emenda aberta, métodos dentro dos estados funcionam (`record S extends B(Int x) { Int dobro() { return x + x } }` medido: `42`), os acessores funcionam, o switch exaustivo funciona. O que não se pode fazer, até o fix, é confiar em `==` entre dois estados. O modelo sobrevive à limitação do material; o que muda é o endereço da igualdade, campo a campo, explicitamente, onde antes seria grátis. É a lição do capítulo 10 em execução: quem modelou primeiro sabe exatamente qual peça emprestada o material ainda não entrega, e isola o rombo num lugar só.

Estado inválido: a mentira que sobreviveu ao processo

A terceira categoria não é erro meu nem recusa do mundo: é o modelo e o mundo em desacordo. A fatura com `estornada = true` e `valor_liquido = -50` não foi escrita por ninguém, nenhum construtor do sistema atual produz essa combinação; ela *surgiu*. As três origens conhecidas de todo estado inválido são tão velhas quanto as bancos de dados: o mundo escreveu por fora (a migração de 2019 que não conhecia a regra de 2023, o SQL do suporte, o humano que editou a linha no admin); o tempo mudou a regra debaixo dos dados (o que era válido ontem passou a ser inválido sem que nenhum registro fosse tocado); e a concorrência produziu combinações que nenhuma transação isolada escreveria (capítulo 15 cobra essa conta com juros).

O que o estado inválido denuncia nunca é o dado: é a distância entre a regra declarada e a regra vigiada. O capítulo 7 nomeou os três guardas; o inventário de um domínio maduro é uma matriz de invariantes contra guardas. Para cada invariante: "estornada não tem líquido negativo", "liquidada tem data", a pergunta honesta é: quem a vigia? Compilador (o tipo torna a violação inexpressável, o `sealed` do capítulo 6, na sua melhor forma)? Runtime (a checagem no construtor ou na fronteira, com o custo medido de rodar só quando alguém passa pelo caminho)? Ou social (a regra existe na documentação, no comentário e na memória, e portanto não existe para o migração de 2019)? Estado inválido em produção é sempre, sem exceção, uma invariante que ninguém alocou: a mentira não furou um sistema de vigilância; ela nasceu onde não havia vigilância nenhuma.

Daí a disciplina que fecha a categoria: **toda fronteira por onde dados entram de fora: API, banco, arquivo, mensagem, é ponto de validação obrigatório, e toda invariante descoberta em produção sobe de camada se for possível.** O saldo negativo que apareceu no relatório vira checagem no construtor no mesmo sprint; a checagem no construtor, se puder virar tipo (dois estados em vez de um booleano), vira tipo no próximo. A direção do tráfego é sempre a mesma: do social para o runtime, do runtime para o compilador, porque o custo de vigiar decresce nessa ordem, e a cobertura cresce. Quando não puder virar tipo: "o lote fecha em T+1" é fato do calendário, não do programa —, o runtime é o endereço definitivo, e o contrato social ao redor dele é o que resta documentar com dono e data.

As três categorias, os três canais, uma mesa

O capítulo fecha com a mesa montada, e ela cabe num parágrafo que a fila de tickets do plantão nunca leu. Erro é meu: o canal é o compilador, o custo é a linha, e a meta de engenharia é migrar para cá tudo que migra, nulabilidade, exaustão, contrato de interface, os SEM que este livro mediu. Falha é do mundo: o canal é o tipo de retorno quando existe decisão no ponto de uso, o `sealed`-Result medido, três desfechos, switch sem `default`, opinião obrigatória, e a exceção quando não existe, com a ressalva contábil de que em Kof multi-backend a exceção hoje só merece confiança no JVM. Estado inválido é do modelo: o canal é a validação na fronteira e a invariante alocada em guarda nomeado, e o sintoma em produção é sempre um mapa entre o que foi declarado e o que é vigiado.

Um sistema que confunde as três categorias paga o preço duplo: exceção para falha esperada, que ninguém trata; valor de retorno para bug, que ninguém lê; e invariante social para o que o tipo cobraria de graça. O capítulo 13 pega a categoria que mais produz estados inválidos, a persistência, onde os dados sobrevivem ao código que os validou, a regra muda sem que ninguém toque nos registros, e a transação é o contrato de honestidade entre o instante e o mundo.

Capítulo 13 — Persistência, transações e integridade

A persistência é a única parte do sistema em que as promessas do modelo são feitas a um público que nunca vai ler o código. O compilador lê; o revisor lê, às vezes; o banco de dados não lê nada. A linha gravada às 14h02 sob as regras de ontem vai ser lida às 3h11 da manhã de 2029, por um job que ninguém escreveu ainda, sob regras que ainda não existem, e ela não carrega no próprio disco nenhuma anotação sobre o que significava "válido" no instante em que foi escrita. Toda a arquitetura de integridade existe para compensar essa assimetria: o processo que conhecia as regras morre, os dados ficam.

O capítulo 12 apresentou a fatura de saldo negativo que sobreviveu ao processo que a criou. Este capítulo é sobre o que aquele caso ensina quando perseguido até a raiz: persistência não é um detalhe de implementação do modelo, é o lugar onde o modelo confessa o que realmente garantiu e o que apenas esperou que não acontecesse.

A transação é um conceito do domínio, não do banco

A palavra "transação" chegou à engenharia de software do mundo dos negócios, onde já significava exatamente uma coisa: *um fato que aconteceu inteiro ou não aconteceu*. Se Ana transfere quarenta reais para Beto, não existe o meio-fato, débito sem crédito não é uma transação pela metade, é um roubo com outro nome. A unidade de integridade vem do domínio; o `BEGIN`/`COMMIT` do banco é uma tentativa de dar sombra a essa unidade, e a sombra só coincide com o corpo quando alguém decidiu, no modelo, onde o corpo termina.

O exemplo medido desta máquina faz a ideia ficar concreta, domínio puro, sem banco, sem ORM, sem `kof.db` (que, registro honesto antes que alguém pergunte: a camada de persistência de Kof é marcada como experimental na própria documentação e não foi exercitada por esta produção; nada aqui depende dela, e o raciocínio sobre transação não precisa de material validado porque é sobre o modelo, não sobre o driver):

```
sealed class ResultadoTransferencia
class TransferenciaOk(Double valor) extends ResultadoTransferencia
class SaldoInsuficiente(Double pedido, Double disponivel) extends
ResultadoTransferencia
class ContasIguais extends ResultadoTransferencia

ResultadoTransferencia transferir(Conta de, Conta para, Double valor) {
    if (de.titular == para.titular) { return ContasIguais() }
    if (valor > de.saldo) { return SaldoInsuficiente(valor, de.saldo) }
    de.saldo = de.saldo - valor
    para.saldo = para.saldo + valor
    return TransferenciaOk(valor)
}
```

Saída medida, incluindo o consumidor exaustivo `relatar`: `transferido 40.0`, `recusado: pedido 999.0 tem 60.0`, `recusado: mesma conta`, e o saldo final de Beto, `50.0`. Repare no que a função *é*: a

unidade de integridade do domínio, nomeada. As duas mutações não são duas operações com um laço de sincronia; são os dois verbos de um único fato, e não existe caminho no código que execute um sem o outro. A invariante do construtor (saldo inicial não negativo, medida no capítulo 7) protege a entrada; o switch exaustivo protege a saída; o meio, o instante entre debitar e creditar, é invisível de fora porque a função inteira é o átomo observável.

Agora a pergunta que decide o capítulo: quando esse domínio encontra um banco de dados, quem manda em onde a transação começa e termina? Se a resposta prática for "o ORM, no primeiro flush", o sistema tem duas unidades de integridade e nenhuma delas é a do negócio. A transação de banco virou sombra de um corpo que ninguém desenhou, e sombras se cortam sozinhas quando o corpo não é visível. O serviço que debitava numa chamada, validava num segundo método, creditava na terceira, e o `@Transactional` envolvia tudo "por garantia", é o meio-fato institucionalizado: o domínio nunca declarou onde o fato começava, e o framework escolheu por ele, mal.

(Duas notas de honestidade sobre o exemplo. A checagem de contas iguais via `titular` é um atalho de identidade, duas contas com o mesmo titular não são a mesma conta —, do tipo que a matriz do capítulo 10 pega na pergunta "o que o domínio realmente distingue?"; serve ao exemplo, não serviria à produção. E a igualdade de String que ela usa funcionou nesta máquina para o caso medido, mas igualdade de strings entre objetos distintos é exatamente o tipo de contrato que este livro só declara depois de medir, a lição da #818 valendo para tudo que envolve ==.)

Os dois erros espelhados

O território entre o modelo e a tabela tem dois buracos conhecidos, e eles são espelhados.

O primeiro trata o banco como depósito de objetos. A entidade do domínio ganha anotações de mapeamento até que o ORM saiba do negócio melhor que o próprio modelo; a lazy loading transforma cada acesso em uma pergunta ao banco dentro de um loop que ninguém revisou como loop; a transação passa a ser "tudo o que couber entre o primeiro getter e o flush". O custo, na moeda do capítulo 3: o leitor de uma função do domínio não consegue mais dizer quais linhas tocam o mundo, o `conta.historico()` que parece acesso a memória é uma ida ao datacenter, e a semântica de null do capítulo 6 ganha um novo significado: "o banco não respondeu".

O segundo trata o objeto como linha da tabela, o anêmico do capítulo 10 morando em novo endereço: DTOs espelhando as colunas, o repositório devolvendo "a tabela paginada", a regra de negócio escrita em SQL no relatório porque no modelo não havia lugar onde ela coubesse. Aqui o domínio não é corrompido pelo banco; ele nunca existiu, e o banco é o único lugar onde algo parecido com modelo morou.

O meio honesto entre os dois não é compromisso estético, é contabilidade de fronteiras: **duas representações, uma conversão, um lugar só onde a conversão acontece.** A entidade do domínio vive com suas invariantes vigiadas (capítulo 12); o registro persistido vive com suas colunas e índices; o repositório é a porta (casa 3 do capítulo 11, capacidade nomeada, contrato checado por SEM043 quando o

material permite), não um espelho da tabela nem um armazém de objetos. A pergunta que mantém a fronteira limpa é sempre a mesma: *esta linha conhece o negócio ou conhece o armazenamento?* As duas respostas são legítimas; a terceira, conhecer os dois ao mesmo tempo, é onde os dois buracos se encontram.

Integridade entre fronteiras: o mito do exactly-once

A transação local termina onde o commit termina. Mas o sistema do mundo real, o que liquida, cobra, avisa, precisa que fatos cruzem fronteiras: para outro serviço, outro banco, outro continente, e a rede entre eles não tem commit. O teorema é antigo e não se negocia: não existe entrega exatamente-uma-vez sobre um canal que pode falhar depois do envio e antes do reconhecimento. O `exactly-once` que aparece em diagramas é uma política, não uma propriedade do transporte, e a política que funciona tem duas peças nomeadas:

Idempotência: o receptor consegue reconhecer que o mesmo fato chegou duas vezes e tratá-lo uma. Repare que a idempotência *pressupõe* a identidade do fato, e identidade de fato é conceito de domínio: a transferência de Ana tem um número que a torna ela-mesma, e o retry com o mesmo número é o mesmo fato reenviado, não um fato novo. O domínio que não deu identidade aos seus eventos está entregando ao acaso a decisão sobre o que é "a mesma coisa". É o capítulo 10 de novo: quem modela primeiro nomeia a identidade do fato; quem não modela descobre a identidade do prejuízo.

Outbox: o estado e o evento "o estado mudou" precisam ser gravados juntos, e como o banco e o broker não compartilham transação, grava-se o evento *na mesma transação do estado*, numa mesa de saída, e um transporte separado cuida de publicar, aceitando, com a idempotência do outro lado, que publicar duas vezes é seguro. O outbox é a confissão elegante de que a transação global não existe: em vez de fingir atomicidade entre sistemas, transfere-se a atomicidade para dentro da fronteira que a tem (o banco) e se negocia a entrega do lado de fora com repetição segura.

Nada aqui é vocabulário de escola, é o inventário padrão de quem já viu o "pagamento confirmado duas vezes" em produção. O ponto deste livro é outro, e é o de sempre: essas técnicas não consertam um modelo frouxo; elas exigem um. Idempotência sem identidade de fato, outbox sem unidade de integridade definida, retry sem a distinção do capítulo 12 entre falha transitória e recusa de domínio, são bandagens sobre a mesma ausência: alguém precisaria ter respondido "o que deve ser verdade de uma vez?" antes de escolher o mecanismo.

Defesa em profundidade não é redundância

Chega-se então na objeção que todo time já ouviu em revisão: "essa regra já está validada no código, por que repetir no banco?" A resposta do capítulo 2, aplicada: contabilize quem cada guarda ataca. A invariante do saldo negativo existe em três camadas, o tipo que poderia tornar a violação inexpressável (o capítulo 6: o ideal ainda não alcançado pelo material), o construtor que recusa na entrada (medido: `throw` no caminho), o `CHECK` do banco que recusa na última fronteira, e cada uma pega um atacante diferente. O construtor pega o

bug do time. O `CHECK` pega o humano com acesso de SQL às duas da manhã, o script de migração de 2019 que não conhecia a regra de 2023, o concorrente que escreveu por fora (as três origens do capítulo 12, com nomes e endereços). E o tipo pegaria as duas coisas sem o runtime nem o DBA, é por isso que ele é o primeiro candidato de toda invariante, não o último.

Redundância é quando duas cópias guardam o mesmo atacante; profundidade é quando cada cópia tem o seu. O teste para a reunião de revisão: *liste os caminhos pelos quais um dado pode chegar a esse campo sem passar pela sua validação.* Se a lista tem um item só, a segunda checagem é teatro; se tem cinco, cada camada que falta é uma aposta de que o humano não vai fazer o que já fez.

E o tempo, que falta em quase toda discussão de integridade: as regras mudam debaixo dos dados que as obedeciam. O saldo que era válido sob a política de crédito de ontem é inválido sob a de hoje, e a migração que roda `UPDATE ... WHERE` para "consertar" o passado está, na verdade, *reescrevendo a história do domínio sem avisar o domínio*. As estratégias para esse conflito, versionar regras com dados, reinterpretar em vez de reescrever, manter o histórico de fatos e tratar a validade como projeção, têm nomes e escolas; o princípio deste livro cabe sem elas: **integridade é uma política no tempo, não um check no instante.** O dado persistido carrega duas perguntas que o esquema quase nunca responde: *sob quais regras fui escrito?* e *o que aconteceu depois de mim?* Um sistema que não sabe responder não tem histórico; tem acidentalmente, e por pouco tempo, uma tabela que envelhece.

O que a persistência confessa

Fechando o capítulo com a tese inteira num espelho. A persistência é o teste de fogo da intenção em código porque ela torna visível o que o resto do sistema perdoa: um modelo que só existia nas dependências de runtime, uma unidade de integridade que ninguém nomeou, uma invariante que morava na memória de uma pessoa. O banco não lê suas intenções; ele aceita o que passou, e é essa surdez que transforma arquitetura em arqueologia. O domínio que sabe o que deve ser verdade de uma vez, quem o vigia, e o que acontece quando a regra muda, chega ao banco com as respostas prontas; o resto chega com `@Transactional` e esperança.

O próximo capítulo pega a mesma fronteira pelo lado de quem está fora: a API, o contrato com quem não vai ler seu código nunca, não conhece suas invariantes, e tem toda razão de esperar que a resposta signifique o que diz.

Capítulo 14 — APIs e fronteiras de responsabilidade

Um módulo interno tem chamadores. Uma API tem *plateia*. A diferença não é de escala, é de natureza: os chamadores do módulo estão no mesmo repositório, podem ser grepados, podem ser avisados, e, no limite, podem ser corrigidos junto com você numa única refatoração atômica. Os consumidores da API não. Alguns são de outro time, outros de outra empresa, um deles é o script de integração de um cliente que ninguém pode chamar porque é concorrência, e todos, absolutamente todos, vão confiar no comportamento atual da sua API de um jeito que nenhum código interno ousa, porque o código interno pode ser consertado embaixo deles e o contrato publicado não.

Tudo o que uma API expõe vira permanente no instante da primeira chamada de um consumidor que você não controla. Não a decisão arquitetural: o detalhe. A coluna "temporária" que o frontend aprendeu a ler. O `null` que aparece em dois campos e significa coisas diferentes, e que alguém, em algum dashboard, interpretou como sinal. A ordem dos elementos de uma lista que "não tem ordem garantida" mas virou paginação por posição. O código de erro que era log e virou dependência. Nenhuma dessas coisas foi prometida; todas foram *observadas*, e observação repetida por quem não pode te chamar vira contrato por uso. A primeira lição do capítulo é a que dói nos times que descobriram tarde: **você não decide o que é compromisso público; seus consumidores decidem, e eles não leem os seus comentários, leem as suas respostas.**

A fronteira como tradutor de contratos

Uma API não é uma camada a mais do sistema; é o lugar onde dois domínios se encontram sem compartilhar vocabulário. O seu domínio tem `SaldoInsuficiente`; o contrato público tem `422` com um motivo legível. O seu tem `String?` que o narrowing do capítulo 6 obriga a tratar; o JSON da resposta tem um campo que *pode faltar*, e o consumidor, em outra linguagem, com outro sistema de tipos, vai resolver essa ausência do jeito dele, sem você. O ponto exato onde o meu `null` vira o seu problema é o ponto onde o meu domínio termina: é a definição operacional de fronteira de responsabilidade, e ela não é desenhada por diagrama, é desenhada por um pedaço de código pequeno, chato e absolutamente crítico que quase todo time escreve às pressas: o mapeamento.

O mapeamento entre o `ResultadoTransferencia` do capítulo 13 e a resposta do contrato é o exemplo honesto da forma, duas hierarquias seladas, uma conversão total entre elas, exaustividade vigiada nos dois sentidos:

```
RespostaApi aoContrato(ResultadoTransferencia r) {
    switch (r) {
        case TransferenciaOk ok: { return Aceita("TX-" + ok.valor()) }
        case SaldoInsuficiente si: { return RecusadaMotivo("pedido " + si.pedido()
+ " excede " + si.disponivel()) }
        case ContasIguais c: { return RecusadaMotivo("mesma conta") }
    }
    return RecusadaMotivo("invariante violada: resultado desconhecido")
}
```

Medido nesta máquina, no dia em que este capítulo foi escrito, o bloco exato acima, executado: `200 TX-40.0`, `422 pedido 999.0 excede 60.0`, `422 mesma conta`, e o `switch` exaustivo garante que nenhum desfecho do domínio chegue ao contrato sem decisão explícita. Mas a história deste código é o melhor argumento possível para a tese do capítulo, porque a forma *natural* de escrevê-lo, o corpo-expressão com `return switch`, a sintaxe que este livro celebrou nos capítulos 6 e 12: **não funciona**. Ela compila. Ela passa no `kof check`. E o bytecode que ela gera é inválido: a classe recusa carregar com `VerifyError: Bad return type`, porque o tipo de junção entre os ramos que constroem subclasses *irmãs de outra hierarquia* foi alargado a `Object` em vez do supertipo declarado. A reprodução mínima coube em dez linhas; a bissecção do bug isolou os controles (irmãos da mesma hierarquia: ok; extração de campo: ok; um ramo construído e um repassado: ok; o caso cruzado: quebra). A issue está aberta como #819 no repositório da linguagem, com o workaround acima, a forma statement, medida funcionando, documentado.

Eu conto este fracasso em detalhe por três razões, e nenhuma delas é modéstia retórica. A primeira: é a prova empírica de que a fronteira é onde os contratos se testam, o bug não morava em nenhuma das hierarquias, morava na *conversão entre elas*, exatamente onde este capítulo diz que a responsabilidade mora. Padrões que funcionam isoladamente, compostos uma única vez, revelam que ninguém tinha medido a composição. A segunda: `kof check` aprovou o código quebrado, e o capítulo 8 avisou que passar no verificador não é estar correto; desta vez o verificador é o que falha, e a lição da #816 vale em dobro: confie no compilador, e confira o compilador, *principalmente* onde ele parece mais confiável. A terceira: nenhum dos exemplos deste livro entra no texto sem rodar, e quando rodar quebra, o livro conta, a alternativa seria publicar exatamente o tipo de promessa que o capítulo 12 classificou como estado inválido: algo que deveria ser impossível e está lá.

Versionar é dizer a verdade sobre o tempo

Um contrato que vive no tempo precisa de uma política para a mudança, e a maioria das políticas de versionamento é encenação de duas perguntas honestas: *isto que mudou era promessa ou detalhe?* e *quem está contando com o comportamento antigo?* A resposta à primeira não é técnica: `200` virar `201` é tecnicamente idêntico a remover um campo que "ninguém usa", e uma das duas mudanças pode derrubar um cliente. Breaking change não é evento do código; é evento do contrato: é o que quebra a expectativa observável de alguém que você não controla. Quem define se uma mudança quebra é a plateia, não o changelog.

Daí o ceticismo que tenho com o `v2` como solução-padrão, não porque versionar seja errado, mas porque "lança a v2 e depreca a v1" virou o equivalente arquitetural de trocar de assunto. Uma API v2 ao lado da v1 significa manter duas verdades vivas: dois contratos, duas superfícies de teste, dois conjuntos de consumidores, e um funeral adiado para a data em que ninguém mais lembra por que a v1 morreu. Cada `vN` é confissão de que a conversa sobre o contrato falhou uma vez; tratá-lo como vitória de processo é esconder o custo. As alternativas não são mais glamourosas e são mais caras: mudanças aditivas com campos opcionais e semântica estável, capacidade de negociação no pedido, depreciação com *tempo de vida medido*, que é o número que quase ninguém olha: quantos consumidores ainda chamam o campo deprecado na semana em que você planeja matá-lo? Se a resposta é "sabemos?", a sua política de versionamento é torcida, não engenharia.

O que merece ficar permanente, no fim das contas, não é a versão: é o vocabulário de erros. Um `422` com motivo estruturado que significa "recusa de domínio" precisa significar isso para sempre, porque foi escrito na cabeça de quem consome. Adicionar um código novo é seguro; ressignificar um antigo é a única verdadeira quebra. É o capítulo 4 ao contrário: a convenção que sobreviveu ao motivo vira obrigação, e a API é o lugar onde essa obrigação fica explícita para todo mundo que não vai ler este livro.

O contrato de erro é o contrato

Toda API tem duas superfícies: a de sucesso, que todo mundo desenha, e a de erro, que quase todo mundo improvisa. A segunda é a que define a confiança, porque no sucesso todos os clientes fazem a mesma coisa; no erro, cada um inventa, retry, alerta, silêncio, ticket. O capítulo 12 deu as três categorias, e a tradução para a fronteira é direta: **erro meu** (bug, estado inválido interno) não merece ser vocabulário público, merece alerta, stack, e um `5xx` genérico que diz "nós sabemos, estamos em cima"; **falha do mundo** (o provedor de câmbio que não respondeu) merece distinção de transitória, o cliente precisa saber se retry ajuda ou agrava; **recusa de domínio** (saldo insuficiente, contas iguais) merece `4xx` com motivo nomeado, porque é decisão que o chamador pode tomar com informação. Os números de status são vocabulário padrão da web, não os inventei aqui, mas a disciplina de *mapear o sealed-Result do domínio nesse vocabulário sem perder a distinção* é sua, e é o `aoContrato` acima fazendo o trabalho pesado: o contrato de resposta é a projeção pública do contrato interno, e um mapeamento que colapsa os dois (exceção de domínio virando `500`, timeout virando "erro genérico") não é API: é ruído estruturado.

O teste da qualidade de um contrato de erro é a pergunta que o consumidor faz às duas da manhã: *"o que eu faço agora?"*, e a resposta certa precisa estar na resposta, não na documentação. Um `422` com `{"motivo": "saldo_insuficiente", "pedido": 999.0, "disponivel": 60.0}` responde; um `400 Bad Request` pergunta de volta.

Quem decide o quê

A pergunta de responsabilidade que uma API responde sem dizer é a mais antiga deste livro reencarnada: o que é decisão de contrato e o que é decisão de implementação? A API decide *o que promete*: os estados observáveis, o vocabulário de erro, a garantia de idempotência, o que significa "aceito". A implementação decide *como cumpre*: quantos serviços, qual banco, se a fila é Kafka ou outbox, se o cálculo é de Ana ou do cache de Ana. A fronteira saudável é onde essas duas listas não se tocam, e o sinal de que estão se tocando tem formas reconhecíveis: o consumidor que precisa saber que "essa API é lenta porque o cache do parceiro expira a cada 40 segundos" para usá-la direito é um contrato vazado; o campo `status` que vale "processando" hoje e "liquidando" amanhã porque o time do meio trocou de nome interno é detalhe virado promessa; e o `200` que às vezes vem com corpo e às vezes sem, "depende de onde caiu", é a ausência de contrato declarada em produção.

A regra prática com que fecho a seção, e que vale para API, módulo e até para a conversa com o time de infraestrutura, é o espelho do capítulo 11: **escreva o contrato na linguagem de quem não pode ler a implementação.** Se o consumidor precisa do código-fonte para saber o que a resposta significa, a fronteira foi desenhada no lugar errado. E se você precisa do código do consumidor para saber o que a sua resposta *causa*, você esqueceu que ele existe, o que é, exatamente, a diferença entre um módulo e uma plateia.

O que o material não entrega, e o capítulo não finge

Registro devido, antes que alguém procure: as bibliotecas de HTTP e web do Kof: `kof.http`, `kof.web`, o servidor embutido do `kof serve`, constam da documentação da linguagem como **experimentais**, e nenhuma delas foi exercitada por esta produção; o dossiê de fontes as lista na seção de não-validados, e este capítulo não desceu ao nível de rota e verb porque não tem medição que o autorize. O que foi medido, as duas hierarquias, a conversão, o `VerifyError`, o workaround, é o suficiente para a tese: API é fronteira de contrato, e o contrato se discute no nível onde os nomes significam coisas, não no nível onde `GET` encontra `200`. Quando o material de web do Kof amadurecer o bastante para ser medido com honestidade, o lugar de falar dele é o apêndice de exemplos validados, com data e versão. Até lá, o silêncio informado é a posição.

O capítulo fecha com a plateia e o bug na mesma sala, e a síntese é a do livro inteiro vista do lado de fora: uma API é intenção em código tornada pública, tudo o que ela não expressa, o consumidor deduz; tudo o que ela expressa e a máquina não vigia, alguém vai violar com boa-fé; e a fronteira entre domínios é o lugar onde a disciplina de medir vale mais que o prêmio de design. O próximo capítulo leva a conversa para o lugar onde nenhuma das garantias até aqui sobrevive sozinha: o tempo dos outros, concorrência, efeitos e a ilusão do instante atômico.

Capítulo 15 — Concorrência, tempo e efeitos

O `transferir` do capítulo 13 é uma unidade de integridade perfeita: duas mutações, um fato, nenhum caminho no código que execute metade. Ele passa em todos os testes. Ele honra todas as invariantes. E ele está errado da mesma forma que um relógio parado está certo duas vezes por dia, porque o modelo respondeu à pergunta "o que acontece quando *alguém* transfere?" e o mundo estava fazendo outra: "o que acontece quando *dois alguéns* transferem ao mesmo tempo?".

Duas threads chamam `transferir(a, b, 40.0)` com um intervalo de nanossegundos entre as leituras de saldo. As duas veem `a.saldo = 100.0`. As duas debitam, as duas creditam, o saldo final de `a` é `20.0` quando a aritmética do domínio exigia `-20.0` ou uma recusa. Nenhum dos dois trechos de código violou uma linha do modelo; a violação é de outra ordem, os dois executaram, cada um a sua vez, a lógica correta, em uma ordem que o modelo não previu porque o modelo não tinha a palavra *vez* nele. A corrida não é um erro de lógica. É a ausência de um acordo sobre o tempo.

Este capítulo fecha a Parte III no lugar mais desconfortável da engenharia: o ponto onde todas as garantias construídas até aqui, tipos, contratos, fronteiras, persistência, precisam de uma premissa extra que nenhuma delas fornece sozinha, porque não é uma propriedade do código: é uma decisão sobre quem pode mudar o quê, quando, e sob a vigilância de quem.

A pergunta arquitetural, não a de biblioteca

A conversa sobre concorrência quase sempre começa no lugar errado: `Thread` vs. `async/await` vs. `goroutine`, o repertório de primitivas. A pergunta que decide o sistema acontece antes de qualquer primitiva, e é a mesma pergunta do capítulo 11 sobre comportamento, agora com o tempo no lugar do espaço: **quem é o dono do estado mutável, e qual é a unidade de serialização?** Todo sistema concorrente honesto responde as duas. As famílias de respostas são poucas:

Estado imutável, a resposta mais barata e mais subestimada. O que não muda não disputa: duas threads lendo o mesmo `record` não têm o que estragar, e o capítulo 5 mediu o preço da garantia (SEM038 recusando escrita em componente). A imutabilidade empurra a pergunta para trás, alguém vai mudar *alguma coisa*, em algum lugar, mas encolhe o problema a cada grau de imutabilidade, e o capítulo 10 já mostrou o caminho: estados como valores, transições como funções puras que devolvem o estado novo. O `transferir` do capítulo 13, reescrito sobre contas imutáveis, devolveria o par de contas novo, a corrida entre duas chamadas então não corrompe nada; a *decisão* de qual resultado vence é empurrada, explícita e nomeada, para um ponto de serialização único. E é aqui que a emenda #818 cobra seu preço editorial: no Kof de hoje, estados imutáveis *dentro de hierarquia selada* perdem igualdade por conteúdo, então o código que compara transições precisa comparar campos explicitamente, o modelo sobrevive, com a ressalva medida no lugar.

Um escritor, muitos leitores, o estado mutável vive atrás de um porteiro: um único ator, uma única thread, uma fila de pedidos. A unidade de serialização é a fila; o dono do estado é o porteiro; todos os outros

conversam com ele por mensagens, nunca por referência. É a casa 3 do capítulo 11 aplicada ao tempo: a capacidade nomeada com contrato verificável, só que o contrato agora inclui *ordem*.

Compartilhamento com sincronização, a resposta clássica e a que este livro observa de fora: locks, mutexes, seções críticas, o preço pago em deadlock e em falsos sentimentos de segurança. Não é proibida; é cara e exige uma disciplina que quase ninguém sustenta, a invariante protegida pelo lock precisa ser *nomeada* junto com o lock, ou o lock protege sintaxe em vez de semântica.

Kof fez uma escolha de projeto reconhecível nesta taxonomia, e o livro a registra como decisão, não como destino: **não expõe `Thread`, `Runnable`, nem primitivas de lock**: a documentação de concorrência o diz e o material medido confirma. O idioma disponível é *spawn* (tarefa com resultado) e *channel* (fronteira por onde o valor viaja em vez de ser compartilhado). É a aposta no segundo e terceiro modelos com o primeiro bloqueado por construção, uma decisão de linguagem que empurra todo código Kof para a arquitetura de message-passing. Se a aposta é certa? O capítulo 23 volta nela com a distância necessária; aqui vale o inventário do que a aposta entrega, medido.

O material medido: tarefas e fronteiras

As duas primitivas centrais foram verificadas no dossiê e ampliadas hoje, nesta máquina. A primeira, a tarefa com dono:

```
Int calcular(Int x) { return x * x }
main() {
    val h1 = spawn calcular(7)
    val h2 = spawn calcular(9)
    println(await h1 + await h2)   // 130
}
```

Medido: `130`. O `spawn` como expressão devolve `Handle<Int>` tipado, a tarefa tem tipo, o tipo tem um dono, e o dono decide quando cobrar: o `await` é o ponto de sincronização *nomeado no código*, não um detalhe de encerramento de processo. Compare com o `spawn` statement, fire-and-forget: `spawn baixar(url)` e segue o fluxo. O capítulo 8 mediu o que acontece com uma tarefa esquecida que lança: o `kof test` do projeto-mãe reporta o stack trace no caminho, em produção, um throw dentro de um fire-and-forget sem dono é a falha invisível, o alerta que ninguém recebe porque a tarefa não tem a quem responder. Fire-and-forget é uma decisão sobre responsabilidade, não uma economia de duas linhas: ou o resultado não importa *para ninguém*, ou alguém precisa ser o dono de descobrir que ele importava.

A segunda, a fronteira que transporta estado em vez de dividi-lo, o bloco exato medido hoje:

```
main() {
    var c = channel<Int>()
    val produtor = spawn enviar(c)
    var total = 0
    total = total + c.receive()
    total = total + c.receive()
    println(total)
    await produtor
}
enviar(Channel<Int> ch) {
    ch.send(10)
    ch.send(32)
}
```

Medido na forma completa (`spawn enviar(c)` na chamada, dois `receive()` no consumidor, `await produtor` no fim): `42`. O detalhe fino, descoberto medindo: o tipo do canal como parâmetro é `Channel<Int>`, a documentação de aprendizado nunca mostra um canal atravessando a fronteira de uma função, e o nome do tipo não vem em lugar nenhum; a sonda confirmou que o compilador o aceita e o JVM o implementa sobre fila bloqueante. Registrado no dossiê como capacidade medida e não documentada, o avesso da #819: aqui o material entrega além do que a doc descreve, o que também é um gap, só que do lado da honestidade do manual.

O padrão produtor-consumidor medido é a unidade de serialização em forma de código: o estado (os inteiros) não é compartilhado entre as duas pontas, é *enviado*, e quem recebe é o dono exclusivo do que recebeu. O `receive()` bloqueante no JVM faz do consumidor um escritor único, e a fila do canal é a ordem que o modelo prometeu. É o capítulo 13 de novo: a unidade de integridade existe no domínio, e o canal é o lugar físico onde ela mora.

O contrato que muda por target, e a tabela que o confessa

Aqui a Parte II cobra a dívida que o capítulo 9 abriu, e o capítulo não pode fingir que não a vê: os três alvos de Kof não são três máquinas iguais com três compiladores, são três *modelos de tempo* diferentes. O material medido é JVM: virtual threads, fila bloqueante de verdade, paralelismo preemptivo. O que a documentação registra para os outros dois, com gaps nomeados e datados: no JavaScript, o runtime é o event loop, single-threaded, cooperativo; `spawn`/`await` deferem de verdade via microtask, mas `cancelled()` retorna sempre `0` e a restrição CONC003-JS-01 existe; concorrência ali é agendamento, não paralelismo, duas tarefas nunca se *interrompem*, só se revezam. No nativo, pthreads com futex (CONC001 fechado), polling de 1ms nos canais, bloqueio real, com granularidade própria. E o inventário de não-medidos por esta produção é longo e declarado: `poll`, `done`, `cancel`, `selectAny`, `awaitTimeout` constam da tabela de gaps de `learn/18` com ressalvas por target; `kof.supervisor` é experimental. Nada disso foi re-rodido aqui, e o capítulo não promete o que não mediu.

A consequência arquitetural é a frase que vale colar na parede de quem escreve multi-target: **o código que assume preempção no navegador é um código com um relógio emprestado que ele não pode ver.** Um

laço de retry que bloqueia `await` numa thread dedicada é saudável no JVM e congela a UI no JS, não porque o Kof errou, mas porque "concorrência" significa coisas diferentes nos dois mundos, e a sintaxe única da linguagem uniformiza a *pergunta* sem uniformizar a *resposta*. A tabela de paridade existe para isso; lê-la é parte do trabalho de quem escolhe o alvo.

O tempo do mundo e o relógio do processo

A concorrência é o caso local de um problema maior, que este livro persegue desde o capítulo 2: o modelo precisa dizer o que é *verdade* e o mundo só entrega *sequências*. O retry do capítulo 13 é a resposta a isso: a rede não tem commit, o provedor de câmbio pode responder duas vezes, e a única defesa honesta é o domínio dar identidade ao fato, a chave de idempotência é o nome com que o domínio diz "isto que aconteceu é *este* evento, não um evento". E identidade de evento exige relógio: não o relógio do processo (monotônico, bom para ordenar o que *eu* fiz), mas o relógio do mundo (calendário, contratos, "liquidado até T+1"), que é o relógio em que a invariante do domínio é escrita. Os dois relógios não se alinham, o processo pode hibernar, o mundo pode atrasar, e o `Long data` do exemplo do capítulo 13 é uma promessa sobre o segundo feita com o primeiro.

A disciplina que fecha a seção é a mesma matriz do capítulo 12 com uma linha nova. Para cada estado mutável do sistema: quem é o dono (uma tarefa, um serviço, ninguém)? Qual a unidade de serialização (uma fila, um canal, um commit, uma esperança)? Que relógio define sua validade (o processo, o banco, o calendário)? E o que acontece quando o dono morre, o `spawn` órfão, o consumidor que não volta, a transação de dois minutos que o mundo já considera perdida? A maioria das arquiteturas concorrentes que falham em produção não falhou por falta de primitiva; falhou porque ninguém respondeu a essas quatro perguntas antes de escolher a primitiva.

Fim da Parte III

Os seis capítulos desta parte construíram o software do título: modelar o domínio antes das classes (10) para saber o que as estruturas prometem; escolher a casa do comportamento (11) pela economia da mudança, não pela tribo; dar às três categorias de problema os três canais (12); tratar a persistência como contrato no tempo (13) e a API como contrato com estranhos (14); e fechar reconhecendo que todo contrato vive sob um relógio que ele não controla (15). A tese da Parte III cabe na pergunta que ela ensinou a fazer antes de qualquer código: *quem decide o quê, quem vigia, e quem espera por quem?*

A Parte IV vira a lente: até aqui, a qualidade olhando para dentro do sistema. Os próximos capítulos olham o sistema de fora, como prová-lo sem provar o formato (16), como medi-lo sem mitologia (17), como distribuir complexidade em vez de escondê-la (18), como envelhecer sem apodrecer (19), e como a segurança pertence ao projeto inteiro, não ao componente mais nervoso (20). O primeiro deles começa pelo vício mais comum da profissão: testar que o código se parece com o que o programador escreveu, em vez de testar que o sistema faz o que o domínio exige.

Parte IV — A qualidade além do arquivo

Capítulo 16 — Testar o comportamento, não a aparência

A refatoração era impecável. A função de trezentas linhas virou quatro funções coesas, cada nome do domínio, o teste de desempenho idêntico, o comportamento preservado linha a linha, e quarenta testes vermelhos no CI. O time passou a sexta-feira à noite "consertando os testes": apontando mocks para os novos nomes, ajustando as contagens de chamadas, espelhando nos stubs a nova estrutura que o código tinha acabado de melhorar. No domingo, verde. Ninguém perguntou a pergunta que arde: **se o comportamento não mudou e os testes mudaram, o que exatamente aqueles quarenta testes estavam testando?**

A resposta, nua: a forma. Cada um deles sabia que `processar` chamava `validar`, que `validar` devolvia tal estrutura intermediária, que o fluxo passava por tal objeto, e nada disso é comportamento do sistema; é a fotografia de como o sistema estava organizado na semana passada. O teste que quebra quando o código melhora não é um teste frágil por acidente: é um teste que nunca foi sobre o sistema. É o capítulo 1 aplicado ao próprio teste, a pergunta "o que esta estrutura diz?" vale para o código de produção e para o código de teste, e a resposta dos quarenta vermelhos foi: *eles diziam como o código era, não o que o sistema faz*.

A pergunta que cada teste responde

Todo teste é uma afirmação sobre alguma coisa; a classificação que importa é sobre *qual* coisa. "Dado este estado do domínio, esta operação produz este resultado" é uma afirmação sobre o sistema, ela sobrevive a qualquer refatoração que preserve o comportamento, e *deve* quebrar quando o comportamento muda, porque é isso que um alarme faz. "Este método chama aquele collaborator com estes argumentos" é uma afirmação sobre o código, ela quebra quando o código muda *para melhor*, e passa quando o código muda *para pior*, desde que a estrutura chamada se mantenha. O segundo tipo não é teste de unidade; é espelho com `assert`.

O caso extremo do espelho é o mock onipresente. O mock existe para isolar: testar a minha lógica quando a sua ainda não existe, quando ela é lenta, quando ela falha. Mas o teste que mocka tudo exceto a linha que chama o mock prova que o mock faz o que o mock disse, um teorema sobre a minha imaginação, verificado por mim mesmo. O dia em que o collaborator real discorda do stub, o verde do CI é uma mentira com boa cobertura. A regra que uso, sem transformá-la em mandamento: **mock é para a fronteira, não para dentro da casa**: o que atravessa a linha do capítulo 14 (rede, banco, relógio do mundo, o provedor de câmbio) pode e deve ser stubado; o que é o meu domínio falando comigo mesmo não deveria precisar de dublê, porque o custo de manter o dublê igual ao original é o custo de manter dois sistemas.

E há o teste que nem espelho é: o teste que existe porque o framework pediu um arquivo por classe. O capítulo 4 já descreveu a anatomia da burocracia, regra sem problema nomeado, e ela se aplica com juros ao código de teste: um teste cuja resposta à pergunta "o que isto protege?" for "cobertura" é ruído executável. Ele paga aluguel (manutenção a cada mudança de forma) sem pagar juro nenhum (nenhuma confiança a mais ao mudar), e pior: infla a métrica que alimenta a si mesmo.

O nome do teste é a especificação

O material medido desta rodada ajuda a fechar uma lacuna do próprio livro. O capítulo 8 apresentou o `kof test` pelo que media: o programa é o teste, PASS/FAIL por código de saída. O dossiê listava o harness estruturado: `test "nome" { }` com `assert(cond, "msg")`, como documentado e não exercitado. Exercitado hoje, nesta máquina, com as duas faces:

```
Int somar(Int a, Int b) { return a + b }

test "soma" {
    assert(somar(2, 3) == 5)
}

test "zero" {
    assert(somar(-1, 1) == 0, "inverso aditivo")
}
```

Saída medida, suíte verde: `PASS soma`, `PASS zero`, `0 failed of 2 tests`, exit code `0` (medido com `PIPESTATUS`, a lição do capítulo 8 sobre pipes valendo). E a suíte com um caso desenhado para falhar reporta por nome e com a mensagem do assert: `FAIL falha deliberada: 2+2 deveria dar 5 segundo este teste ruim`, exit code 1. O harness funciona como a documentação descreve, registro positivo que o método exige: não é só bug que se encontra medindo; é também a confirmação de que o material entrega a promessa, o que vale a um projeto de linguagem exatamente o que um teste verde vale a um time: evidência, não opinião.

Repare, porém, no nome dos meus testes: `soma`, `zero`. São nomes de *função*, não de *comportamento*. Nenhum deles responde "o que o sistema garante?", respondem "existe um teste para cada método", a doença deste capítulo em miniatura, cometida por quem a diagnostica. Um nome de teste honesto é uma afirmação do domínio: `transferencia recusada quando saldo insuficiente`, `estornada nao liquida de novo`, `mesmoDia vale true para liquidacoes de mesmo valor e data`, este último, extraído diretamente do exemplo medido do capítulo 10, onde a igualdade por conteúdo é justamente o comportamento que a #818 ameaça: o teste com esse nome não descreve o `mesmoDia`; ele *registra a promessa do domínio* e congela a emenda do material como alarme, não como surpresa. Quando o `FAIL falha deliberada` imprime o nome na tela, o nome é a interface do harness, e interface de teste é especificação executável. Se o nome não sobreviver ao teste "e se o comportamento mudar?", ele descreve forma.

A economia: juro, aluguel e a ordem vermelha

Trate cada teste como um ativo financeiro e a conversa muda de figura. O **juro** é a confiança para mudar: quanto mais verde confiável atrás de uma fronteira de comportamento, mais refatoração o time faz, mais rápido o código envelhece bem (capítulo 19 cobra essa conta). O **aluguel** é o custo de mantê-lo vivo: todo

teste acopla quem o escreve a uma forma de o código existir, acopla à assinatura do construtor, ao nome do método, à estrutura do mock, e cobra a cada mudança de forma, mesmo as boas. O teste de comportamento tem juro alto e aluguel baixo: a forma muda embaixo dele e ele não percebe, porque só conhece a fronteira. O teste de forma tem aluguel alto e juro que vence cedo demais: ele garante que o código de hoje chama o código de hoje.

A métrica que quase ninguém calcula é a razão entre os dois, e ela explica por que times com 90% de cobertura refatoram com medo: cobertura mede linhas *tocadas*, não comportamento *protegido*. Um arquivo pode estar coberto por quarenta espelhos e protegido por zero afirmações.

Daí a defesa que faço do test-first, despidos de liturgia: escrever o teste antes não é disciplina moral, é o único procedimento simples que **garante que o teste pode falhar**. O teste escrito depois do código verde passa de primeira, e um teste que nunca falhou não é evidência de nada, pode estar provando o comportamento, pode estar provando a minha convicção, e você não tem como distinguir sem quebrar algo de propósito. O vermelho antes do verde é o teste do teste. (Isso vale para o `falha deliberada` que escrevi hoje antes de precisar dele: o caso que *deveria* falhar me disse, em segundos, que o harness reporta por nome e devolve exit code 1, sem ele, o verde da suíte não significaria nada.)

A escada de evidência e a falácia do degrau errado

Nenhuma camada substitui a outra, e a confusão mais cara em testes é usar o degrau errado para a pergunta errada. A escada, do barato ao caro:

O **compilador** é o primeiro verificador e o mais barato de todos, o capítulo inteiro 8 existe para isso, e os SEM deste livro (SEM043 cobrando implementação, SEM081 cobrando exaustão, SEM048 recusando ausência sem procedência) são testes que rodam de graça em cada build, para sempre. Tudo que você consegue empurrar para baixo, para o tipo, economiza um degrau inteiro da escada.

O **teste unitário** pega o que o tipo não expressa: a regra de cálculo, a transição proibida, o arredondamento. Ele prova o comportamento *das partes*, em isolamento, e o "em isolamento" é a promessa e o limite.

O **teste de contrato e de integração** pergunta o que nenhuma unidade sabe: as juntas funcionam? O `422` do capítulo 14 corresponde ao `SaldoInsuficiente` do domínio, depois de atravessar o mapeamento, a serialização e a transportadora? É aqui que mora a traição silenciosa, o bug #819 que este livro encontrou *justamente* na conversão entre duas hierarquias que individualmente passavam em tudo. Duas peças verificadas, uma junta não.

A **produção** é o último degrau e não tem substituto: escala, concorrência real, relógio do mundo, o cliente que chama duas vezes porque o retry dele não é idempotente. O que a produção ensina, os degraus anteriores deviam ter pego; o que os degraus anteriores pegam, a produção não precisa descobrir. A pergunta honesta depois de cada incidente não é "por que o teste não pegou?", é **"em que degrau esta falha pertencia?"**.

Timeout que derruba a UI: integração com o mundo real, nenhum unit pega. Estado inválido que migração criou: nem isso, é o degrau da produção, e a resposta certa é observabilidade e rollback, não "mais testes".

A falácia simétrica é esperar do degrau de baixo a garantia do de cima: time que celebra cobertura como prova de ausência de bugs comete o erro lógico com nome bonito, teste demonstra presença de comportamento, nunca ausência de falha; e time que exige teste de integração para cada função queima o aluguel de uma escada que o compilador subiria de graça. O capítulo 2 inteiro cabe aqui: cada verificador tem seu atacante, e a defesa em profundidade do capítulo 13 vale para a evidência como vale para a invariante.

O teste como intenção em código

Fecho onde o livro sempre fecha. Um teste é código, e código deste livro é representação: a pergunta "o que esta estrutura diz sobre o sistema?" vale para a suíte inteira. A suíte que nomeia comportamentos do domínio: `estornada nao liquida de novo`, é documentação executável: ela diz a quem chega o que o sistema *é* e o que ele *recusa ser*, e sobrevive às reescritas da forma. A suíte que nomeia métodos: `testaSoma`, `testaProcessarValidaRetorna`, é o espelho da semana passada, e o dia em que ela quebra sem o comportamento mudar, ela te contou a verdade sobre si mesma: ela nunca foi sobre o sistema.

A Parte III perguntava quem decide o quê, quem vigia e quem espera por quem. A Parte IV pergunta como o sistema se comporta quando medido de fora, e o primeiro degrau dessa medição é a evidência que você produz antes do primeiro clique em deploy. O próximo capítulo sobe para o degrau que todo mundo acha que entende e quase ninguém mede direito: a performance, entre a superstição do palpite e a mitologia do benchmark.

Capítulo 17 — Performance sem superstição

A reunião tinha três opiniões sobre o `recalcularPosicoes` e zero medições. Um dizia que era o laço interno, "sempre é o laço interno"; outra jurava que era a alocação do vetor temporário, "vi um artigo"; o terceiro apostava que o problema real era o garbage collector, "JVM é tudo igual". Cada um apresentou sua tese com a mesma confiança e nenhum apresentou um número. A discussão durou quarenta minutos e terminou com a decisão clássica dos times que não medem: otimizar o laço interno, porque era o que dava menos trabalho debater.

Duas semanas depois, um perfil de produção, feito por quem foi obrigado a medir porque o SLA venceu, mostrou o que os três erraram juntos: o laço e a alocação respondiam por menos de um quinto do tempo; o resto era uma chamada HTTP síncrona dentro do caminho de cálculo, feita *por posição*, que ninguém tinha considerado porque "rede não conta como CPU". A otimização do laço interno tinha melhorado o pior terço do problema em mais ou menos nada.

Este capítulo existe para tornar essa reunião impossível. E abre com uma confissão metodológica rara, que a produção deste livro produziu sem procurar: **a ferramenta de medição da própria linguagem mentiu neste ciclo de escrita.** O `kof profile`, exercitado pela primeira vez para este capítulo, com um laço puro de duzentos milhões de iterações, reportou `wall: 229 ms` e `cpu (user/sys): 0,24 / 0,02 s`, e ao lado do tempo de CPU, teimosamente, `(0% of wall)`. Zero vírgula nada: duzentos e sessenta milissegundos de CPU medidos contra duzentos e vinte e nove de parede são mais de cem por cento, não zero. O número derivado estava errado em duas execuções de duas cargas diferentes, o padrão denuncia o parsing (os segundos impressos com vírgula decimal, "0,24", provavelmente lidos como zero pelo cálculo da porcentagem). A issue #820 está aberta no repositório da linguagem, e o episódio entrou neste capítulo porque é a sua tese em forma de anedota: **medir é um ofício com ferramentas, e as ferramentas também erram, inclusive, e principalmente, as que você usa para acusar as outras.** Quem aprende a desconfiar do código sem aprender a desconfiar do medidor troca uma superstição por outra.

A pergunta antes do número

"Está lento?" não é uma pergunta respondível, e a reunião dos três palpites morreu aí. Lento tem três variáveis livres que precisam de valor antes de qualquer medição fazer sentido: *para quem* (o usuário que espera, a máquina que cobra, o time que mantém, a economia da seção cinco), *fazendo o quê* (o caso de uso fim a fim, não a função isolada que o artigo otimizou), e *quanto* (a diferença entre "perceptível" e "aceitável" é um orçamento, não uma emoção). "O recálculo de 500 posições em 4 segundos é inaceitável porque o editor trava acima de 100 milissegundos" é uma pergunta de engenharia; "o sistema está lento" é um humor.

A hierarquia do custo é o segundo filtro, e ela é velha e ignorada na mesma proporção: **algoritmo > estrutura de dados > I/O e fronteiras > micro-otimização.** Um O(n²) que vira O(n log n) ganha de qualquer ajuste fino com a margem de quem troca de escala; a escolha da estrutura que elimina a busca

interna ganha da troca de linguagem; e o que quase sempre domina o tempo real de um sistema de negócio está do outro lado da fronteira do capítulo 14, a chamada HTTP, o disco, o lock do banco. O laço de duzentos milhões de iterações que rodei para este capítulo: 229 ms de parede no JVM, o que dá um nanossegundo por iteração, um desempenho honesto de loop, é justamente a exceção que ensina a regra: quando o trabalho é *puro CPU em loop fechado*, a micro-arquitetura importa; quando o trabalho é o que sistemas reais fazem, o gargalo mora fora do código que a reunião estava olhando.

E aqui o material medido cobra pedágio de honestidade, duas vezes. A primeira: o próprio laço que produziu os números é um exemplo didático de custo de algoritmo: `somarAte(n)` iterativo é O(n) onde a forma fechada `n * (n + 1) / 2` é O(1); a 200 milhões de iterações, a diferença não é "um pouco mais rápido", é *outra categoria de programa*. A segunda é mais incômoda e maior: **o laço devolveu uma resposta errada, e devolveu rápido.** `somarAte(100000)` imprimiu `705082704`; a soma verdadeira é `5000050000`; a diferença é exatamente 2^{32}, o `Int` de Kof é de 32 bits e o overflow wrap-around é silencioso, comportamento *documentado* na referência (o dossiê §5 o lista entre os limites da linguagem) e demonstrado nesta máquina ao medir um exemplo de um capítulo sobre desempenho. A forma fechada, escrita em `Int`, estouraria igual. A lição atravessa o capítulo inteiro e vale mais que qualquer dica de cache: **a primeira pergunta do desempenho nunca é "quão mais rápido?", é "a que custo continua sendo certo?".** Resposta certa e rápida é o único resultado que existe; resposta rápida e errada não é otimização, é incidente com métrica bonita.

O que este livro pode afirmar com número na mão, e o que não pode

A regra de evidência do prompt de produção é explícita: números quantificados exigem medição com metodologia declarada. Então, com as mãos tremendo de honestidade, o que *posso* dizer:

As latências de ciclo deste projeto, compilar e rodar um exemplo pequeno: da ordem de **meio a dois segundos no alvo JVM, um segundo e meio no nativo, seis no JavaScript**: são medidas do dossiê §2, N=1 por alvo, uma máquina, um dia, sem warmup nem repetições. O que elas sustentam é a afirmação qualitativa do capítulo 8: *a ordem de grandeza do ciclo de iteração muda o comportamento do programador*: dois segundos permitem tentar, seis segundos convidam a prever, e nada além disso. O que elas não sustentam, sob nenhuma hipótese: qualquer comparação de velocidade entre os alvos ("JS é 3× mais lento que JVM", com N=1, isso é folclore com decimais), ou qualquer afirmação sobre throughput, escalabilidade, ou o desempenho do Kof em produção. O `wall: 229 ms` de hoje é do mesmo naipe: uma execução, carga sintética, e o percentual derivado da ferramenta era, como sabemos agora, inventado.

O que eu *não* posso dizer, e o livro não dirá em lugar nenhum sem construir a bancada primeiro: nenhum número de throughput, nenhuma comparação com linguagem X, nenhuma curva de escala. O repositório Kof4j mantém uma pasta `benchmarks/` com metodologia própria, que esta produção não executou nem auditou, e números de terceiros sem metodologia lida são rumores bem-vestidos. O `kof bench` existe no CLI com a promessa documentada de "compile, run, validate output, collect metrics, compare baseline", infra de metodologia séria, visivelmente pensada para warmup, iterações e limiar de regressão, mas a minha

tentativa de exercitá-lo devolveu `no benchmarks found` e o formato de descoberta de benchmarks não está em nenhum documento que o dossiê verificou. Fica registrado como ferramenta documentada e não-exercitada, e o episódio merece a linha que ele prova: **uma ferramenta cujo contrato de uso não está documentado é uma ferramenta que não existe para o time**: a infra de medição que ninguém consegue invocar é exatamente tão inútil quanto a medição que ninguém fez.

O método, na ordem

Com as perguntas certas e a humildade numérica no lugar, o método de desempenho é curto e chato, o que não o torna menos violado:

Primeiro, o caso de uso fim a fim. A métrica que o usuário sente, medida do lado de fora, antes de qualquer otimização: "o recálculo de 500 posições leva 4 segundos". Essa linha é o orçamento; todo o resto é despesa. Otimizar sem ela é a reunião dos três palpites com um profiler aberto.

Segundo, o perfil antes do palpite. Onde o tempo *realmente* mora, e o perfil de produção do exemplo inicial fez em uma tarde o que quarenta minutos de opinião não fizeram. Com a ressalva da nossa própria anedota de abertura: o perfil é uma ferramenta, ferramentas têm bugs (#820), e números que contradizem a aritmética elementar (113% de CPU como 0%) devem acionar desconfiança na ferramenta, não obediência.

Terceiro, a hierarquia. Algoritmo, estrutura, I/O, fronteiras, desce-se a escada nessa ordem porque o topo paga mais. O `inspect` do capítulo 8 mostrou o caminho de baixo: a abstração de biblioteca tem corpo mensurável: 37 operações de `Pair` para um `reduce` de três elementos. Em caminho frio, irrelevante, e a abstração fica; em caminho quente chamado por quadro, é o custo que o perfil nomeou, e a troca por um laço explícito é decisão com evidência dos dois lados. É o capítulo 2 aplicado ao runtime: a complexidade acidental da conveniência cobrada no lugar onde ela dói deixa de ser filosofia e vira linha de orçamento.

Quarto, uma mudança por vez, medida contra o orçamento. A otimização que não melhora o número do caso de uso não é otimização, é refatoração com custo de legibilidade, e o capítulo 3 já precificou essa conta: código mais rápido e menos legível é código que alguém paga duas vezes, a segunda em forma de bug na manutenção.

A economia de quem paga

Toda decisão de desempenho transfere custo entre três pagantes, e nomeá-los destrava metade das discussões. O **usuário** paga latência, e latência tem um degrau perceptível (o olho acompanha até ~100 ms sem quebrar a sensação de imediato; além disso, o editor trava, a página "morre") que explica por que o SLA do caso de uso do início era 100 ms e não "o mais rápido possível". A **máquina** paga em dinheiro direto: RSS de 48 MB por processo, o número medido pelo profile de hoje, plausível e não-otimizável a zero, vezes mil instâncias é fatura; times que tratam custo de infra como problema do financeiro descobrem, tarde, que arquitetura é orçamento. E o **time** paga em complexidade: cada truque de velocidade é uma dívida de

legibilidade com juros compostos, e o capítulo 4 descreveu o destino final de todo código que ninguém entende, intocável, depois morto.

A troca entre os três não tem coeficiente universal, tem contexto, e o contexto nomeável é o que separa as decisões. No caminho de renderização por quadro, o truque de bits que economiza alocação paga o aluguel de legibilidade *porque o orçamento do usuário é o contrato do produto*. No serviço de relatório que roda de madrugada, o mesmo truque é vandalismo: ninguém sente os 40 ms, e quem mantém o código sente os duzentos. "Depende" não é a resposta preguiçosa deste livro, a resposta é *depende de qual pagante tem o orçamento apertado*, e isso é mensurável.

O fantasma da otimização prematura, exorcizado sem a frase

Há uma máxima famosa demais para precisar de fonte, atribuída correntemente a Knuth, cuja formulação exata este livro se recusa a reproduzir sem ter conferido no original [a verificar: artigo de 1974 sobre programação estruturada], e o seu conteúdo sobrevive sem a citação porque é verdadeiro por estrutura, não por autoridade. A versão que circula em corredor, "não otimize antes de precisar", é mais fraca que o original e produz o erro gêmeo: times que *nunca* medem achando que estão sendo disciplinados.

A versão correta não fala de *quando* otimizar, fala de *com o quê* decidir. Otimização prematura não é otimizar cedo demais; é **otimizar sem evidência**: a estrutura de dados escolhida pelo palpite do capítulo sobre o caso de uso que ninguém mediu, a abstração defensiva contra um volume que o produto nunca vai ver, o cache adicionado "por garantia" antes do perfil apontar a leitura repetida. O antídoto não é esperar o incêndio, é o método da seção anterior rodando contínuo: orçamento declarado, perfil no lugar de opinião, hierarquia respeitada. Um sistema projetado com medição desde o dia um não "otimiza cedo nem tarde"; ele escolhe com informação, que é a única alternativa que existe ao lado da superstição. E o oposto do vício não é a virtude da lentidão: há arquiteturas cujo custo de performance *é* a decisão de design, escolher o formato de serialização, o modelo de consistência, o granularity do canal do capítulo 15, e adiar essas escolhas "para medir depois" é a mesma superstição de cabeça para baixo, porque depois é caro demais mudar.

Fechando com a régua e o susto

O capítulo fecha com as duas medições que ele produziu e uma síntese. O laço de 100 mil iterações imprimiu um número errado com a maior naturalidade do mundo, o overflow silencioso de 32 bits, documentado, medido, demonstrado, e o laço de 200 milhões provou que a ferramenta que media o primeiro era cega na porcentagem. Um sistema de produção é feito de loops que ninguém perfilou e números que ninguém perguntou "certo a que custo?"; a disciplina deste capítulo é uma régua com duas marcas: **meça antes de tocar, e desconfie do medidor.** A régua se aplica ao código; o próximo capítulo aplica ao mapa inteiro, arquitetura, que é onde a complexidade mora quando ninguém está olhando, e distribuí-la é o trabalho que nenhuma otimização local faz por você.

Capítulo 18 — Arquitetura como distribuição de complexidade

O diagrama era lindo. Caixas alinhadas, setas sem cruzamento, legenda colorida por camada, e uma frase no rodapé que fechou a reunião: "arquitetura simples e desacoplada". Seis meses depois, adicionar um campo ao pedido de estorno exigia tocar cinco caixas, e ninguém sabia por que cinco, o diagrama mostrava o que cada componente *era* e nunca disse onde a dificuldade *morava*. A simplicidade que a equipe lembrava era a do desenho; a do sistema tinha ido para outro lugar, que é exatamente o que acontece quando se confunde o mapa com o território.

As caixas são o resíduo das decisões; a arquitetura é a decisão. E a decisão, formulada com honestidade, é uma só: **onde a complexidade vai morar quando ninguém estiver olhando.** Este capítulo existe para trocar o vocabulário da discussão, camadas, serviços, barramentos, grades, pelo vocabulário que presta conta do que acontece de verdade: o movimento da dificuldade.

A lei de conservação, em escala de sistema

O capítulo 2 apresentou a contabilidade: complexidade essencial não se destrói, e a acidental se compra e se vende. Como heurística de trabalho, valeu generalizar em uma frase que nenhum experimento contradisse até hoje: **a dificuldade não desaparece; muda de endereço, de dono e de linguagem.** Toda decisão arquitetural, dividir, envolver, camadas, filas, caches, serviços, é uma transferência bancária de complexidade: tira de um lugar, deposita em outro, e o recibo quase nunca é preenchido. O diagrama de caixas é precisamente o documento que registra os depósitos sem registrar os saques: mostra as fronteiras que o sistema *tem* e silencia sobre as perguntas que ele agora *exige*.

Daí os dois nomes que este livro usa para avaliar qualquer desenho, herdadas do orçamento cognitivo do capítulo 3. Um sistema tem **complexidade concentrada** quando a dificuldade do domínio mora em poucos lugares nomeados, quem mexe no cálculo do líquido encontra *um* módulo, e a pergunta "o que isto esconde de quem?" tem resposta curta. E tem **complexidade diluída** quando a mesma dificuldade está espalhada em doze serviços na forma de `if` de exceção, ou morando na memória de três pessoas, ou, a forma mais traiçoeira, escondida *atrás de uma abstração que prometeu escondê-la*: o framework que absorve a complexidade sem que ninguém no time saiba mais explicar o que ele faz quando dá errado. A diferença entre as duas não é o total de dificuldade; é a **enumerabilidade do raio de mudança**. A pergunta diagnóstica que resolve reuniões de arquitetura é esta: *quando o provedor, o preço ou o calendário mudarem, quantos lugares eu toco, e como sei que toquei todos?* Uma arquitetura de distribuição boa torna a resposta verificável, um módulo, uma busca, uma lista do compilador. A ruim torna a resposta "depende de quem lembra".

O sonho dessa enumerabilidade já existe em escala de arquivo, e quem leu a Parte II o reconheceu: `sealed` com switch exaustivo é arquitetura aplicada ao tipo, o SEM081 *enumera os lugares a tocar* quando o mundo ganha um caso novo. A arquitetura de sistemas é o mesmo gesto em escala maior: tornar o blast radius da

mudança listado em algum lugar que não a memória da equipe. Onde nenhum verificador enumera, nem o tipo, nem o teste, nem a CI, ali a complexidade foi diluída, e o custo aparece no dia em que alguém toca um lugar e esquece os outros onze.

O critério dos módulos, sem a citação

A tradição que formalizou o que um módulo *é*, contra o que um arquivo com nome de classe é, associou-se ao nome de David Parnas e aos seus escritos de 1972 sobre critérios de decomposição [a verificar: título e dados exatos da publicação antes de citar nominalmente na revisão final, a bibliografia carrega a pendência]; a ideia sobrevive intacta sem a nota de rodapé, e é ela que importa aqui: **um módulo bom esconde as decisões mais prováveis de mudar e expõe as menos prováveis.** Não "esconde implementação" como mantra vago, esconde *a instabilidade específica* que ele jurou absorver.

Invertida, a regra nomeia o cheiro arquitetural mais caro, porque ele é invisível no diagrama: a camada que **expõe o instável** e **esconde o estável**. O formato de resposta do provedor de câmbio aparecendo nos objetos do domínio é o instável vazando para dentro; a regra de arredondamento do líquido distribuída em doze serviços que "sabem" o cálculo de cor é o estável evaporando para todos os lados. O teste da camada, então, não é "ela desacopla?", é "ela esconde *o quê*, e esse quê muda *mais* do que o que ela mostra?". Camada é aposta temporal: aposto que o que fica atrás dela muda mais rápido que o que fica na frente. Acertada, ela absorve ondas de mudança sem que a frente veja; errada, ela é aluguel sem renda, o depósito do capítulo 7, guardando a chave de si mesmo.

E a honestidade simétrica que este livro deve ao capítulo 4: camada que nunca trocou o que esconde não é fraude. O adaptador que isolou um provedor que nunca mudou ainda entregou as outras duas moedas, isolou *falha* (o timeout morava nele, não no domínio) e tornou o contrato *testável* (a escada do capítulo 16 tem um degrau onde o stub vale, e é exatamente na fronteira que ele esconde). A auditoria que proponho, escrever, para cada caixa do diagrama, a mudança que ela absorve, serve para encontrar a caixa sem resposta, não para demolir as que responderam devagar.

As fronteiras cobram pedágio, e este livro tem recibos

Se arquitetura é distribuição, as fronteiras são os pontos de controle, e é nelas que a dificuldade se acumula por definição: tudo o que cruza paga conversão. O material deste projeto mediu três recibos, e eles merecem reaparecer aqui em fila porque contam a mesma história em três escalas.

O primeiro é o contrato verificado: `implements Shape` sem `area()` não passa do compilador: SEM043, medido no capítulo 7 com a mensagem nomeando classe, método e interface. É a fronteira com segurança de verdade: a promessa da caixa é cobrada na porta, não na reunião. Poucas arquiteturas têm o equivalente ao SEM043 nas suas fronteiras de serviço, a maioria confia no contrato que ninguém executa.

O segundo é o contrato que muda de significado por target: a tabela de paridade do capítulo 9: `kof.ui` que é no-op fora do navegador, `catch` que no nativo captura tudo, `super` com gap nomeado. É a fronteira entre o código e o mundo de execução, e o recibo diz: a decisão de "compilar para três mundos" distribuiu complexidade para *dentro de cada feature*, que agora precisa responder "qual alvo?" antes de "o quê?". Toda arquitetura multi-é-isto paga essa conta; a honestidade está em escrevê-la numa tabela datada em vez de descobri-la num incidente.

O terceiro é o mais instrutivo, porque este livro o produziu: o bug #819 mora **na conversão entre duas hierarquias seladas**, cada uma individualmente verificada e funcional, a junção entre estruturas corretas é que gerava bytecode inválido, silenciosamente aprovado pelo verificador de cada lado. É a parábola perfeita da tese deste capítulo em escala de arquivo: **o risco não mora nas caixas, mora nas setas.** Componentes testados individualmente, composição nunca medida, é o teorema do capítulo 16 na arquitetura: nenhuma camada da escada de evidência substitui o teste da junção, e a maioria dos sistemas em produção é feita de junções que ninguém mediu, ninguém nomeou e ninguém testou de fora a fora.

O debate monolito × serviços, devolvido à pergunta certa

A discussão arquitetural mais barulhenta dos últimos vinte anos é, lida com este vocabulário, uma discussão sobre **duas distribuições**: de falha e de mudança, que os participantes insistem em tratar como uma discussão sobre caixas.

O monolito concentra: uma unidade de build, uma transação com dono único (o commit do capítulo 13 é barato quando há um banco), a serialização do estado resolvida por vizinhança, e paga em escala (um processo, um gargalo, um deploy que carrega tudo) e em atrito organizacional (times diferentes, um código, merge e fila). Os serviços distribuem: falha isolada por processo, deploy pelo ritmo do time, escala por peça, e cobram pedágio em *cada* seta: a rede sem commit do capítulo 13 (exactly-once é mito, idempotência é projeto), o contrato que vira permanente no instante da primeira chamada externa (capítulo 14), e a pergunta que o capítulo 15 ensinou a não pular: **quando dois serviços acham que são donos do mesmo dado, quem é a unidade de serialização?** A resposta "o banco compartilhado" tem um nome na literatura de prática: monolito distribuído, paga-se o imposto da rede *e* divide-se a responsabilidade pela consistência, o pior dos dois mundos com o diagrama dos dois.

Nada disso é argumento por monolito nem por serviços; é argumento pelo **critério**. A escolha se responde por duas perguntas de distribuição: *onde a falha deve parar* (consequência de processo define isolamento de falha, e processo é caro), e *onde a mudança deve fluir* (fronteiras de deploy acompanham fronteiras de time e de ritmo de mudança, o git-log do capítulo 11 em escala arquitetural: o que muda junto, vive junto). Todo o resto, a quantidade de caixas, o vocabulário de moda, é estética com orçamento. Um sistema bem particionado em três processos pode ser mais honesto que um em vinte serviços que compartilham tabela; a diferença não aparece no diagrama de ninguém.

A aposta no tempo, e as costuras

Arquitetura é a decisão arquitetada para durar mais que suas implementações, e é por isso que ela é a mais cara de reverter: código se move dentro da arquitetura; a arquitetura se move junto com a organização, os contratos publicados e os dados persistidos. O capítulo 19 vai cobrar essa conta em detalhes. Aqui fica o corolário prático de tudo acima: **projete a distribuição pelo que você sabe sobre o ritmo de mudança, e deixe costuras onde espera a próxima.** Costura não é "generalidade para todo futuro", é a fronteira nomeada, o adaptador de um caso, o evento no outbox que ainda ninguém consome: o ponto onde a mudança esperada tem endereço, e o resto do sistema não precisa adivinhar. É o meio-termo exato entre a catedral do capítulo 10 e o "a gente refatora quando doer": a catedral antecipa tudo e paga aluguel por nada; o "depois" ingênuo descobre que a junção não testada virou o buraco onde a #819 morava. A costura custa pouco porque é estreita: um nome, uma fronteira, um contrato, e é o único tipo de futuro que a arquitetura consegue de fato comprar.

Fechando o capítulo com a pergunta que ele substitui. A reunião de arquitetura não deveria começar com "quais caixas?" nem terminar com "quais tecnologias?"; deveria caber inteira em quatro perguntas: **onde mora a dificuldade de cada caso de uso? quem pode vê-la, e quem é obrigado a lembrá-la? quando o mundo mudar, quantos lugares toco e como sei que toquei todos? e qual mudança eu espero, para onde estou deixando costura?** O diagrama que responder essas quatro é documentação; o que só responde "o que é cada caixa" é decoração. A Parte IV fecha com o capítulo que observa o desenho escolhido pelo único júri que importa: o tempo, manutenção, compatibilidade, e a evolução de tudo que este livro chamou de aposta.

Capítulo 19 — Manutenção, compatibilidade e evolução

O capítulo 18 terminou com o júri marcado: o tempo. A sentença chega em três formas reconhecíveis, e todo código-fonte vivo as conhece. Volte a um módulo que você mesmo escreveu há dois anos, e uma de três coisas acontece: você o entende como se fosse de outra pessoa, e aí o código sobreviveu bem; você o entende e refatora em uma tarde, e aí ele foi *projetado* para isso; ou você olha, suando, para doze chamadores que dependem de detalhes que você não lembra por que escolheu, e faz a única coisa que a situação permite: nada. O terceiro caso não é falha de quem mantém. É a conta de quem construiu, entregue no vencimento.

A manutenção é o estado natural do software; escrita sem manutenção é o evento raro que os livros celebram. O capítulo 3 precificou a leitura, o orçamento cognitivo de quem entende; este capítulo precifica a mudança, o orçamento de quem *altera*, que é a mesma cabeça com menos tempo e mais medo. E a primeira conclusão é a que separa as três saídas do parágrafo anterior: **um código é mantível na exata medida em que suas decisões são recuperáveis e suas mudanças são baratas**: intenção no texto (capítulo 1), distribuição no desenho (capítulo 18), evidência na escada (capítulo 16). Manutenção não é o que acontece quando a engenharia acaba; é o teste de que ela aconteceu.

Quem paga a mudança, a política antes da técnica

Compatibilidade quase é tratada como problema de release notes: o que entra na lista de "breaking changes". É uma política disfarçada de burocracia, e a pergunta política é: *quando isto muda, quem paga?* Há três pagantes, e um sistema maduro tem uma resposta explícita para cada um.

O **consumidor externo** paga com o dinheiro dele, o código dele. O capítulo 14 estabeleceu por que ele é o mais caro: tudo o que a API expõe virou contrato por observação, e o observador não leu seus comentários. Para ele, compatibilidade é a regra default e quebra é decisão com custo justificado, não acidente de sprint.

O **time interno** paga com o tempo dele, e o preço cresce com a diluição (capítulo 18): uma regra que doze serviços duplicaram não muda mais, não porque alguém proibiu, mas porque o custo da mudança é uma força-tarefa. A decisão de há dois anos: "cada um copia o arredondamento, é mais rápido do que criar o módulo", virou a corrente que agora segura o produto. É o aluguel do capítulo 16 vencendo em escala de arquitetura: cada cópia era um espelho, e espelhos não se refatoram, se substituem doze vezes.

O **dado persistido** paga com a verdade dele, e é o pagante que quase ninguém orça. O capítulo 13 nomeou o mecanismo: a regra mudou debaixo do registro. Migrar dados é reescrever o passado sob as regras do presente, e o presente erra tanto quanto o passado: a migração que "conserta" os saldos negativos de 2019 pode estar destruindo o único registro de que a política de crédito de 2019 era outra. A decisão honesta não é "nunca migrar", é migrar com o mesmo respeito com que se trata contrato publicado: regra nova vale daqui para frente; o passado é reinterpretado em leitura, não reescrito em pedra; e quando reescrever for inevitável, a migração é ela mesma versionada, testada (capítulo 16) e reversível.

As três respostas formam a política de compatibilidade do sistema, e "não temos uma" também é uma resposta, só que paga no escuro.

Depreciação é disciplina, não aviso

Toda equipe deprecia alguma coisa; poucas depreciam *bem*, porque depreciação bem-feita é processo com quatro peças e quase sempre alguém corta três. O nome do dono, quem responde pela coisa deprecada, porque "o time" não atende telefone. A data, não "em breve", mas um limite com calendário, porque sem vencimento a deprecação vira ornamento e o código morto acumula em cima do código vivo. O substituto, a seta que aponta para onde ir, porque a pessoa que consome a coisa velha não leu a ata da reunião que decidiu o novo caminho. E o número que quase ninguém olha: **quantos consumidores ativos ainda chamam a coisa na semana do corte.** Sem esse número, a remoção é fé, e fé, no capítulo 14, foi classificada como o que mora onde a verificação não vai.

O caso mais instrutivo de deprecação difícil não é em produto: é em linguagem, e este livro o vive em primeira mão. As emendas abertas que as medições dos capítulos 5 a 17 produziram: #816 (`val` anotado sem imutabilidade), #818 (record com `extends` sem igualdade por conteúdo), #819 (switch-expression cruzando hierarquias com bytecode inválido), não são só bugs; são *comportamento publicado por omissão*. Desde que foram descobertas, existe código, o deste livro inclusive, aprendendo a conviver com elas: conversões escritas na forma statement, comparações campo a campo, ressalvas no texto. O fix correto da #818, devolvendo igualdade por conteúdo a records com `extends`, **é uma breaking change para quem aprendeu a depender da identidade**: o `Set` que deduplicava por referência, o `==` que servia como comparação de existência. Rodinei as três sondas hoje de novo, contra o mesmo jar (o `lib/kof.jar` de 8 de outubro, o que o CLI carrega): comportamento idêntico, bug por bug. A re-rodagem não é futilidade, é o método deste capítulo em uma linha: *medição com data e artefato é a única unidade de documentação de evolução que não mente.*

[PENDENTE DE REVISÃO EDITORIAL, a direção de cada fix (corrigir para valor, congelar com nota, ou introduzir mecanismo novo) é decisão de projeto da autora da linguagem, registrada aqui como dilema, não como recomendação. O que o livro pode afirmar é a estrutura do dilema: numa linguagem 0.x, o silêncio do compilador também é contrato, e todo bug tem uma data de validade que a comunidade define ao descobri-lo.]

A convenção honesta da versão `0.x` existe exatamente para isso: ela declara "nada aqui é permanente ainda" e transfere o custo da mudança para quem consome, que é o preço de chegar cedo. A partir da `1.0`, a moeda inverte. O meio-termo que a prática consagrou e que este livro adota como vocabulário, sem citar a especificação com página, que a bibliografia carrega como pendente, é o *versionamento semântico*: o número maior avisa "quebrei promessas", o do meio "adicionei sem quebrar", o menor "consertei sem quebrar". A ideia que sustenta os três dígitos não é contabilidade: é *o consumidor poder decidir com o número na mão se precisa ler o changelog*. Versão que não pinça comportamento, e o caso deste projeto é o

exemplo vivo: `VERSION` dizendo `0.6.1`, CLI se identificando `0.5.0-beta`, jar datado de 8 de outubro, é rótulo, não promessa.

Refatorar é mover, e evidência é o piso

O capítulo 2 deu a definição operacional: refatorar é redistribuir complexidade sem mover o contrato. O capítulo 18 mostrou por que a redistribuição é o único movimento que a arquitetura permite sem demolição. Falta o que torna o movimento seguro, e é a tese do capítulo 16 cobrando sua quota: **a liberdade de um código de mudar de forma é exatamente a espessura da evidência que envolve o seu comportamento.** Com os testes nomeando comportamento na fronteira, a forma fica livre, você pode trocar a árvore inteira por baixo e a escada te avisa se o contrato escapou. Sem eles, a forma congela: cada classe é um andaime que alguém, em algum lugar, está sustentando pelo formato, não pela função. Por isso o código "impossível de refatorar" raramente é complexo demais, é *testado contra si mesmo*: seus testes descrevem a estrutura atual, e mudar a estrutura é, por definição, quebrá-los. A refatoração que não pode ser feita é sempre, em última instância, uma dívida de evidência, não de código.

A disciplina que transforma isso em prática cabe em três movimentos: estreitar o contrato antes de mover o corpo (a fronteira nomeada e verificada do capítulo 18: SEM043 na escala do módulo); mover com o verde atrás (a escada do capítulo 16 no degrau que cobre a junção que vai se mexer); e registrar o endereço novo (a caixa que agora guarda a dificuldade ganha nome no diagrama, senão a distribuição melhorou e ninguém sabe, e o próximo time dilui de novo).

Legibilidade é ativo de balanço

O "não mexe nisso que funciona" do capítulo 3 merece o assento contábil que ele é. Um módulo intocável não é dívida técnica no sentido de código feio, é **capital congelado**: funciona, ninguém toca, e cada dia que passa o mundo ao redor muda sem ele, até que a primeira mudança forçada encontra o módulo congelado com as regras de outro ano. O custo de entender um código não é pago uma vez na escrita; é pago a cada mudança que o código recebe, para sempre, por todas as pessoas que nunca vão conhecer o autor. Um desenho que empurra custo de entendimento para o futuro tem nome: empréstimo. E a taxa dele não aparece em nenhuma métrica de sprint, aparece na velocidade com que o produto inteiro desacelera dois anos depois, quando ninguém mais consegue tocar nada sem a bênção de quem lembra.

A boa notícia da contabilidade é que o ativo é recuperável, refatoração com evidência existe exatamente para descongelar capital, e o capítulo inteiro acima é o procedimento. A má notícia é que a recuperação também é paga em mudança, e mudança sem contrato congelado é a única barata: por isso a legibilidade se projeta no dia um, nomes do domínio (capítulo 10), casas certas para o comportamento (capítulo 11), fronteiras com contrato (capítulos 14, 18), e não se conserta no dia dois sem orçamento.

Evolução é deriva de distribuição

Fechando o capítulo com a lente do 18 apontada para o tempo. Arquitetura não apodrece porque o código envelhece, o código não tem idade, tem histórico. Ela apodrece porque **a distribuição de mudança do mundo anda, e a distribuição de complexidade do sistema ficou parada.** O módulo que escondia com maestria a instabilidade do provedor de câmbio vira burocracia no dia em que o câmbio estabiliza e a instabilidade migra para o calendário de liquidação, e o calendário, ele, mora exposto em doze lugares. A auditoria de evolução não pergunta "o código está limpo?"; pergunta *"o que este desenho esconde ainda é o que mais muda? e o que ele mostra ainda é o que mais importa?"*. Quando as duas respostas divergem, a arquitetura não está errada, está *vencida*, e o movimento é redistribuir (com evidência, pelos três passos da seção anterior), não reformar por gosto.

É por isso que o capítulo 18 terminou em costuras: a costura é o ponto do desenho onde a deriva esperada tem endereço, o lugar onde a próxima mudança vai bater *e já sabe onde*. Um sistema sem costuras é um sistema que aposta que o mundo parou. Nenhuma equipe admite fazer essa aposta; toda equipe que não deixa costuras a faz.

A Parte IV fecha no próximo capítulo com a propriedade que nenhuma das anteriores entrega sozinha, e que a manutenção expõe sem misericórdia: segurança. Não o componente nervoso com nome de firewall, a segurança como consequência de arquitetura, de distribuição de confiança, de quem pode mudar o quê sem ser notado: a propriedade do projeto inteiro, ou de nenhum.

Capítulo 20 — Segurança como propriedade do projeto

A auditoria passou. O relatório listou as verificações: criptografia aprovada (biblioteca reconhecida, algoritmos atuais), autenticação aprovada (tokens assinados, expiração correta), cabeçalhos aprovados, dependências varridas contra o banco de vulnerabilidades públicas. Seis semanas depois, o incidente não usou nenhuma das peças que falharam, usou uma que não estava na lista: um endpoint interno de relatório, exposto "temporariamente" durante uma migração, sem autenticação porque "é rede interna", que aceitava um filtro `id` e devolvia qualquer conta de qualquer cliente, em lote, porque a paginação do relatório não conferia se quem pedia tinha acesso àquela linha. Nenhum componente estava vulnerável. O sistema estava.

A pergunta "este sistema é seguro?" está malformada, e a auditoria de checklist é a forma institucionalizada de não percebê-lo. A pergunta bem-formada tem três espaços para preencher: *seguro contra quem* (o atacante aleatório, o insider, o concorrente com orçamento, ameaças diferentes, orçamentos diferentes), *de proteger o quê* (quais dados, quais fundos, quais identidades, a coisa que ninguém nomeia é a que ninguém protege), *com que orçamento* (a fricção que o produto aceita pagar pela fricção que o atacante pode infligir). Segurança não é uma peça que se instala; é uma **propriedade do projeto inteiro**: do tipo ao deploy, da mensagem de erro à tabela de paridade, ou não é de nada. Este capítulo fecha a Parte IV mostrando por que as quatro qualidades anteriores, correção, desempenho, manutenibilidade, arquitetura, convergem aqui, e por que a linguagem, que ocupou a Parte II inteira, é apenas um dos campos onde a propriedade se decide.

O que a linguagem tira da mesa, e só da mesa

A distinção prometida na introdução chega ao veredito. A segurança **oferecida pela linguagem** é real, mensurável e limitada: cada classe de bug que o tipo torna inexpressável é uma classe de ataque que deixa de existir naquele código. Nulabilidade no tipo elimina a família inteira dos null dereference no caminho verificado; imutabilidade de `record` elimina a mutação compartilhada entre componentes que se ignoram; a exaustão do `sealed` elimina o "esqueci de tratar o caso novo" que, em parser de entrada hostil, é porta; o contrato de interface cobrado pelo SEM043 elimina o componente que finge implementar o que promete. São ganhos estruturais, atacam *classes*, não instâncias, e por isso valem o que valem: para sempre, em todo o código, sem depender de que alguém lembre. É o capítulo 1 no seu papel de segurança: a estrutura que impede é a revisão que nunca dorme.

A segurança **exigida pela arquitetura** é tudo o que sobra, e sobra quase tudo o que importa. Nenhum sistema de tipos decide *quem pode*; nenhum compilador sabe se o `id` que chegou no parâmetro pertence ao dono da sessão. O confused deputy não é bug de tipo, é desenho que emprestou autoridade sem nomear procuração. A escala horizontal do relatório do início não era um valor fora do tipo: era um inteiro perfeitamente tipado carregando uma intenção que ninguém validou. A linguagem remove classes de *erro*; a arquitetura decide sobre *malícia*, e malícia não respeita gramática, ela procura exatamente a fronteira onde o seu modelo termina e o do atacante começa. A frase que separa os dois mundos: **a linguagem protege o**

programador de si mesmo; a arquitetura protege o sistema de quem não é o programador. Confundir as duas coisas produz o pior artefato da indústria, que é o sistema criptografado de ponta a ponta com o segredo no repositório público, cada componente impecável, a composição ingênua.

A stdlib medida, e o que "medida" quer dizer em segurança

Kof tem uma superfície de segurança documentada: `kof.security`: senhas com PBKDF2, AES-GCM, JWT com HS256, comparação em tempo constante, rate limiting, sessões, com a ambição declarada de *secure by default*. Este livro não publica promessa sem medição, e segurança é o domínio onde a medição própria vale menos do que parece: um hash que "roda e devolve algo" não está verificado; está apenas se repetindo. A verificação mínima honesta é **acordo com uma referência independente**. Foi o que fiz com o material desta rodada:

```
main() {
    var digest = crypto.sha256("kof")
    println(digest)
    var h = passwords.hash("hunter2")
    println(passwords.verify("hunter2", h))
    println(passwords.verify("wrong", h))
}
```

`crypto.sha256("kof")` devolveu, nos três alvos: JVM, nativo e JavaScript, o digest `06885364…8e7a6635`, byte a byte idêntico ao `hashlib.sha256` do CPython rodado na mesma máquina como oráculo independente. O ciclo hash→verify de senha devolveu `true` para a senha correta e `false` para a errada, nos três alvos, com o hash visivelmente salgado e não-trivial. É a verificação mais forte que este livro produziu, e defina bem o que ela cobre: *os primitivos medidos concordam com a referência neste build, nestes targets*. O que ela não cobre, e o texto não pode afirmar: as propriedades que não aparecem num round-trip, a força real dos parâmetros do PBKDF2, a resistência da comparação em tempo constante a ataque de temporização local, a ausência de vazamento nos modos de erro do AES-GCM. Essas vivem na confiança da implementação, que é exatamente a moeda que este capítulo veio desconfiar: *confie, e confira, inclusive o verificador*.

E a tabela de paridade, que o capítulo 9 apresentou como documento arquitetural, revela-se documento de segurança: os gaps SECN que a própria documentação registra, primitivas ausentes em riscv64/aarch64 (SECN000), ChaCha20 fora de x86 nativo (SECN002), `Secret`/`KeyHandle` fora da JVM (SECN008), troca de chaves só na JVM (SECN012): *não foram re-medidos por esta produção, são lidos da documentação oficial com data*, e dizem o que qualquer leitura honesta da multi-target diria: **"a linguagem oferece X" muda de significado por alvo, e segurança é justamente o domínio onde essa ressalva deixa de ser detalhe de rodapé e vira decisão de arquitetura.** Um produto que precisa de troca de chaves no nativo não tem, hoje, essa linguagem nesse alvo, e descobrir isso em produção é descobrir tarde demais. A tabela de

paridade lida como documento de segurança é a versão de linguagem do princípio do resto do capítulo: a superfície confiável é a que foi *nomeada e verificada*, não a que a brochura descreve.

Distribuição de confiança: a segurança do capítulo 18

O capítulo 18 definiu arquitetura como decisão sobre onde a complexidade mora. Lido pela lente da segurança, o enunciado ganha a sua forma mais dura: **arquitetura é decisão sobre onde a confiança mora, e cada seta é uma decisão sobre o que ela custa quando a confiança estava errada.** Toda fronteira no desenho responde, quer seja perguntada, a uma pergunta de segurança: quem autentica o chamador deste módulo? quem autoriza este acesso? o que vaza se este componente for *completamente comprometido*, a pergunta que o desenho de perímetro nunca fez e o incidente responde por ele?

O menor privilégio, nesse vocabulário, deixa de ser frase de política e vira a distribuição do capítulo 18 em modo consequência: acesso é complexidade de confiança, e diluir acesso "por garantia", o serviço com credencial de administrador "porque às vezes precisa", o secret copiado no config "temporário" desde 2021, o endpoint interno sem autenticação "porque está na rede interna", é a diluição do arredondamento em doze serviços, só que com o atacante no lugar do esquecimento. A regra que vale para as duas: **toda cópia de privilégio é uma superfície; toda superfície precisa de dono, data e motivo nomeável.** O `if (tipo == 'E')` duplicado doze vezes produz um bug difícil de achar; o privilégio duplicado doze vezes produz uma porta difícil de fechar.

A validação na fronteira, que apareceu no capítulo 12 como disciplina dos estados inválidos e no 13 como política do dado persistido, fecha o círculo aqui: **tudo que cruza uma fronteira de confiança é entrada hostil até que o contrário seja verificado**: não por paranoia, mas porque a alternativa é confiar no que você não pode auditar: o cliente, o parceiro, o outro serviço, o humano com SQL. A entrada validada no parse, rejeitar antes de interpretar, nomear o formato aceito, é o `sealed` do capítulo 1 aplicado ao mundo: os estados inválidos deixam de existir no interior, porque foram barrados na porta com motivo e sem stack trace no erro. E o erro do capítulo 14 completa o circuito: a mensagem que responde "o que faço agora" para o parceiro legítimo responde também, para o curioso, "o que você não deveria saber", o stack trace no `500` é a fronteira mais barata de fechar e a que quase ninguém fecha.

O tempo ataca de graça

O capítulo 19 estabeleceu que a arquitetura vence quando a distribuição de mudança anda e o desenho fica. A segurança é o domínio onde essa deriva cobra juros compostos: o sistema nascido impecável envelhece num mundo que ataca de graça. A dependência sem CVE no dia do deploy ganha uma na terça; o algoritmo considerado forte na revisão de design é quebrado por um paper dois anos depois; o funcionário com acesso legítimo muda de time e esquece de perder o acesso; o endpoint "temporário" sobrevive a três reorganizações porque ninguém é dono dele, e a depreciação sem dono nem data do capítulo 19 é exatamente o inventário de

superfícies que a auditoria de segurança vai encontrar e não conseguir fechar, porque ninguém sabe quem pode matar.

Daí as três disciplinas que fecham a seção, todas ecos de capítulos anteriores com o sobrenome trocado: **nomear o dono** de cada superfície, credencial e exceção, sem dono, o ativo vira passivo no primeiro inventário; **rodar o inventário**: o que está exposto hoje, para quem, com que privilégio, e a resposta precisa ser um artefato executado, não uma memória coletiva (a escada de evidência do capítulo 16: segurança sem verificação automatizada é opinião); e **tratar o disclosure como projeto**: o `SECURITY.md` do próprio Kof4j é material didático aqui: canal privado para vulnerabilidade, política explícita de "PoC bem-vindo; exploit weaponizado, não", recusa de issue pública antes do fix coordenado. É a deprecação honesta aplicada ao ataque: a janela entre descobrir e corrigir é o problema do *outro*, e a disciplina de fechá-la em coordenação é o que separa segurança de postura.

E a versão, sempre a versão: o projeto que não sabe *qual* build roda, e a deriva `VERSION 0.6.1` contra CLI `0.5.0-beta` que atravessa este livro inteiro é o exemplo vivo e medido, não pode auditar, não pode provar patch, não pode responder ao incidente com a única informação que importa: *você está afetado?* Nome do artefato, data da medição, alvo: a fórmula do capítulo 19 vale palavra por palavra em segurança, porque a pergunta "estamos vulneráveis?" é uma pergunta de *medição com data*, não de confiança em rótulo.

O livro aplica o critério a si mesmo

Fecho com a honestidade que a Parte II exigiu da linguagem e que agora cabe ao próprio projeto. Este livro passou 19 capítulos defendendo que a credibilidade de uma filosofia se mede na aplicação à ferramenta de quem a defende. A ferramenta deste livro, o compilador com que ele produziu cada exemplo, é uma 0.x com cinco issues abertas que ele mesmo descobriu e reportou: #816, #817, #818, #819, #820. Nenhuma é de segurança explorável; todas são exatamente o que a seção anterior descreveu como perigo: *comportamento publicado por omissão*, promessas da constituição declarada que a efetiva não honra, descobertas por quem mediu em vez de quem citou. O livro não esconde as suas porque o seu critério não permite: a superfície confiável é a nomeada e verificada. Um leitor que use o Kof seguindo este livro sabe, com data e artefato, o que foi medido, o que é documentado sem re-medição, e o que está aberto, que é a definição operacional de *confiança informada*, a única espécie que a segurança do projeto inteiro consegue sustentar.

A Parte IV começou perguntando como provar o comportamento, como medir sem mitologia, onde a complexidade e a confiança moram, e o que acontece quando o tempo cobra. A resposta de tudo é a mesma: segurança não é o quinto requisito ao lado dos outros quatro; é o nome que se dá aos outros quatro quando existe um adversário. A Parte V fecha o livro na pergunta que atravessa 2026 inteira, o que muda na engenharia orientada à intenção quando metade do código deixa de ser escrita por quem o assina, e por que a resposta, no fim, é a mesma de sempre, dita para uma máquina: *intenção que outra inteligência possa ler, e verificar.*

Parte V — Engenharia em um mundo automatizado

Capítulo 21 — Programar com inteligência artificial

A geração automática de código alterou o custo de produzir instruções, mas não alterou a física do software. Uma função gerada em dez segundos por um modelo estatístico parece tão limpa, tão idiomática e tão confiável quanto uma função escrita à mão por quem passou três horas nela. A diferença não está na superfície: está na ausência de intenção por trás da digitação.

Quando o custo de produzir código cai radicalmente, a posição de quem programa muda: passamos a ser leitores responsáveis por texto que ninguém escreveu com intenção prévia. A pergunta central não é se a ferramenta substitui a engenharia, mas o que acontece quando o volume de código cresce sem que o entendimento cresça na mesma proporção: quando produzir instruções barateia, o que passa a escassear?

O gargalo mudou de lugar

Meio século de prática organizou a engenharia de software em torno de uma escassez: escrever código é caro. Prazos, metodologias, arquitetura, revisão de código; tudo, no fundo, raciona a escrita. A geração em volume desloca o gargalo para os dois pontos que sempre importaram e sempre foram minoria na conversa: saber o que se quer (especificação: nomes, invariantes, fronteiras, o que é entrada hostil) e poder provar o que se obteve (verificação: a escada de evidência do capítulo 16). O código que antes custava cem linhas de digitação agora surge num instante; o item escasso passa a ser a decisão sobre o que o sistema deve ser.

É aqui que a tese deste livro ganha peso operacional: código de qualidade é código cuja estrutura expressa a intenção do sistema com o mínimo de complexidade acidental possível. Quando quem produziu o código não tinha intenção alguma, a intenção precisa estar inteira na estrutura e nos testes, ou não estará em lugar nenhum.

Código gerado é código sem memória

O capítulo 3 descreveu o sistema como orçamento de atenção e nomeou o pior custo: o de lembrar, as verdades que o código não carrega e alguma cabeça precisa carregar no lugar dele. Quando uma pessoa escreve código, uma parte da intenção sobrevive nela: por que o campo é opcional, o que acontecia no incidente de março, qual condicional protege o sistema contra o fornecedor que reenvia faturas. Essa memória humana é imperfeita, mas existe. O código gerado por máquina não tem nem isso: por trás do trecho idiomático não há ninguém que um dia sofreu com aquele caso de borda.

A consequência é direta: tudo o que a prática chamava de "está na cabeça de alguém" deixa de existir por construção, e tudo o que a estrutura não carrega fica desprotegido. Um time que aceita código gerado sem exigir intenção expressa na estrutura está aceitando um colaborador com memória zero e acesso total. Daí as duas exigências que transformam a ferramenta em aliada: a estrutura que impede (capítulo 1: tipos, exaustão,

contratos) e o teste que nomeia comportamento (capítulo 16: especificação executável). O código gerado não enfraquece esses requisitos; torna-os condição elementar de sobrevivência.

A função que compila, roda e mente

Um exemplo concreto ilustra o problema. Duas funções de média aritmética, ambas aprovadas pelo compilador, uma delas com falha silenciosa:

```
Int media(Int a, Int b) {
    return (a + b) / 2
}
```

O comando de checagem passa sem erros. Com valores pequenos, `media(3, 5)` devolve `4`. O problema surge no limite: `media(2000000000, 2000000002)` devolve `-147483647`. A soma estoura a capacidade do inteiro de 32 bits em silêncio, e a média de dois números positivos enormes devolve um número negativo.

A versão correta, `a + (b - a) / 2`, passa pelo mesmo compilador e devolve `2000000001`. As duas versões são indistinguíveis para qualquer analisador sintático comum. O que as separa é uma pergunta de engenharia: certo para quais entradas? A média correta nunca sai do intervalo entre os operandos; essa invariante pertence ao domínio, não à sintaxe.

Esse é o verdadeiro sentido do que o mercado chama de alucinação: um estado inválido (capítulo 12). O código existe num estado proibido pelo domínio, sem nenhum guarda designado para aquele portão. O compilador vigia a forma; o runtime vigia o que foi declarado; o teste vigia o que alguém nomeou. A resposta não é desconfiar abstratamente da ferramenta; é aplicar a escada de evidência com rigor. O gerador não pode ser a última opinião sobre o próprio trabalho, exatamente como nenhum autor humano pode.

A revisão muda de foco

Quando o código foi gerado mecanicamente, a revisão não pode mais perguntar "o que você pensou ao fazer isso?", pois não houve pensamento. As perguntas relevantes passam a ser outras: o que esta estrutura diz sobre o sistema? (capítulo 1). O que este teste protege, e ele é capaz de falhar? (capítulo 16). Onde a dificuldade mora, e quem pode vê-la? (capítulos 18 e 20). A revisão de código vira, em definitivo, revisão de intenção. Procura-se a intenção que o sistema de fato possui: a que está no tipo, no teste, na fronteira.

O volume traz uma conta adicional: dez vezes mais código para revisar significa dez vezes mais superfície de manutenção se a evidência não absorver a carga. A escada de evidência é o sistema digestivo do volume: onde há contrato verificado pelo compilador, o código inadequado quebra na entrada; onde há teste de comportamento nomeado, quebra na integração. Sem isso, o código entra e o incidente em produção cobra a revisão com atraso e juros.

O deslocamento da atenção

Sem recorrer a estatísticas artificiais de produtividade, é possível observar claramente o deslocamento da atenção em equipes que adotam ferramentas generativas:

Especificar passa a ser a tarefa primordial: definir critérios claros, limites de domínio e regras de negócio antes de solicitar código. Verificar passa a ser o gargalo: a capacidade de gerar supera a de validar, tornando a evidência o bem mais valioso. E desconfiar passa a ser competência técnica: checar o que a máquina entrega, confirmar contra o compilador e confrontar com os testes.

A ferramenta não elimina a engenharia do processo; elimina a ilusão de que digitar linhas com pressa equivale a construir software sólido. A engenharia continua sendo o que sempre foi nos times experientes: decidir o que é correto e construir a evidência verificável de que aquilo foi alcançado.

O próximo passo

Se o gargalo está na especificação e na verificação, e se o código pode nascer sem autor humano, surge a pergunta inevitável: de quem é o sistema? O capítulo 22 busca essa resposta: o desenvolvedor como responsável pelo sistema, num cenário em que a responsabilidade não decorre da digitação das linhas, mas da decisão consciente sobre o que entra em produção.

Capítulo 22 — O desenvolvedor como responsável pelo sistema

O capítulo anterior terminou com a pergunta sem resposta confortável: se o código pode não ter autor, se a escrita barateou a ponto de poder ser delegada a uma máquina sem memória nem vergonha —, então de quem é o sistema? A indústria respondeu em alto e bom som, sem querer, no modo default de quase toda organização: **de ninguém.** O incidente chega, a reunião pergunta "quem escreveu isso?", e a pergunta já é a confissão: a empresa inteira organizou seus processos em torno de uma categoria, o autor, que o próprio processo de produção tornou obsoleta. Enquanto se procura o culpado que não existe, o responsável que poderia existir nunca é nomeado.

Este capítulo recusa a saída cômoda (declaração de princípios sobre ética profissional) e a saída falsa (a culpa do tempo: "os tempos mudaram, a responsabilidade se diluiu"). Propõe uma troca de fundamento: **responsabilidade em sistemas de software não decorre de autoria. Decorre de posição.** E a posição, num sistema técnico, tem um nome específico que a gerência adora esvaziar e que este capítulo vai encher de conteúdo até ele doer.

A posição que detém o "não passa"

Toda organização técnica tem, em algum lugar, um ponto onde a mudança se torna fato: o merge que entra no trunk, o build que vai para produção, a porta que abre o endpoint. Esse ponto é uma **posição de veto**: quem o ocupa decide, no último instante, se o sistema ganha ou não ganha aquela forma. E a tese deste capítulo cabe numa frase: **o dono do sistema é quem pode dizer "não passa" e diz.** Não quem escreveu a linha, essa resposta morreu com a geração em volume. Não quem assina o orçamento, esse pode dizer "não pago", que é outra coisa e outro capítulo. Quem detém, na prática, o veto informado sobre o que entra.

Dessa definição seguem duas consequências que a indústria prefere não olhar. A primeira: se ninguém ocupa essa posição, se o merge entra porque o CI está verde e o CI está verde porque ninguém escreveu o teste que perguntaria, se o deploy vai porque "sempre foi assim", se o endpoint ficou aberto porque ninguém é dono do "temporário", então **o sistema não tem responsável, e a organização escolheu isso** por omissão, que é a forma mais barata de escolha. A segunda consequência é a que dá título ao capítulo: onde a organização não nomeia a posição, ela existe de fato em quem a exerce, e quem a exerce sem nomeação é o desenvolvedor de plantão, no merge das onze da noite, sem orçamento, sem mandato e sem o vocabulário deste capítulo. A responsabilidade não desaparece por falta de designação; apenas fica com quem não foi convidado a carregá-la.

O que a posição exige, formulado como deveres e não como virtudes, é a convergência de tudo o que os vinte e um capítulos anteriores construíram, cada qualidade deste livro, vista pelo lado de quem responde, vira um dever específico.

Os quatro deveres da posição

Primeiro: entender o comportamento em produção, ou saber onde está o limite do seu entendimento. O capítulo 3 estabeleceu que o custo cognitivo é a fatura real do sistema; o capítulo 17 mostrou o HTTP síncrono que ninguém considerou porque "rede não conta como CPU". O dever não é onisciência, é honestidade mapeada: saber o que o sistema faz sob as cargas, falhas e entradas que o produto encontra de verdade, e nomear por escrito o que não se sabe. "Não sei ainda se o relatório aguenta o cliente novo" é um ato de engenharia completo quando tem dono e prazo; é a mesma frase sem dono e prazo que, dois capítulos depois, virou o incidente com nome de desastre natural. O capítulo 12 deu a forma: o estado inválido denuncia a invariante sem vigia; o estado *desconhecido* denuncia a pergunta sem dono.

Segundo: manter viva a evidência do que se afirma correto. O capítulo 16 deu a escada, compilador, unitário, contrato, produção, e o dever correspondente: o verde que você não questionou é a sua assinatura falsa. Quem ocupa a posição de veto assina, a cada merge, que *a evidência disponível sustenta aquela mudança*; assinar sem ler a evidência, ou sem saber que ela não cobre a fronteira que mudou, não é otimismo, é falsidade ideológica com commit hash. A disciplina concreta vem do capítulo 21 aplicada a toda fonte de código, humana ou não: perguntar "certo para quais entradas?" antes de "compilou?". A média que devolve `-147483647` não distingue quem a escreveu; o verde no CI não distingue o que foi verificado do que foi repetido.

Terceiro: conhecer as fronteiras de confiança, e tratá-las como o capítulo 20 exige. Toda seta do diagrama é uma decisão sobre o que se aceita sem verificar; o dever é que a decisão seja *tomada*, com dono e motivo, em vez de herdada do desenho que alguém fez "temporário" em 2021. O desenvolvedor responsável não é o que confia menos, é o que sabe *onde* está confiando, e consegue apontar cada fronteira sem hesitar. A pergunta do dever: "se este componente estiver completamente comprometido, o que acontece?", se a resposta demora, a fronteira não foi desenhada, foi presumida.

Quarto: nomear o que não se sabe, com dono e prazo. Este é o dever mais estranho e o mais decisivo, porque os três anteriores falham sem ele: o entendimento, a evidência e as fronteiras têm sempre um resto, e o resto ou é nomeado ("não sei se o retry do parceiro é idempotente; dono: eu; prazo: quinta; até lá: o endpoint devolve 501") ou é invisível, e o invisível é onde o sistema morre. O capítulo 14 chamou de fé o que fica sem número na semana do corte; o capítulo 19 chamou de depreciação sem dono o que vence sem aviso. Nomear a ignorância com prazo é a única forma de gestão que funciona contra o que não se pode medir ainda, e é a marca que separa o responsável do otimista, que também nunca sabe, mas assina como se soubesse.

A delegação não transfere a posição

Aqui o capítulo recolhe a analogia que o 21 deixou armada, e ela não é nova: a indústria já resolveu este problema uma vez, com o nome de *terceirização*. Ninguém aceita hoje que "o fornecedor fez" funcione como resposta para o incidente, a responsabilidade sobre o que o sistema faz não migra com o trabalho

braçal; ela fica onde fica o veto. O framework que você adotou não assina o seu post-mortem. A biblioteca que traz o CVE não atende o cliente. O consultor que desenhou o modelo foi embora com o conhecimento tácito, e o "não mexe" ficou para quem ficou. O capítulo 14 generalizou: tudo o que expõe vira permanente; o corolário é que tudo o que se delega continua *sendo seu*, a delegação move a execução, nunca a posição.

A geração por IA não é a primeira tecnologia a dissolver a autoria na produção; é a primeira a dissolver *por inteiro*, sem contrato, sem fornecedor identificável, sem o e-mail para o qual apontar o dedo. E a conclusão se segue sem exceção histórica: quem adota o gerador e não assume o dever dos quatro itens não terceirizou o trabalho; **terceirizou a responsabilidade, que é a única coisa que não se pode terceirizar sem que o sistema perca o dono.** A máquina não pede o veto; alguém tem que segurá-lo. O desenvolvedor que diz "a IA escreveu" está na posição, é *por isso* que a frase é uma confissão, e não uma defesa.

O incidente é a unidade de responsabilidade

Tudo acima é abstrato até a madrugada. O incidente é o momento em que a organização descobre, em tempo real, quem ocupa a posição, e o ritual que ela escolhe diz tudo sobre a arquitetura moral do sistema. A cultura da culpa pergunta "quem fez isso?", e obtém três danos: a resposta errada (ninguém fez, ou fez quem não devia ser punido), a ocultação do próximo erro (quem teme a pergunta esconde a evidência, e o capítulo 16 morreu de causas naturais), e a perda da única informação que importa, que está *no sistema*: por que a posição de veto deixou aquilo passar, por que a evidência não cobria, qual fronteira estava presumida. A cultura responsável pergunta "que decisão de design, de processo ou de evidência tornou isso possível, e quem agora é dono de cada uma?", e trata a resposta como o post-mortem do capítulo 19 trata a deprecação: **dono, data, substituto.** Sem dono, a ação corretiva vira ata; sem data, vira intenção; sem substituto para a prática que falhou, o mesmo incidente espera a mesma reunião daqui a um ano, com as mesmas caras, mais cansadas.

O detalhe que fecha a seção e prepara a próxima: o desenvolvedor responsável é, também, quem *constrói as condições* para os quatro deveres existirem, e isso inclui, quando a organização não as fornece, recusar a posição decorativa. Um veto sem mandato é teatro; uma evidência sem tempo para existir é cobrança, não cultura. Nomear isso é o dever número cinco que ninguém escreve nos documentos: **recusar os arranjos que tornam a responsabilidade impossível de exercer**: e essa recusa, feita cedo e em voz alta, é o ato mais raro e mais alto da profissão.

O caso do próprio livro, sem romantismo

Este projeto tem um exemplo pequeno e verificável da tese, e ele não envolve heroísmo. A linguagem que serve de objeto ao livro recebeu, nestas semanas de escrita, cinco issues públicas (#816 a #820) nascidas de medição, um compilador que aceita `val` sem vigiar, um switch que mente no bytecode, um profiler que imprime zero por cento sempre. As issues existem no repositório, com reproduções, datas e artefatos. O que este capítulo pode afirmar sobre a responsabilidade da autora da linguagem é *exatamente isso e nada além*:

os defeitos foram descobertos por medição externa, registrados em canal público e coordenado, sem ocultação, e os fixes pertencem ao futuro, com a autora decidindo a direção de cada um, inclusive a dilema de que corrigir alguns é breaking change (o caso da #818, que o capítulo 19 deixou em aberto). Não há história pessoal para contar aqui, e o livro não inventa uma: **[PENDENTE DE REVISÃO EDITORIAL, se este parágrafo deve incluir a perspectiva real da autora sobre como recebe bugs públicos na própria linguagem, essa palavra é dela; a estrutura do exemplo, não.]** O que o caso demonstra é a forma: responsabilidade sem autoria romântica, a linguagem é *dela* pela posição que ocupa, não pelo código que ainda escreve sozinha; e a medição externa que a incomoda é o mecanismo, não a ofensa. É o capítulo 20 fechando o círculo: *confie, e confira, inclusive o verificador*, vale para a ferramenta, para a máquina, e para a pessoa que assina.

O capítulo 21 dissolveu a autoria; este devolveu o que importa no lugar dela: a posição, os deveres, e o nome de quem os exerce. Falta a última pergunta da Parte V, a que só uma pessoa que construiu uma linguagem pode fazer sem fingimento: se programar é decidir o que pode ser dito, o que é construir a língua em que se decide? Construir linguagens não é escrever compiladores, é fabricar possibilidades e condenar outras. O capítulo 23 fecha o argumento teórico do livro; o 24 recolhe tudo.

Capítulo 23 — Construir linguagens é construir possibilidades

Toda decisão de engenharia pode ser revertida dentro do seu próprio nível: o módulo se refaz, a arquitetura se particiona de novo, o banco se migra. Existe uma classe de decisão que não, e é a que este livro persegue desde o capítulo 5: decidir a linguagem é legislar o pensamento de todo código escrito sob ela. Uma biblioteca inadequada troca-se num ciclo curto; uma linguagem que torna a ausência invisível produz, durante anos, sistemas que ignoram que valores podem faltar.

Por isso, construir uma linguagem é o experimento onde todas as abstrações deste livro se tornam concretas: cada sintaxe é uma distribuição de custo cognitivo (capítulos 3 e 5), cada tipo é uma aposta sobre o que o mundo permite dizer (capítulo 6), cada mensagem de erro é uma interface de usuário (capítulo 8), e cada funcionalidade é uma promessa que a constituição efetiva precisa honrar (capítulos 19 e 21). Não existe engenharia mais direta que a de linguagens, porque não existe decisão cujo custo de erro apareça multiplicado no trabalho de tantas pessoas.

O manifesto confrontado com o compilador

Kof possui um manifesto público, documentado em `PHILOSOPHY.pt_BR.md`, composto por sete princípios. Um livro que passou vinte e dois capítulos exigindo que a constituição efetiva seja avaliada com base em evidências deve tratar o próprio material de origem com o mesmo rigor.

I. "A linguagem deve aproximar o código da intenção": o documento afirma que, se uma parte do código existe apenas porque a ferramenta exige, sua necessidade deve ser questionada. O capítulo 1 definiu intenção como o que a estrutura torna recuperável; a medição prática mostrou a linguagem buscando honrar isso através de nulabilidade no tipo, exaustão vigiada e contratos de interface cobrados pelo compilador. Também revelou onde o compilador ainda precisa evoluir, como nos casos das issues abertas durante os testes deste livro. O princípio permanece válido, e a implementação avança de forma aberta.

II. "Simplicidade não significa limitação": o princípio defende que a complexidade deve existir porque o problema exige, não porque a ferramenta impõe. É a síntese do capítulo 2. O caso emblemático é a ausência de operadores como `?.` e `?::` o narrowing explícito impede que a decisão sobre ausência fique escondida em atalhos sintáticos. A simplicidade aqui cobra verbosidade consciente para manter a intenção visível no ponto de chamada.

III. "A máquina importa": abstração de alto nível não precisa significar alienação do hardware. O comando `inspect` demonstra isso ao expor as operações geradas por um `reduce` de biblioteca; o profiler exibe consumo de memória e trocas de contexto; o backend nativo compila diretamente para assembly. A abstração se sustenta quando é possível examinar o funcionamento interno sem depender de suposições.

IV. "Pequenas coisas, bem feitas": no espírito Unix, ferramentas devem possuir responsabilidades delimitadas e composáveis. A interface de linha de comando reflete isso ao separar tarefas claras de checagem, execução, testes, inspeção e perfilamento. Cada componente resolve um problema focado.

V. "Não existe arquitetura sagrada": a arquitetura deve responder ao problema concreto, não a modismos. Uma linguagem que não impõe microsserviços nem monolito preserva a liberdade de escolha do time, assumindo a responsabilidade de operar com eficiência em ambos os modelos.

VI. "A plataforma deve trabalhar para o programador": automação deve remover burocracia, nunca autonomia. A máquina deve assumir tarefas repetitivas, geração mecânica e validações de rotina, deixando o julgamento de engenharia para quem decide. Uma ferramenta que decide pelo desenvolvedor retira autonomia; uma ferramenta que valida o que foi decidido remove burocracia.

VII. "Poder sem cerimônia": conceitos fundamentais devem compor com naturalidade. A combinação de tipos selados, correspondência de padrões, imutabilidade e funções puras entrega soluções completas sem a necessidade de recursos sobrecarregados na linguagem.

Construir em aberto

O registro de decisões arquiteturais em `docs/decisions/` contém decisões pontuais, enquanto o manifesto estabelece diretrizes amplas. A distância entre os princípios gerais e as decisões detalhadas reflete o amadurecimento natural de uma linguagem jovem. É exatamente nesse espaço que o trabalho de medição se insere: documentar o comportamento observado, apontar divergências e consolidar a especificação com base em evidências verificáveis.

Uma linguagem que publica sua tabela de compatibilidade com clareza e acolhe correções públicas demonstra maturidade de processo. A credibilidade técnica não depende de infalibilidade, mas da disposição para medir, reconhecer limites e corrigir desvios.

Toda equipe legisla sua própria linguagem

Construir compiladores é um caso específico; a natureza da decisão linguística se repete sempre que uma equipe define como seu código será escrito:

A equipe que escolhe uma biblioteca ou framework está distribuindo custo cognitivo para os próximos anos. O que a ferramenta facilita será escrito com frequência; o que ela dificulta será contornado.

A equipe que estabelece padrões de código está criando uma sintaxe interna. Se a regra perder o vínculo com o problema que a originou, transforma-se em burocracia.

A equipe que adota geração automatizada precisa reforçar contratos e testes, pois sem intenção explícita na estrutura, o código perde coerência ao longo do tempo.

Em qualquer escala, quem escolhe as ferramentas define o espaço do que a equipe consegue expressar. Compreender essa dinâmica é o passo fundamental para praticar engenharia com critério e responsabilidade.

O capítulo 24 fecha o livro reunindo os princípios desenvolvidos ao longo da obra em critérios práticos para o dia a dia.

Capítulo 24 — Uma engenharia orientada à intenção

O título deste capítulo é o nome que o livro inteiro vinha procurando sem pronunciar, e ele chega tarde de propósito: um nome no começo seria uma bandeira, e bandeira é o que os capítulos anteriores desmontaram em cada escola que visitaram. Vinte e três capítulos depois, a expressão pode ser dita sem dever nada a ninguém, porque ela não é uma escola nova, é o nome do critério que apareceu em cada página, a pergunta que se repetiu com vocabulário diferente desde a primeira revisão de código da introdução. Este capítulo fecha o corpo do livro fazendo três coisas que uma conclusão honesta precisa fazer: reunificar o argumento sem repeti-lo, responder "e na segunda-feira?" sem virar manual, e recusar as caricaturas que o próprio sucesso dele produziria, inclusive a de que este livro é um método.

O argumento, uma vez, inteiro

Tudo começou com uma frase sobre qualidade e a recusa de tratá-la como lema. A frase: *código de qualidade expressa a intenção do sistema com o mínimo de complexidade acidental possível*, foi desmontada em três variáveis, e cada uma se revelou um território de engenharia com instrumentos próprios.

Intenção mostrou não ser o que o autor quis dizer, mas o que a estrutura torna recuperável por quem não estava lá, e ganhou uma hierarquia: o que a estrutura impede vale mais do que o que ela separa, que vale mais do que o que ela nomeia. O compilador que recusa o `null` sem procedência, o `sealed` que enumera os lugares a tocar, a interface que cobra o método prometido: são intenções que sobreviveram ao seu autor porque deixaram de depender da memória de alguém. A Parte III mostrou a operação completa: modelar o domínio antes das classes, enumerar estados, eventos e invariantes com as palavras do negócio, para só então alocar cada regra ao vigia capaz de segurá-la: o tipo, o runtime, ou a boa-fé documentada. E a Parte V fechou o círculo com a descoberta que ninguém previu: quando o código pode não ter autor, a intenção estrutural deixa de ser virtude e passa a ser a única fonte que resta.

Complexidade mostrou não ser uma grandeza, mas uma contabilidade: a essencial, que o problema impõe e que nenhum desenho elimina, apenas nomeia, concentra e vigia; e a acidental, que compramos de bom ou mau grado, e cujo preço merece recibo. A lei de conservação, a dificuldade muda de endereço, de dono e de linguagem, atravessou a arquitetura inteira do livro: da camada que esconde sem absorver nada, ao serviço que distribui a falha e cobra pedágio em cada seta, ao `sealed` que transfere a memória de doze pessoas para um verificador que não dorme. O erro simétrico, nomeado e precificado: tratar como eliminável o que o mundo exige produz o modelo superficial que é compreensível até o dia em que o domínio cobra, e o dia sempre chega com juros.

O mínimo possível revelou a terceira variável que a frase esconde, e a mais humana: o orçamento de quem paga todas as contas, a cabeça finita que compreende, navega e lembra, que dorme, entra de férias e um dia pede demissão. A partir dele, tudo o mais do livro se reorganizou sozinho: convenções são decisões comprimidas que viram burocracia quando perdem o por quê; a linguagem é distribuição de custo cognitivo

com força de lei; os testes que descrevem a forma congelam exatamente o que a evidência deveria liberar; a segurança é o que acontece quando as setas de confiança ficam sem dono; e a responsabilidade não mora em quem escreveu, mora em quem pode dizer "não passa" e diz.

As três variáveis nunca foram independentes, e essa é a unidade do argumento: **intenção sem contabilidade de custo é elegância; contabilidade sem orçamento humano é ASCII art de arquitetura; e o "mínimo possível" só significa alguma coisa para um sistema que expressa intenções que alguém, em algum lugar, ainda precisa conseguir recuperar.** A engenharia orientada à intenção é a disciplina que faz as três perguntas ao mesmo tempo, na ordem em que a situação exigir, sem deixar nenhuma delas virar dogma.

Na segunda-feira: o caderno, não o checklist

Um livro que passou vinte e três capítulos desconfiando de listas não pode terminar com uma. Mas pode terminar com o que a lista tenta ser e nunca consegue: perguntas vivas, com as condições em que cada uma arde. O caderno do engenheiro orientado à intenção cabe em seis perguntas, e cada uma é uma porta para um capítulo onde ela foi trabalhada com o cuidado que um resumo não permite:

Diante de uma mudança: *o que esta estrutura diz sobre o sistema, e o que ela custa para dizer?* (capítulo 1). A pergunta se desdobra na revisão, no desenho, na auditoria da camada que ninguém sabe o que absorve. Se a resposta exigir memória institucional, o trabalho não é comentar, é mover a intenção para um dos três níveis: impedir, separar, nomear.

Diante de um problema difícil: *esta dificuldade é do mundo ou da nossa mesa?* (capítulo 2). O teste de realidade, aplicado sem dó: trocadas todas as ferramentas, a pergunta continua? Se sim, nomeie-a no modelo; se não, conte o que a sua resposta custa e para quem, e desconfie de qualquer decisão cujo custo ninguém consegue apontar.

Diante de uma regra, um processo, uma convenção: *qual problema ela criou, quem paga quando ela erra, e ela consegue dizer não?* (capítulos 4 e 19). As três perguntas separam a convenção viva da regra órfã, e a mesma contabilidade vale para o seu próprio guia de estilo, para o template do pull request e para a reunião semanal que ninguém lembra por que existe.

Diante de uma decisão de linguagem ou ferramenta: *o que ela torna barato dizer, o que torna caro, e o que a máquina vigia da diferença?* (capítulos 5 a 9). E a ressalva que a Parte II inteira ensinou a exigir: leia a constituição efetiva, não a declarada, o catálogo de diagnósticos, a tabela de paridade com data, os números que a ferramenta imprime quando você roda o caso exato que ela jura cobrir.

Diante de um prazo: *o que estou movendo para a cabeça de alguém, e qual é a data em que essa pessoa não estará lá?* (capítulos 3 e 10). A dívida que mora em pessoas vence sem aviso; a única renegociação disponível é mover a verdade para a estrutura, o tipo, o teste, o módulo, o contrato, antes que o entendimento vire intocável e o intocável vira legado no dia em que ainda era novo.

Diante do deploy: *o que eu não sei sobre este sistema, quem é o dono dessa ignorância, e qual é o prazo dela?* (capítulos 12, 14 e 22). A pergunta mais difícil de responder com honestidade e a única que separa a confiança da aposta: "não sei ainda se X" com dono e prazo é engenharia; a mesma frase sem dono é a próxima post-mortem com nome de azar.

Nenhuma dessas perguntas tem resposta automática, e a ausência de automação é o ponto. O caderno não substitui o julgamento; ele é o julgamento, exercitado até virar reflexo, na única forma que sobrevive à mudança do mundo: perguntas que continuam certas quando as respostas de ontem envelheceram.

O que isto não é

Quatro recusas finais, cada uma contra uma caricatura que este livro já viu acontecer com os seus antecessores, e a quinta, a mais difícil, contra a que ele mesmo merece.

Não é minimalismo. O livro não defende menos código, menos camadas, menos conceitos; defende que cada linha que existe diga o que faz e faça o que diz. Um `sealed` com três casos e um switch exaustivo é mais código que o `null` que ele substitui, e menos sistema para entender. A economia que importa é a da atenção, não a dos caracteres.

Não é pureza tipada. Houve páginas defendendo tipos como linguagem franca da intenção, e houve páginas mostrando o exato limite onde eles param: os termos do contrato que o tipo não alcança, o estado inválido que nenhum sistema expressa, a fronteira onde só a validação de runtime e a disciplina de arquitetura seguram. Quem terminar este livro achando que "tipar tudo" resolve o problema do capítulo 10 leu os capítulos 6 e 7 sem a ressalva que eles carregam.

Não é anti-metodologia. O livro não argumenta contra testes, revisão, arquitetura ou processo, argumenta contra a prática que sobreviveu ao seu porquê. Cada método visitado foi tratado pelo que ele pega para si, não pela tribo que o vende. E o capítulo 16 dedicou páginas a demonstrar que a evidência sistemática é a única que permite mudar sem medo: o inimigo deste livro nunca foi o método; foi o método sem dono.

Não é um manual de regras. E aqui a recusa se aplica ao caderno da seção anterior: as seis perguntas são critérios, não checklist, elas não dizem o que fazer, dizem o que perguntar, e a resposta pertence ao sistema, ao time e ao orçamento de cada segunda-feira. Um leitor que transformar as seis perguntas em sete regras quebra o capítulo 4 citando o capítulo 24. A diferença entre critério e regra é que o critério sobrevive ao seu autor; a regra precisa dele para continuar parecendo sensata.

E o livro não é a exceção ao próprio critério. Esta é a última recusa, e a mais necessária: *Intenção em Código* é, ele mesmo, um texto que pode virar burocracia. As frases dele circularão em reuniões sem os argumentos que as produziram; as perguntas virarão respostas de manual; o "o que esta estrutura diz?" será brandido como um porrete de gosto. Contra isso, o livro não tem defesa, tem, no máximo, a herança que deixou espalhada pelas suas próprias páginas: **desconfie deste livro com o mesmo rigor que ele aplicou a todos os outros.** Cada afirmação aqui tem data, medição, ressalva e um endereço de onde o contraexemplo

mora. Se alguma tese sobreviver a isso, sobreviveu por ser verdadeira; se cair, que caia pelo método que ela ensinou, e o caderno fica maior.

A aposta final

Por que a intenção? Por que não chamar a tese de "clareza", "simplicidade", "qualidade", palavras que a indústria já consumiu até o fim? Porque nenhuma delas nomeia o que está realmente em jogo, e a escolha da palavra é a última decisão de engenharia deste livro, tomada com plena consciência do que ela compromete.

Software é a única construção humana em que o artefato pensa no escuro: ele executa exatamente as decisões de quem o escreveu, para sempre, sem perguntar se o mundo ainda concorda. Um prédio em ruína denuncia a si mesmo; um sistema em ruína continua rodando e respondendo com confiança, e foi o que o capítulo 10 chamou de catedral, o que o 19 chamou de deriva, o que o 21 viu na média que devolve `-147483647` com o check verde. A alternativa a um artefato que pensa no escuro não é um artefato que pensa melhor sozinho, é um sistema que **carrega a luz consigo**: a intenção de quem decidiu, recuperável por quem não estava, verificável por quem chegou depois, sobrevivendo ao autor, à moda, ao incidente e ao tempo. Essa carga de luz tem nome, e o nome é o que este livro apostou: intenção.

O posfácio fecha o livro onde ele começou, na mesa de revisão, com o diff na tela, a pergunta na mão e a única promessa que a engenharia pode fazer sem virar brochura: *aqui dentro, alguém sabia por que as coisas são assim, e deixou isso escrito de um jeito que você consegue conferir.*

Posfácio

O livro começou numa mesa de revisão com duas pessoas errando pela mesma razão: cada uma segurava uma regra sem segurar o porquê dela. Termina na mesma mesa, com a cena invertida: alguém propõe uma mudança, e outra pessoa pergunta o que esta estrutura diz sobre o sistema e o que ela custa para dizer. A pergunta não veio de mim. Veio de um dos dois do começo do livro, o que estava certo e não sabia explicar, aprendendo a discordar com critérios. É a única forma de sucesso que este livro aceitou desde a primeira página.

O que este livro fez de diferente não foi ter respostas. Foi ter mediado as próprias. Cada afirmação sobre Kof tem uma linha no log de validação com data, comando e saída; cada citação externa tem um estado de verificação na bibliografia, algumas ainda pendentes, e o livro não esconde quais; cada tese tem o capítulo onde o contraexemplo mora. Isso não é modéstia de formatação: é a tese aplicada ao objeto. Se intenção é o que a estrutura torna recuperável, um livro sobre intenção que escondesse suas incertezas seria a demonstração negativa do que defende. Eu preferi o risco de ser conferido ao conforto de ser citado.

A regra que atravessou os capítulos, de confiar nos verificadores e ainda assim conferi-los, funcionou contra a ferramenta: os defeitos encontrados na linguagem durante a produção deste livro, registrados em issues públicas no repositório, não foram procurados; foram encontrados medindo exemplos que este livro ia publicar. Uma forma de trabalhar que não encontra defeitos na própria ferramenta não é melhor engenharia; é a que mediu menos.

Funcionou também contra o texto: números de produtividade sem fonte foram recusados, datas de prática sem memória verificável foram cortadas, frases inventadas para dar fluência à prosa foram eliminadas na revisão. O teste de autenticidade aplicado ao fim de cada capítulo não é burocracia de estilo: é o capítulo 16 aplicado ao livro.

E funcionou contra a linguagem: o manifesto de Kof foi confrontado princípio a princípio com o compilador, e a distância entre as promessas e os registros de decisão não foi escondida; foi publicada como dado. Uma linguagem que aceita ser lida pela própria constituição efetiva, com issues abertas e tabela de gaps datada, é mais confiável do que uma que só aparece em brochura. A aposta do capítulo 23 era essa, e ela não dependia de o Kof estar certo: dependia de ele estar aberto.

Fica o que o leitor leva, e aqui o livro tem uma última exigência, a mais difícil de todas: discordar. Não concordar comigo sobre nulabilidade, sobre arquitetura, sobre a escada de evidência, sobre as seis perguntas do caderno; usar. Cada critério deste livro é uma ferramenta com condições de uso nomeadas, e ferramenta que nunca falha na mão de ninguém é ferramenta que ninguém testou. Se você levou o teste de realidade para a sua reunião e ele derrubou uma decisão minha, o livro funcionou. Se as minhas frases circularem sem os argumentos que as produziram, se virarem regras, se o "o que esta estrutura diz?" virar porrete de revisão, o capítulo 4 venceu, e a derrota será minha, com data e sem culpado. É a assimetria correta para um livro

sobre responsabilidade: o texto é meu; o uso é seu; a verificação é de quem vier depois, que não me conhece e tem toda a razão de desconfiar.

A mesa de revisão fecha como abriu: com duas pessoas olhando a mesma mudança e enxergando coisas diferentes. A diferença é que agora as duas sabem o nome da pergunta que as separa, e nenhuma das duas vai responder por autoridade. É pouco, como filosofia. É tudo, como ofício.

Intenção em Código termina aqui. O diff, como sempre, continua aberto, e é para isso que ele serve.

Glossário

Os termos que este livro opera, definidos com as ressalvas que as medições impuseram. Cada entrada remete ao capítulo onde o conceito foi construído, o glossário é mapa, não substituto: um termo lido aqui sem o argumento do capítulo é exatamente a regra sem porquê que o capítulo 4 descreve.

Alucinação (como estado inválido sem guarda): o código gerado que compila, roda, parece idiomático e viola uma invariante que nenhum verificador conhecia; renomear assim desloca a discussão de "a máquina mente" para "ninguém designou um vigia", problema de arquitetura, não de sinceridade. *(caps. 12, 21)*

Camada (como aposta temporal): toda abstração intermediária declara implicitamente que o que ela esconde muda mais rápido que o que ela expõe; a auditoria da camada é nomear a mudança que ela absorve, sem resposta, é aluguel sem renda. *(cap. 18)*

Capital congelado: módulo intocável: funciona, ninguém altera, e cada dia sem mudança aumenta a conta que a primeira mudança cobrará; o nome contábil para o "não mexe nisso que funciona". *(caps. 3, 19)*

Casa do comportamento: as três localizações possíveis de uma regra, dentro do estado (método), ao lado (função sobre a hierarquia), acima (serviço/porta), escolhidas pela economia da mudança, não por tribo: o que muda mais dita onde a regra mora. *(cap. 11)*

Complexidade acidental: a dificuldade que as nossas escolhas de ferramenta, método e representação adicionam ao problema; não é xingamento nem pecado: quase todo ganho de engenharia é compra de acidental que elimina uma obrigação essencial de memória. O erro não é pagar; é pagar sem contabilizar. *(cap. 2)*

Complexidade concentrada × diluída: a dificuldade do domínio mora em poucos lugares nomeados (concentrada) ou está espalhada por duplicações, memória e exceções (diluída); a diferença decisiva não é o total, é a enumerabilidade do raio de mudança. *(cap. 18)*

Complexidade essencial: a que o problema impõe independentemente da implementação; não se elimina, nomeia-se, concentra-se e vigia-se. Confundi-la com acidental produz o modelo superficial; tratá-la como intocável produz o desleixo. *(cap. 2)*

Constituição declarada × efetiva: a primeira é o que a linguagem, framework ou processo promete em manifesto e documentação; a segunda é o conjunto exato de regras que a máquina aplica com código, coordenada e mensagem, ou deixa de aplicar em silêncio. Só a efetiva cobra; por isso é a única que merece confiança, calibrada por medição datada. *(caps. 8, 19, 23)*

Custo cognitivo: a fatura real de todo sistema, paga pela cabeça finita que compreende, navega e lembra; os três custos têm sintomas e remédios distintos, e confundi-los produz refatoração cara que não melhora nada. *(cap. 3)*

Decisão comprimida: toda convenção útil é um julgamento tomado uma vez para não ser tomado de novo; como todo arquivo comprimido, exige o descompressor, o contexto que a criou, e sem ele vira regra órfã. *(cap. 4)*

Depreciação (como disciplina): aposentadoria honesta de uma prática, interface ou privilégio: com dono nomeado, data com calendário, substituto apontado e o número que quase ninguém olha, consumidores ativos na semana do corte. Sem as quatro peças, é aviso; sem o número, é fé. *(caps. 14, 19, 22)*

Escada de evidência: os degraus de confiança disponível: o que o compilador prova, o que o teste unitário prova, o que o teste de contrato e integração prova, o que só a produção sabe. A falácia clássica é exigir do degrau de baixo a garantia do de cima, ou gastar o de cima pagando o aluguel do de baixo. *(cap. 16)*

Estado inválido: a situação que o domínio proíbe e o sistema permite existir; sempre denúncia de uma invariante sem vigia, e as três origens conhecidas são o mundo escrevendo por fora, o tempo mudando a regra debaixo do registro e a concorrência produzindo combinações que nenhuma transação isolada escreveria. *(caps. 12, 13)*

Exactly-once (como mito): a entrega exatamente-uma-vez é impossível sobre canal que pode falhar depois do envio e antes do reconhecimento; a resposta de engenharia é at-least-once com idempotência, e a idempotência exige que o domínio tenha dado identidade ao fato. *(cap. 13)*

Fire-and-forget (como decisão): disparar tarefa sem dono de resultado: uma escolha sobre responsabilidade, não uma economia de duas linhas; ou o resultado não importa para ninguém, ou alguém precisa ser o dono de descobrir que importava. *(cap. 15)*

Fronteira de confiança: todo ponto onde dados, chamadas ou autoridade cruzam entre domínios que não compartilham verificação; entrada hostil até que o contrário seja verificado, e cada fronteira não nomeada é uma decisão de confiança tomada por omissão. *(caps. 14, 18, 20)*

Geração sem memória: código produzido sem autor não carrega nem a memória imperfeita da intenção, não há quem perguntar "por quê"; a intenção precisa estar inteira na estrutura e nos testes, ou não estará em lugar nenhum. *(cap. 21)*

Igualdade por valor × por identidade: duas instâncias com os mesmos campos são o mesmo fato (valor) ou dois objetos distintos (identidade)? A escolha é do domínio, e a ressalva medida deste livro: no Kof 0.6.1, estados selados escritos na forma compacta perdem a igualdade por valor em silêncio (issue #818), até o fix, comparação campo a campo via acessores. *(caps. 5, 10; ressalva: validacao.md 12/10/2026)*

Intenção (em código): não o que o autor quis dizer, mas o que a estrutura torna recuperável por quem não estava lá; verificável em dois sentidos: contra o comportamento (testes) e contra a representação (estrutura). Vive no que a estrutura impede, no que separa ou compartilha e só então no que nomeia, nessa ordem de confiança. *(cap. 1)*

Intenção em código (a tese): código de qualidade é código cuja estrutura expressa a intenção do sistema com o mínimo de complexidade acidental possível; três variáveis independentes, cada uma território de

engenharia com instrumentos próprios, e a disciplina de fazer as três perguntas ao mesmo tempo. *(caps. 1, 24)*

Linguagem ubíqua: o vocabulário que especialistas do domínio e código compartilham, sem tradução entre eles; a estrutura e a linguagem do código devem corresponder ao domínio (Evans, *Domain-Driven Design*, 2003, ver nota de verificação na bibliografia). O teste do termo: uma palavra do negócio que exige dois nomes no código indica dois significados que alguém ainda não separou. *(cap. 10)*

Medição com data e artefato: a única unidade de documentação de evolução que não mente: o que foi medido, em qual versão de qual ferramenta, em que dia. Um número sem artefato é rumor; uma promessa sem data de re-verificação é brochura. *(caps. 17, 19)*

Micro-linguagem: toda escolha de stack, guia de estilo, template de pull request ou convenção interna legisla o que a equipe consegue dizer, com os mesmos custos e a mesma responsabilidade de quem legisla uma linguagem de verdade. *(cap. 23)*

Modelo anêmico (por abandono × por escolha): dados puros com comportamento ao lado: abandono quando o fechamento mora na memória da equipe (doze `if` sem exaustão); escolha quando o fechamento está no tipo (sealed, switches exaustivos) e as operações são estáveis com interpretações múltiplas. Forma idêntica, engenharia oposta. *(caps. 10, 11)*

Outbox: gravar o evento "o estado mudou" na mesma transação do estado, aceitando publicar depois com repetição segura; a confissão elegante de que a transação global não existe, transferindo atomicidade para dentro da fronteira que a tem. *(cap. 13)*

Posição de veto: o ponto onde a mudança se torna fato, e seu detentor: o dono do sistema é quem pode dizer "não passa" e diz. Não decorre de autoria, decorre de posição, e a delegação (a framework, fornecedor, IA) move a execução, nunca a posição. *(cap. 22)*

Raio de mudança enumerável: a propriedade que separa arquitetura honesta de brochura: quando o mundo muda, os lugares a tocar são listáveis por um verificador (o tipo, o teste, a CI) em vez de depender de quem lembra. O `sealed` com SEM081 é o caso em escala de arquivo. *(caps. 1, 18)*

Recusa honesta (como critério): a obrigação de dizer o que não se sabe, o que não se mediu e o que não se pode fazer, aplicada a ferramentas, a números, a citações e a livros sobre o próprio assunto. Um sistema que não pode recusar não é confiável: é semântica vazia com uptime. *(caps. 14, 20, 24)*

Refatoração (como movimento de complexidade): redistribuir dificuldade sem mover o contrato; a liberdade de fazê-lo é exatamente a espessura da evidência que envolve o comportamento, código impossível de refatorar é, em última análise, dívida de evidência, não de código. *(caps. 2, 16, 18, 19)*

Segurança (como propriedade do projeto): não componente instalável nem lista de verificações: a qualidade que emerge (ou não) da distribuição inteira de confiança, superfícies e tempo; a pergunta bem-formada tem três espaços, contra quem, proteger o quê, com que orçamento. *(cap. 20)*

Unidade de integridade: o que deve ser verdade de uma vez, definido pelo domínio; a transação de banco é a tentativa de dar sombra a essa unidade, e a sombra só coincide com o corpo quando alguém desenhou o corpo primeiro. *(cap. 13)*

Unidade de serialização: o ponto onde se decide a ordem entre operações que disputam o mesmo estado: uma fila, um canal, um commit, uma esperança. Dois donos sem unidade declarada é o monolito distribuído, o imposto da rede pago com a consistência dividida. *(cap. 15)*